

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG ĐA ECU GIAO TIẾP
CAN MỨC THANH GHI THEO KIẾN TRÚC
AUTOSAR CLASSIC THU GỌN**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **NGUYỄN HỮU HẢO**

MSSV: 22161250

DƯƠNG NHÃ KHANG

MSSV: 22161265

TP. HỒ CHÍ MINH, 6/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG ĐA ECU GIAO TIẾP
CAN MỨC THANH GHI THEO KIẾN TRÚC
AUTOSAR CLASSIC THU GỌN**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **NGUYỄN HỮU HẢO**

MSSV: 22161250

DƯƠNG NHÃ KHANG

MSSV: 22161265

Hướng dẫn: **ThS. ĐẶNG PHƯỚC HẢI TRANG**

TP. HỒ CHÍ MINH, 6/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Nguyễn Hữu Hào	MSSV: 22161250
Email: 22161250@student.hcmute.edu.vn	Điện thoại: 0369407251
Họ và tên sinh viên: Dương Nhã Khang	MSSV: 22161265
Email: 22161265@student.hcmute.edu.vn	Điện thoại: 0795844183

2. Thông tin đề tài

- Tên đề tài: Thiết kế hệ thống đa ECU giao tiếp CAN mức thanh ghi theo kiến trúc AUTOSAR Classic thu gọn.
- Đơn vị quản lý: Bộ môn Kỹ Thuật Máy Tính – Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Tp. Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày 2/3/2026 đến ngày 13/6/2026
- Thời gian bảo vệ trước hội đồng: Ngày / / 20..

3. Lời cam đoan của sinh viên

Tôi Nguyễn Hữu Hào, Dương Nhã Khang cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Thạc sĩ Đặng Phước Hải Trang. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...

SV thực hiện đồ án
(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Tp. HCM, ngày ... tháng ... năm 20..

Xác nhận của Bộ Môn

Giảng viên hướng dẫn
(Ký, ghi rõ họ tên và học hàm – học vị)

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

(Theo mẫu của Bộ môn)

BẢN NHẬN XÉT CỦA GIÁO VIÊN PHẢN BIỆN

(Theo mẫu của Bộ môn)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế hệ thống đa ECU giao tiếp CAN mức thành phần

Kiểu mẫu: KTUN.SAR Classic thu gọn

Sinh viên: + Nguyễn Khắc Hào

MSSV: 22161252

+ Đường Nhữ Khang

MSSV: 22161255

Hướng dẫn: Thạc Sĩ. Đặng Phước Hải Trang

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể, đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng
Đề tài có tính ứng dụng

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thiết kế hợp lý
Giải quyết vấn đề

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài, phân tích và đánh giá/ kiểm định thiết kế hợp lý; có minh chứng kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả đạt yêu cầu

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp một số đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp

5. Hình thức trình bày và bố cục báo cáo:

Văn phong mạch lạc, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tác văn bản trừu tượng.

Trình bày rõ ràng, logic

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Sinh viên có khả năng nghiên cứu và làm việc nhóm

7. Tài liệu trích dẫn:

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Trích dẫn tài liệu tham khảo đúng quy định

8. Đánh giá về sự trung thực của đề tài:

Cần không đánh giá sai có trùng lặp hay không? Nếu có, đó ghi rõ ra mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Mức độ trung thực đạt yêu cầu

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép không được phép bảo vệ khóa luận tốt nghiệp".

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp

Tp. HCM, ngày ... tháng ... năm 20...

Người nhận xét

(Ký và ghi rõ họ tên)

Đặng Phước Hải Trang

Đặng Phước Hải Trang

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trước hết, nhóm em xin gửi lời cảm ơn chân thành đến quý Thầy/Cô Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh, đặc biệt là quý Thầy/Cô Khoa Điện - Điện tử và Bộ môn Kỹ thuật Điện tử - Thông tin, đã tận tình giảng dạy, truyền đạt kiến thức chuyên môn và tạo điều kiện thuận lợi trong suốt quá trình học tập tại trường. Những kiến thức nền tảng về điện tử, viễn thông, vi mạch, và hệ thống nhúng là cơ sở quan trọng giúp nhóm em thực hiện đề tài khóa luận tốt nghiệp này.

Nhóm em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Đặng Phước Hải Trang, người đã trực tiếp hướng dẫn, định hướng và góp ý trong quá trình thực hiện đề tài “Thiết kế hệ thống đa ECU giao tiếp CAN mức thanh ghi theo kiến trúc AUTOSAR Classic thu gọn”. Sự hướng dẫn tận tình của thầy đã giúp chúng em có thêm kiến thức về hệ thống nhúng ô tô, mạng truyền thông CAN và kiến trúc AUTOSAR Classic. Thông qua việc thiết kế hệ thống đa ECU và lập trình mức thanh ghi, sinh viên hiểu được cách tổ chức phần mềm ECU, cơ chế giao tiếp giữa các bộ điều khiển và quy trình phát triển phần mềm nhúng theo định hướng công nghiệp ô tô.

Mặc dù đã cố gắng trong quá trình thực hiện, khóa luận không thể tránh khỏi những thiếu sót do giới hạn về thời gian, kinh nghiệm nghiên cứu và điều kiện thực nghiệm. Chúng em kính mong nhận được những ý kiến đóng góp quý báu từ quý Thầy/Cô để đề tài được hoàn thiện hơn trong tương lai.

TÓM TẮT

Trong những năm gần đây, các hệ thống điều khiển điện tử trên ô tô ngày càng phát triển theo xu hướng phân tán chức năng thành nhiều bộ điều khiển điện tử (ECU) giao tiếp với nhau thông qua mạng truyền thông CAN. Bên cạnh đó, kiến trúc phần mềm AUTOSAR Classic được sử dụng rộng rãi nhằm chuẩn hóa thiết kế, tăng khả năng tái sử dụng và mở rộng hệ thống.

Đề tài “Thiết kế hệ thống đa ECU giao tiếp CAN mức thanh ghi theo kiến trúc AUTOSAR Classic thu gọn” tập trung nghiên cứu, thiết kế và xây dựng mô hình gồm ba ECU: Main ECU, Airbag ECU và ABS ECU. Hệ thống được phát triển trên vi điều khiển STM32, sử dụng giao thức CAN để trao đổi dữ liệu giữa các ECU. Phần mềm được xây dựng theo mô hình phân lớp của AUTOSAR Classic thu gọn, bao gồm các lớp quản lý ECU, dịch vụ hệ thống, giao diện truyền thông và lớp ứng dụng.

Phương pháp thực hiện bao gồm nghiên cứu kiến trúc AUTOSAR Classic, giao thức truyền thông CAN và lập trình vi điều khiển ở mức thanh ghi. Trên cơ sở đó, các module phần mềm được thiết kế, hiện thực và tích hợp để mô phỏng hoạt động của hệ thống đa ECU. Quá trình kiểm thử được thực hiện thông qua việc truyền nhận bản tin CAN giữa các ECU và giám sát dữ liệu trên hệ thống.

Kết quả thực hiện cho thấy hệ thống hoạt động ổn định, các ECU có thể khởi tạo, trao đổi dữ liệu và xử lý chức năng theo đúng thiết kế. Đề tài góp phần củng cố kiến thức về hệ thống nhúng, mạng CAN, lập trình mức thanh ghi và kiến trúc AUTOSAR Classic, đồng thời tạo nền tảng cho việc nghiên cứu và phát triển các hệ thống điều khiển điện tử trong lĩnh vực ô tô.

MỤC LỤC

DANH MỤC HÌNH.....	IX
DANH MỤC BẢNG.....	XII
CÁC TỪ VIẾT TẮT.....	XIV
CHƯƠNG 1 TỔNG QUAN.....	16
1.1 BỐI CẢNH VÀ TÍNH CẤP THIẾT CỦA ĐỀ TÀI.....	16
1.2 TỔNG QUAN NGHIÊN CỨU VÀ CÁC HƯỚNG TIẾP CẬN.....	17
1.3 VẤN ĐỀ NGHIÊN CỨU, MỤC ĐÍCH VÀ MỤC TIÊU.....	19
1.3.1 Vấn đề nghiên cứu.....	19
1.3.2 Mục đích nghiên cứu.....	19
1.3.3 Mục tiêu nghiên cứu.....	20
1.4 GIẢ THIẾT NGHIÊN CỨU VÀ GIỚI HẠN ĐỀ TÀI.....	22
1.4.1 Giả thiết nghiên cứu.....	22
1.4.2 Giới hạn đề tài.....	23
1.5 PHƯƠNG PHÁP NGHIÊN CỨU.....	24
1.6 BỐ CỤC ĐỒ ÁN.....	27
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	28
2.1 GIAO THỨC CAN.....	28
2.1.1 Tổng quan về mạng CAN (Controller Area Network).....	28
2.1.2 Cấu trúc khung dữ liệu trong CAN.....	30
2.1.3 Cơ chế phát hiện lỗi trong mạng CAN.....	32
2.1.4 Cơ chế xử lý lỗi và fault confinement.....	35
2.1.5 Thanh ghi và cơ chế xử lý lỗi CAN trên vi điều khiển STM32.....	37
2.2 TỔNG QUAN VỀ AUTOSAR CLASSIC.....	39
2.2.1 Khái niệm.....	39
2.2.2 Kiến trúc phân lớp AUTOSAR Classic.....	40
2.2.3 AUTOSAR Classic thu gọn.....	47
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG.....	48
3.1 YÊU CẦU CỦA HỆ THỐNG.....	48

3.2	MÔ HÌNH HỆ THỐNG.....	49
3.2.1	Mô hình tổng thể của hệ thống.....	49
3.2.2	Sơ đồ khối hệ thống.....	50
3.2.3	Chức năng các khối trong hệ thống.....	51
3.2.4	Nguyên lý hoạt động của hệ thống.....	51
3.3	THIẾT KẾ PHẦN CỨNG.....	52
3.3.1	Chức năng của phần cứng.....	52
3.3.2	Sơ đồ khối phần cứng.....	53
3.3.3	Thiết kế từng khối.....	55
3.3.4	Thiết kế mạch điều khiển.....	68
3.4	THIẾT KẾ PHẦN MỀM.....	69
3.4.1	Mục tiêu thiết kế phần mềm.....	70
3.4.2	Cấu trúc hệ thống phần mềm.....	71
3.4.3	Thiết kế phần mềm Main ECU.....	74
3.4.4	Thiết kế phần mềm ABS ECU.....	75
3.4.5	Thiết kế phần mềm Airbag ECU.....	80
3.4.6	Thiết kế truyền thông CAN.....	82
3.4.7	Thiết kế cơ chế Fault Confinement.....	86
3.4.8	Kết luận chương.....	89
CHƯƠNG 4	KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.....	90
4.1	KẾT QUẢ MÔ HÌNH THI CÔNG.....	90
4.2	KẾT QUẢ MÔ PHỎNG.....	90
4.2.1	Mục tiêu thực nghiệm.....	90
4.2.2	Phương pháp thu thập dữ liệu thực nghiệm.....	91
4.3	QUY ƯỚC DỮ LIỆU SỬ DỤNG TRONG THỰC NGHIỆM.....	92
4.3.1	Quy ước CAN ID.....	92
4.3.2	Quy ước payload frame Main đến ABS.....	93
4.3.3	Quy ước payload frame ABS đến Main.....	93
4.3.4	Quy ước payload frame Airbag đến Main.....	94
4.3.5	Quy ước payload frame diagnostic CAN.....	94

4.4	KẾT QUẢ THỰC NGHIỆM.....	96
4.4.1	Thực nghiệm 1: Kiểm tra khởi động hệ thống và trạng thái CAN bình thường.....	96
4.4.2	Thực nghiệm 2: Truyền lệnh từ ECU Main đến ECU ABS.....	97
4.4.3	Thực nghiệm 3: Điều khiển động cơ trên ECU ABS.....	99
4.4.4	Thực nghiệm 4: Phản hồi trạng thái từ ECU ABS về ECU Main.....	102
4.4.5	Thực nghiệm 5: Phát hiện va chạm trên ECU Airbag.....	105
4.4.6	Thực nghiệm 6: Hệ thống đa ECU hoạt động đồng thời.....	108
4.4.7	Thực nghiệm 7: Giám sát lỗi ACK.....	111
4.4.8	Thực nghiệm 8: Lỗi đường truyền CAN.....	113
4.4.9	Thực nghiệm 9: Timeout và phản ứng an toàn của ECU ABS.....	116
4.5	KẾT QUẢ THỰC NGHIỆM TỔNG HỢP.....	117
CHƯƠNG 5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	120
5.1	KẾT LUẬN.....	120
5.2	HƯỚNG PHÁT TRIỂN.....	122
	PHỤ LỤC.....	124
	TÀI LIỆU THAM KHẢO.....	125

DANH MỤC HÌNH

Hình 2.1 Mô hình kết nối nhiều ECU trên mạng CAN Bus.....	28
Hình 2.2 Cơ chế phân xử ưu tiên trên mạng CAN Bus.....	29
Hình 2.3 Nguyên lý truyền tín hiệu vi sai CAN_H và CAN_L.....	29
Hình 2.4 Cấu trúc khung dữ liệu chuẩn CAN 2.0A.....	30
Hình 2.5 Nguyên lý phát hiện lỗi bằng cơ chế Bit Monitoring.....	32
Hình 2.6 Minh họa cơ chế Bit Stuffing trong CAN.....	33
Hình 2.7 Nguyên lý kiểm tra lỗi CRC trong CAN.....	33
Hình 2.8 Cơ chế xác nhận ACK trong mạng CAN.....	34
Hình 2.9 Cấu trúc Error Frame trong giao thức CAN.....	35
Hình 2.10 Nguyên lý hoạt động của bộ đếm lỗi TEC và REC trong CAN.....	36
Hình 2.11 Thanh ghi CAN_ESR trong bộ điều khiển bxCAN của STM32.....	38
Hình 2.12 So sánh mô hình phát triển phần mềm truyền thống và kiến trúc AUTOSAR.....	40
Hình 2.13 Kiến trúc phân lớp AUTOSAR Classic Platform.....	41
Hình 2.14 Cơ chế truyền dữ liệu thông qua RTE.....	43
Hình 2.15 Cấu trúc lớp Basic Software trong AUTOSAR Classic.....	44
Hình 2.16 Ngăn xếp truyền thông CAN trong AUTOSAR.....	45
Hình 3.1 Sơ đồ khối tổng quát hệ thống.....	50
Hình 3.2 Lưu đồ giải thuật của hệ thống.....	52
Hình 3.3 Sơ đồ khối phần cứng chi tiết.....	54
Hình 3.4 Board STM32F407 Discovery.....	56
Hình 3.5 Module MCU230 SN65HVD230.....	58
Hình 3.6 Module SN65HVD230 sử dụng trên ABS ECU và Airbag ECU.....	59
Hình 3.7 Board STM32F103C8T6.....	60
Hình 3.8 Cảm biến LIS331HH.....	62
Hình 3.9 Động cơ giảm tốc GA25-370 tích hợp encoder.....	64
Hình 3.10 Mạch điều khiển động cơ XY-160D.....	65

Hình 3.11 Màn hình LCD 16x2 I2C.....	66
Hình 3.12 Nút nhấn.....	67
Hình 3.13 Biến trở 10 k Ω	68
Hình 3.14 Phần cứng hệ thống.....	69
Hình 3.15 Kiến trúc phần mềm tổng thể của hệ thống.....	71
Hình 3.16 Lưu đồ thuật toán của Main ECU.....	84
Hình 3.17 Lưu đồ thuật toán ABS ECU.....	69
Hình 3.18 Lưu đồ thuật toán của Airbag ECU.....	81
Hình 3.19 Kiến trúc truyền thông CAN của hệ thống.....	83
Hình 3.20 Sơ đồ trạng thái Fault Confinement.....	81
Hình 4.1 Mô hình hoàn thiện.....	90
Hình 4.2 Log khởi động ECU Main.....	96
Hình 4.3 Log khởi động ECU ABS.....	97
Hình 4.4 Log khởi động ECU Airbag.....	97
Hình 4.5 MAIN gửi frame 0x121.....	98
Hình 4.6 Trạng thái động khi điều chỉnh ADC về 0.....	101
Hình 4.7 Trạng thái động khi điều chỉnh biến trở ADC.....	102
Hình 4.8 Frame 0x122 từ ECU ABS về ECU Main trên USB-CAN tool.....	103
Hình 4.9 Log UART ECU ABS khi gửi frame phản hồi 0x122.....	103
Hình 4.10 Log UART Main khi nhận frame phản hồi 0x122 từ ABS.....	103
Hình 4.11 Log UART Main khi nhận trạng thái ABS thay đổi chiều quay.....	103
Hình 4.12 Log UART ECU Main khi nhận trạng thái ABS dừng motor.....	104
Hình 4.13 Airbag ở trạng thái bình thường.....	106
Hình 4.14 Airbag ở trạng thái phát hiện va chạm.....	107
Hình 4.15 Log Airbag khi phát hiện va chạm.....	107
Hình 4.16 Log Main nhận cảnh báo Airbag.....	96
Hình 4.17 Màn hình LCD hiển thị cảnh báo va chạm.....	97
Hình 4.18 Các frame được truyền trên USB-CAN	97
Hình 4.19 Log ECU Airbag khi xảy ra ACK Error.....	110
Hình 4.20 Log ECU Airbag trong quá trình phục hồi sau lỗi ACK.....	112

Hình 4.21 Log ECU Airbag sau khi TEC trở về 0 và trạng thái CAN phục hồi bình thường.....	112
Hình 4.22 Log diagnostic khi rút bỏ node CAN Airbag khỏi bus.....	115
Hình 4.24 Log diagnostic khi xuất hiện Form Error do tác động nguồn điện.....	116

DANH MỤC BẢNG

Bảng 2.1 Chức năng các trường trong CAN Frame.....	31
Bảng 2.2 Các cơ chế phát hiện lỗi trong CAN.....	35
Bảng 2.3 Trạng thái Fault Confinement trong CAN.....	37
Bảng 2.4 Các thanh ghi CAN quan trọng trên STM32.....	39
Bảng 3.1 So sánh các vi điều khiển.....	56
Bảng 3.2 Thông số STM32F407VG.....	57
Bảng 3.3 So sánh các transceiver CAN.....	58
Bảng 3.4 Thông số kỹ thuật module MCU230 SN65HVD230.....	59
Bảng 3.5 Thông số chính của module SN65HVD230.....	61
Bảng 3.6 Thông số kỹ thuật board STM32F103C8T6.....	67
Bảng 3.7 So sánh các cảm biến gia tốc.....	62
Bảng 3.8 Thông số kỹ thuật của LIS331HH.....	63
Bảng 3.9 So sánh các loại động cơ.....	64
Bảng 3.10 Thông số kỹ thuật của động cơ GA25-370.....	65
Bảng 3.11 Thông số chính của XY-160D.....	66
Bảng 3.12 Thông số chính của LCD 16x2 I2C.....	64
Bảng 3.13 Trạng thái điều khiển động cơ.....	83
Bảng 3.14 Danh sách CAN ID sử dụng trong hệ thống.....	83
Bảng 3.15 Cấu trúc bản tin CAN ID 0x121.....	83
Bảng 3.16 Cấu trúc bản tin CAN ID 0x122.....	84
Bảng 3.17 Cấu trúc bản tin CAN ID 0x111.....	84
Bảng 3.18 Một số giá trị LEC thường gặp.....	87
Bảng 4.1 Các đối tượng cần kiểm tra.....	91
Bảng 4.2 Danh sách CAN ID sử dụng trong thực nghiệm.....	92
Bảng 4.3 Cấu trúc frame 0x121 Main đến ABS.....	93
Bảng 4.4 Cấu trúc frame 0x122 ABS → Main.....	93
Bảng 4.5 Quy ước trạng thái motor trong frame 0x122.....	94

Bảng 4.6 Cấu trúc frame 0x111 Airbag đến Main.....	94
Bảng 4.7 Quy ước trạng thái DEPLOY.....	94
Bảng 4.8 Cấu trúc frame diagnostic CAN.....	94
Bảng 4.9 Quy ước mã LEC.....	95
Bảng 4.10 Quy ước ErrorState và ControllerMode.....	95
Bảng 4.11 Kết quả khởi động hệ thống.....	96
Bảng 4.12 Sự thay đổi của ADC và PWM trên ECU ABS.....	98
Bảng 4.13 Kết quả thực nghiệm Airbag.....	105
Bảng 4.14 Chuỗi hoạt động đa ECU trong một chu kỳ thực nghiệm.....	109
Bảng 4.15 Kết quả thực nghiệm lỗi ACK.....	111
Bảng 4.16 Kết quả thực nghiệm lỗi đường truyền CAN.....	113
Bảng 4.17 Kết quả thực nghiệm timeout ABS.....	116

CÁC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
ABS	Anti-lock Braking System
ACK	Acknowledgement
ADC	Analog to Digital Converter
AUTOSAR	AUTomotive Open System ARchitecture
BOFF	Bus Off
BSW	Basic Software
bxCAN	Basic Extended CAN Controller
CAN	Controller Area Network
CANH	CAN High
CANL	CAN Low
CRC	Cyclic Redundancy Check
DLC	Data Length Code
ECU	Electronic Control Unit
EOF	End of Frame
EPVF	Error Passive Flag
EWGF	Error Warning Flag
GPIO	General Purpose Input Output
I2C	Inter-Integrated Circuit
IDE	Identifier Extension Bit
LCD	Liquid Crystal Display
LEC	Last Error Code
MCAL	Microcontroller Abstraction Layer
MCU	Microcontroller Unit
PWM	Pulse Width Modulation
RAM	Random Access Memory

REC	Receive Error Counter
RPM	Revolutions Per Minute
RTE	Runtime Environment
RX	Receive
SOF	Start of Frame
SPI	Serial Peripheral Interface
STM32	STMicroelectronics 32-bit Microcontroller
TEC	Transmit Error Counter
TIM	Timer
TX	Transmit
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus

CHƯƠNG 1

TỔNG QUAN

1.1 BỐI CẢNH VÀ TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Trong những năm gần đây, ngành công nghiệp ô tô đang trải qua quá trình chuyển đổi mạnh mẽ với sự phát triển của các hệ thống điện tử và phần mềm nhúng. Trên các phương tiện hiện đại, số lượng bộ điều khiển điện tử (Electronic Control Unit – ECU) ngày càng gia tăng nhằm thực hiện các chức năng như điều khiển động cơ, chống bó cứng phanh (ABS), túi khí an toàn (Airbag), điều khiển thân xe và các hệ thống hỗ trợ lái nâng cao (ADAS). Theo nghiên cứu của Navarro và cộng sự [1], một chiếc ô tô hiện đại có thể tích hợp từ 50 đến hơn 100 ECU kết nối với nhau thông qua các mạng truyền thông trong xe.

Để đảm bảo khả năng trao đổi dữ liệu giữa các ECU với độ tin cậy cao và chi phí thấp, mạng truyền thông CAN (Controller Area Network) được phát triển bởi Bosch và nhanh chóng trở thành chuẩn giao tiếp phổ biến nhất trong lĩnh vực ô tô [2]. CAN Bus cho phép nhiều ECU cùng chia sẻ một đường truyền chung, hỗ trợ cơ chế ưu tiên bản tin, phát hiện lỗi và truyền dữ liệu thời gian thực. Nhờ những ưu điểm đó, CAN hiện vẫn được sử dụng rộng rãi trên hầu hết các phương tiện giao thông và hệ thống công nghiệp.

Cùng với sự gia tăng số lượng ECU, độ phức tạp của phần mềm nhúng trong ô tô cũng tăng lên đáng kể. Việc phát triển phần mềm riêng lẻ cho từng ECU dẫn đến nhiều khó khăn trong quá trình tích hợp, bảo trì và tái sử dụng. Để giải quyết vấn đề này, tổ chức AUTOSAR (AUTomotive Open System ARchitecture) được thành lập vào năm 2003 nhằm xây dựng một kiến trúc phần mềm chuẩn hóa cho ngành công nghiệp ô tô [3]. AUTOSAR cung cấp các lớp phần mềm tiêu chuẩn, tách biệt ứng dụng với phần cứng, giúp tăng khả năng tái sử dụng mã nguồn và giảm thời gian phát triển sản phẩm.

Theo báo cáo của AUTOSAR Consortium [4], phần lớn các nhà sản xuất ô tô lớn như BMW, Mercedes-Benz, Volkswagen, Ford, Toyota và General Motors đều áp dụng AUTOSAR trong các nền tảng điều khiển điện tử hiện đại. Điều này

cho thấy AUTOSAR đã trở thành một trong những tiêu chuẩn quan trọng của ngành công nghiệp ô tô.

Tuy nhiên, tại các cơ sở đào tạo đại học, việc tiếp cận và nghiên cứu AUTOSAR vẫn còn nhiều hạn chế do yêu cầu phần cứng chuyên dụng, chi phí bản quyền phần mềm cao và độ phức tạp của hệ thống thực tế. Phần lớn các mô hình thực hành chỉ dừng lại ở việc giao tiếp CAN cơ bản hoặc điều khiển một ECU đơn lẻ, chưa phản ánh được kiến trúc đa ECU đang được sử dụng trên xe hiện đại [5].

Bên cạnh đó, các hệ thống an toàn chủ động như ABS và Airbag đều yêu cầu khả năng trao đổi dữ liệu thời gian thực giữa nhiều ECU khác nhau. Các nghiên cứu của Kumar và cộng sự [6] cho thấy việc xây dựng mô hình thực nghiệm đa ECU giúp sinh viên và kỹ sư hiểu rõ hơn về cơ chế truyền thông CAN, quản lý lỗi mạng và kiến trúc phần mềm trên ô tô hiện đại.

Xuất phát từ những yêu cầu thực tiễn nêu trên, đề tài “Thiết kế hệ thống đa ECU giao tiếp CAN mức thanh ghi theo kiến trúc AUTOSAR Classic thu gọn” được thực hiện nhằm xây dựng một mô hình thực nghiệm gồm ba ECU: Main ECU, ABS ECU và Airbag ECU. Hệ thống sử dụng các vi điều khiển STM32F407 và STM32F103 kết nối với nhau thông qua mạng CAN Bus, đồng thời áp dụng tư tưởng phân lớp của AUTOSAR Classic trong quá trình thiết kế phần mềm. Đề tài không chỉ giúp nghiên cứu nguyên lý hoạt động của mạng CAN và kiến trúc AUTOSAR mà còn tạo nền tảng cho việc phát triển các hệ thống điều khiển điện tử ô tô trong tương lai.

1.2 TỔNG QUAN NGHIÊN CỨU VÀ CÁC HƯỚNG TIẾP CẬN

Trong những năm gần đây, cùng với sự phát triển của ngành công nghiệp ô tô, các hệ thống điều khiển điện tử trên xe ngày càng trở nên phức tạp. Thay vì sử dụng một bộ điều khiển trung tâm duy nhất, các chức năng trên xe được phân chia cho nhiều bộ điều khiển điện tử (Electronic Control Unit - ECU) khác nhau nhằm nâng cao hiệu năng xử lý, khả năng mở rộng và độ tin cậy của hệ thống [7].

Các nghiên cứu trong lĩnh vực điện tử ô tô cho thấy số lượng ECU trên một phương tiện hiện đại có thể dao động từ vài chục đến hơn một trăm ECU tùy theo mức độ trang bị của xe [8]. Các ECU này đảm nhận nhiều chức năng khác nhau như điều khiển động cơ (Engine ECU), chống bó cứng phanh (ABS ECU), túi khí an toàn (Airbag ECU), điều khiển thân xe (Body ECU), điều khiển hộp số (TCU) và các hệ thống hỗ trợ lái nâng cao (ADAS).

Để các ECU có thể trao đổi dữ liệu với nhau một cách hiệu quả, nhiều chuẩn truyền thông trong xe đã được phát triển như LIN, CAN, FlexRay và Ethernet Automotive. Trong đó, CAN Bus được xem là chuẩn truyền thông phổ biến nhất nhờ chi phí thấp, khả năng chống nhiễu tốt và hỗ trợ truyền dữ liệu thời gian thực [2]. CAN Bus sử dụng cơ chế truyền thông quảng bá (broadcast), cho phép nhiều ECU cùng chia sẻ một đường truyền vật lý và thực hiện cơ chế phân xử ưu tiên dựa trên CAN Identifier [9].

Một hướng tiếp cận phổ biến trong nghiên cứu và đào tạo hiện nay là xây dựng các mô hình đa ECU sử dụng CAN Bus nhằm mô phỏng hoạt động của hệ thống điện tử trên ô tô. Các nghiên cứu của Kumar và cộng sự [6] đã chỉ ra rằng việc triển khai mô hình thực nghiệm đa ECU giúp đánh giá hiệu quả truyền thông thời gian thực, khả năng phát hiện lỗi và tính ổn định của mạng CAN trong các ứng dụng ô tô.

Bên cạnh truyền thông CAN, kiến trúc phần mềm cũng là một nội dung được quan tâm trong các hệ thống điều khiển hiện đại. Khi số lượng ECU ngày càng tăng, việc phát triển phần mềm riêng biệt cho từng ECU dẫn đến nhiều khó khăn trong quá trình tích hợp và bảo trì hệ thống. Để giải quyết vấn đề này, kiến trúc AUTOSAR (Automotive Open System ARchitecture) đã được phát triển nhằm chuẩn hóa cấu trúc phần mềm trên các ECU ô tô [3].

AUTOSAR phân chia phần mềm thành nhiều lớp chức năng bao gồm Application Layer, Runtime Environment (RTE) và Basic Software (BSW), giúp tách biệt phần mềm ứng dụng khỏi phần cứng và tăng khả năng tái sử dụng mã nguồn [5]. Mặc dù AUTOSAR hiện được áp dụng rộng rãi trong công nghiệp ô tô, việc triển khai đầy đủ AUTOSAR đòi hỏi phần mềm chuyên dụng và chi phí phát triển lớn. Do đó, trong các mô hình nghiên cứu và đào tạo, nhiều tác giả lựa chọn phương pháp xây dựng kiến trúc AUTOSAR thu gọn nhằm giữ lại các nguyên tắc thiết kế cốt lõi nhưng giảm độ phức tạp của hệ thống [10].

Ngoài ra, các hệ thống an toàn như ABS và Airbag là những đối tượng nghiên cứu tiêu biểu trong lĩnh vực điện tử ô tô. ABS ECU sử dụng tín hiệu tốc độ bánh xe để điều khiển lực phanh nhằm tránh hiện tượng bó cứng bánh xe khi phanh gấp [11]. Trong khi đó, Airbag ECU sử dụng dữ liệu từ cảm biến gia tốc để phát hiện va chạm và kích hoạt cơ cấu bảo vệ trong thời gian rất ngắn [12]. Việc xây dựng mô hình tích hợp ABS ECU và Airbag ECU trên cùng mạng CAN cho phép mô phỏng tương đối đầy đủ cơ chế hoạt động của hệ thống điện tử trên ô tô hiện đại.

Xuất phát từ các hướng nghiên cứu nêu trên, đề tài lựa chọn cách tiếp cận xây dựng hệ thống đa ECU gồm Main ECU, ABS ECU và Airbag ECU sử dụng mạng truyền thông CAN Bus. Phần mềm được thiết kế theo tư tưởng phân lớp

của AUTOSAR Classic thu gọn và được lập trình trực tiếp ở mức thanh ghi trên vi điều khiển STM32. Cách tiếp cận này vừa giúp nghiên cứu nguyên lý hoạt động của mạng CAN và AUTOSAR, vừa tạo môi trường thực nghiệm phục vụ đào tạo và nghiên cứu các hệ thống điện tử ô tô.

1.3 VẤN ĐỀ NGHIÊN CỨU, MỤC ĐÍCH VÀ MỤC TIÊU

1.3.1 Vấn đề nghiên cứu

Trong các phương tiện giao thông hiện đại, số lượng các bộ điều khiển điện tử ECU (Electronic Control Unit) ngày càng gia tăng nhằm thực hiện các chức năng chuyên biệt như điều khiển động cơ, hệ thống chống bó cứng phanh (ABS), túi khí an toàn (Airbag), hệ thống thân xe và nhiều chức năng hỗ trợ lái khác. Để các ECU có thể phối hợp hoạt động hiệu quả, việc trao đổi dữ liệu giữa các bộ điều khiển phải được thực hiện một cách nhanh chóng, chính xác và có độ tin cậy cao.

Hiện nay, mạng truyền thông CAN Bus (Controller Area Network) là chuẩn truyền thông được sử dụng rộng rãi trong ngành công nghiệp ô tô nhờ khả năng truyền dữ liệu thời gian thực, chống nhiễu tốt, chi phí triển khai thấp và độ tin cậy cao. Bên cạnh đó, kiến trúc phần mềm AUTOSAR (AUTomotive Open System ARchitecture) cũng được các hãng ô tô áp dụng nhằm chuẩn hóa phần mềm ECU, tăng khả năng tái sử dụng và mở rộng hệ thống.

Tuy nhiên, trong môi trường đào tạo và nghiên cứu, phần lớn các mô hình CAN Bus chỉ dừng lại ở mức truyền nhận dữ liệu giữa hai vi điều khiển đơn lẻ, chưa thể hiện được cấu trúc đa ECU và cơ chế phối hợp chức năng như trên xe thực tế. Đồng thời, việc tiếp cận AUTOSAR thường gặp khó khăn do hệ thống có độ phức tạp cao, yêu cầu phần cứng và phần mềm chuyên dụng.

Xuất phát từ thực tế đó, việc xây dựng một mô hình đa ECU sử dụng mạng CAN Bus và áp dụng kiến trúc AUTOSAR Classic thu gọn là cần thiết nhằm mô phỏng các chức năng cơ bản của hệ thống điện – điện tử trên ô tô, đồng thời tạo môi trường thực nghiệm phục vụ nghiên cứu, học tập và đánh giá hoạt động của mạng CAN Bus trong các hệ thống nhúng thời gian thực.

1.3.2 Mục đích nghiên cứu

Mục đích của đề tài là nghiên cứu nguyên lý hoạt động của mạng truyền thông CAN Bus và ứng dụng vào việc xây dựng mô hình hệ thống đa ECU trên ô tô. Thông qua việc thiết kế và triển khai thực nghiệm, đề tài giúp làm rõ cơ chế trao đổi dữ liệu giữa các ECU, cơ chế xử lý sự kiện và khả năng phối hợp hoạt động giữa các khối chức năng trong hệ thống điện tử ô tô.

Đề tài đồng thời hướng đến việc áp dụng kiến trúc phần mềm AUTOSAR Classic thu gọn trong phát triển phần mềm nhúng nhằm tăng tính mô đun, khả

năng tái sử dụng và mở rộng của hệ thống. Trên cơ sở đó, mô hình được xây dựng sẽ mô phỏng các chức năng cơ bản của hệ thống ABS và Airbag, cho phép đánh giá hiệu quả truyền thông CAN Bus cũng như khả năng vận hành của hệ thống đa ECU trong điều kiện thực nghiệm.

1.3.3 Mục tiêu nghiên cứu

Để đạt được các mục đích trên, đề tài thực hiện các mục tiêu cụ thể như sau:

Về phần cứng

- Thiết kế và xây dựng mô hình hệ thống đa ECU gồm:
 - Main ECU sử dụng vi điều khiển STM32F407VG.
 - ABS ECU sử dụng vi điều khiển STM32F103C8T6.
 - Airbag ECU sử dụng vi điều khiển STM32F103C8T6.
- Xây dựng mạng truyền thông CAN Bus kết nối giữa ba ECU thông qua các module CAN transceiver SN65HVD230.
- Thiết kế hệ thống nguồn gồm:
 - Nguồn đầu vào 12 VDC.
 - Mạch hạ áp LM2596 tạo nguồn 5 VDC.
 - Mạch ổn áp 3.3 V cấp nguồn cho các module CAN Bus.
- Tích hợp các phần cứng ngoại vi gồm:
 - Động cơ DC giảm tốc GA25-370 có encoder Hall hai kênh.
 - Driver điều khiển động cơ XY-160D.
 - Cảm biến gia tốc 3 trục LIS331HH.
 - Màn hình LCD 16x2.
 - Biến trở điều chỉnh tốc độ.
 - Nút nhấn điều khiển hướng quay.
 - Nút nhấn phanh khẩn cấp.
 - Còi cảnh báo (Buzzer).

Về phần mềm

- Xây dựng phần mềm Main ECU thực hiện:
 - Đọc giá trị ADC từ biến trở.
 - Đọc trạng thái các nút nhấn điều khiển.

- Gửi lệnh điều khiển đến ABS ECU.
- Nhận dữ liệu phản hồi từ ABS ECU và Airbag ECU.
- Hiển thị trạng thái hệ thống trên LCD 16x2.
- Xây dựng phần mềm ABS ECU thực hiện:
 - Điều khiển tốc độ và chiều quay động cơ bằng PWM.
 - Đọc tín hiệu encoder của hai bánh xe.
 - Tính toán tốc độ quay của động cơ.
 - Gửi dữ liệu phản hồi tốc độ về Main ECU.
- Xây dựng phần mềm Airbag ECU thực hiện:
 - Thu thập dữ liệu gia tốc từ cảm biến LIS331HH.
 - Phát hiện va chạm dựa trên ngưỡng gia tốc cài đặt.
 - Kích hoạt còi cảnh báo khi xảy ra va chạm.
 - Truyền trạng thái Airbag về Main ECU thông qua CAN Bus.
- Xây dựng các thành phần phần mềm theo hướng AUTOSAR Classic thu gọn gồm:
 - Application Layer.
 - Runtime Environment (RTE).
 - Communication Layer.
 - Driver Layer.

Về truyền thông CAN Bus

- Thiết kế các khung dữ liệu CAN phục vụ trao đổi thông tin giữa các ECU.
- Xây dựng cơ chế truyền nhận dữ liệu CAN Bus ở mức thanh ghi (Register Level Programming).
- Thực hiện truyền dữ liệu giữa:
 - Main ECU và ABS ECU.
 - Main ECU và Airbag ECU.
- Giám sát trạng thái hoạt động của mạng CAN thông qua:
 - Sequence Counter.
 - Lost Frame Counter.
 - Error Counter.

- Diagnostic Message.
- CAN Controller State.

Về đánh giá hệ thống

- Đánh giá khả năng truyền nhận dữ liệu giữa các ECU thông qua mạng CAN Bus.
- Đánh giá độ ổn định và độ tin cậy của hệ thống truyền thông.
- Đánh giá khả năng phản hồi của hệ thống khi xảy ra sự kiện va chạm.
- Đánh giá cơ chế dừng khẩn cấp (Emergency Stop) khi Airbag ECU phát hiện va chạm.
- Đánh giá khả năng áp dụng kiến trúc AUTOSAR Classic thu gọn trong phát triển hệ thống đa ECU trên nền tảng vi điều khiển STM32.

1.4 GIẢ THIẾT NGHIÊN CỨU VÀ GIỚI HẠN ĐỀ TÀI

1.4.1 Giả thiết nghiên cứu

Đề tài được thực hiện dựa trên giả thiết rằng việc xây dựng hệ thống đa ECU sử dụng mạng truyền thông CAN Bus có thể mô phỏng được nguyên lý hoạt động cơ bản của hệ thống điện – điện tử trên ô tô hiện đại. Các ECU trong hệ thống được giả thiết hoạt động ổn định, sử dụng chung một đường truyền CAN Bus và tuân theo cơ chế truyền nhận dữ liệu của chuẩn CAN 2.0A.

Các vi điều khiển STM32F407 và STM32F103C8T6 được giả thiết có đủ năng lực xử lý để thực hiện các chức năng điều khiển, giám sát và trao đổi dữ liệu thời gian thực giữa các ECU. Các module giao tiếp CAN SN65HVD230 được giả thiết đáp ứng tốt yêu cầu truyền dữ liệu trong môi trường phòng thí nghiệm với tốc độ truyền được cấu hình phù hợp.

Đề tài cũng giả thiết rằng cảm biến gia tốc LIS331HH có khả năng cung cấp dữ liệu gia tốc đủ chính xác để phục vụ việc phát hiện va chạm trong mô hình Airbag ECU. Đồng thời, encoder tích hợp trên động cơ GA25-370 có thể cung cấp tín hiệu phản hồi tốc độ phục vụ cho chức năng mô phỏng ABS ECU.

Về mặt phần mềm, đề tài giả thiết rằng kiến trúc AUTOSAR Classic thu gọn được xây dựng có khả năng tổ chức các thành phần phần mềm theo hướng mô đun hóa, cho phép các ECU trao đổi dữ liệu thông qua Runtime Environment (RTE) và Communication Layer tương tự nguyên lý triển khai trên hệ thống AUTOSAR thực tế [13].

Ngoài ra, đề tài giả thiết rằng các thông số đo được từ hệ thống thực nghiệm như tốc độ động cơ, trạng thái CAN Bus, trạng thái Airbag và dữ liệu chẩn đoán có thể phản ánh được hoạt động của mô hình đa ECU trong điều kiện thử nghiệm.

1.4.2 Giới hạn đề tài

Do giới hạn về thời gian thực hiện, kinh phí và điều kiện thiết bị, đề tài được triển khai với một số giới hạn như sau:

Về quy mô hệ thống

Hệ thống chỉ xây dựng mô hình gồm ba ECU:

- Main ECU.
- ABS ECU.
- Airbag ECU.

Các ECU khác thường xuất hiện trên ô tô như Engine ECU, BCM, TCU, EPS, HVAC hoặc Gateway ECU chưa được triển khai trong phạm vi đề tài.

Về mạng truyền thông

Đề tài chỉ tập trung nghiên cứu và triển khai mạng CAN Bus trên một đường truyền vật lý duy nhất. Các giao thức truyền thông khác trong ô tô như LIN, FlexRay, Automotive Ethernet hoặc CAN FD chưa được xem xét.

Việc đánh giá hiệu năng mạng CAN chỉ thực hiện trong phạm vi mô hình thực nghiệm với số lượng ECU hạn chế, chưa đánh giá trong môi trường mạng có tải lớn hoặc nhiều node truyền thông.

Về chức năng ABS

ABS ECU trong đề tài chỉ mô phỏng chức năng điều khiển động cơ, đo tốc độ bánh xe thông qua encoder và gửi dữ liệu phản hồi về Main ECU.

Các thuật toán điều khiển chống bó cứng phanh thực tế như Slip Ratio Control, Wheel Deceleration Detection hay Hydraulic Pressure Modulation chưa được triển khai.

Về chức năng Airbag

Airbag ECU sử dụng cảm biến gia tốc LIS331HH để phát hiện va chạm dựa trên ngưỡng gia tốc được cài đặt trước.

Hệ thống không triển khai cơ cấu kích nổ túi khí thực tế mà chỉ sử dụng còi buzzer để mô phỏng trạng thái kích hoạt Airbag nhằm đảm bảo an toàn trong quá trình thử nghiệm.

Về kiến trúc AUTOSAR

Đề tài áp dụng kiến trúc AUTOSAR Classic theo hướng thu gọn nhằm phục vụ mục đích nghiên cứu và đào tạo.

Các thành phần AUTOSAR chuyên sâu như:

- ECU Configuration Description (ECUC)
- Basic Software đầy đủ (BSW)
- Diagnostic Event Manager (DEM)
- Diagnostic Communication Manager (DCM)
- Network Management (NM)
- AUTOSAR OS chưa được triển khai đầy đủ như trong các hệ thống AUTOSAR thương mại [2].

Về đánh giá hệ thống

Việc đánh giá hệ thống chủ yếu tập trung vào:

- Khả năng truyền nhận CAN Bus.
- Độ ổn định của mạng CAN.
- Hoạt động của ABS ECU.
- Hoạt động của Airbag ECU.
- Cơ chế Emergency Stop.
- Khả năng phối hợp giữa các ECU.

Các thử nghiệm trong điều kiện rung động mạnh, nhiễu điện từ (EMI/EMC), nhiệt độ khắc nghiệt hoặc môi trường thực tế trên ô tô chưa nằm trong phạm vi nghiên cứu của đề tài.

1.5 PHƯƠNG PHÁP NGHIÊN CỨU

Để thực hiện đề tài “Thiết kế hệ thống đa ECU giao tiếp CAN Bus theo kiến trúc AUTOSAR Classic thu gọn”, nhóm tiến hành nghiên cứu theo quy trình kết hợp giữa nghiên cứu lý thuyết, thiết kế hệ thống, phát triển phần cứng, xây dựng phần mềm nhúng và đánh giá thực nghiệm. Các nội dung nghiên cứu chính được thực hiện như sau:

Nghiên cứu cơ sở lý thuyết:

- Tìm hiểu nguyên lý hoạt động của mạng truyền thông CAN Bus, bao gồm cơ chế truyền nhận dữ liệu, phân xử ưu tiên thông điệp, phát hiện lỗi và xử lý lỗi trên mạng CAN.
- Nghiên cứu kiến trúc phần mềm AUTOSAR Classic, đặc biệt là mô hình phân lớp gồm Application Layer, Runtime Environment (RTE), Basic Software (BSW) và Microcontroller Abstraction Layer (MCAL).
- Tìm hiểu cấu trúc phần cứng và các ngoại vi của vi điều khiển STM32F407VG và STM32F103C8T6 phục vụ cho việc xây dựng các ECU trong hệ thống.
- Nghiên cứu nguyên lý hoạt động của cảm biến gia tốc LIS331HH, encoder Hall hai kênh, driver điều khiển động cơ XY-160D và các module giao tiếp CAN sử dụng chip SN65HVD230.
- Tổng hợp và phân tích các công trình nghiên cứu liên quan đến hệ thống điều khiển phân tán trên ô tô, mạng CAN Bus và ứng dụng AUTOSAR trong hệ thống nhúng hiện đại nhằm làm cơ sở xây dựng mô hình nghiên cứu.

Thiết kế hệ thống:

- Xây dựng kiến trúc hệ thống gồm ba ECU bao gồm Main ECU, ABS ECU và Airbag ECU được kết nối với nhau thông qua mạng truyền thông CAN Bus.
- Thiết kế sơ đồ khối tổng thể của hệ thống, xác định chức năng của từng ECU và luồng trao đổi dữ liệu giữa các nút trên mạng CAN.
- Thiết kế khối nguồn cung cấp điện áp 12V, 5V và 3.3V cho các thành phần phần cứng trong hệ thống.
- Thiết kế mạch nguyên lý và bố trí các thành phần phần cứng trên mô hình thực nghiệm nhằm đảm bảo tính ổn định và thuận tiện cho quá trình kiểm thử.

Phát triển phần cứng:

- Xây dựng Main ECU sử dụng vi điều khiển STM32F407VG làm bộ điều khiển trung tâm, đảm nhiệm chức năng giám sát hệ thống, xử lý dữ liệu đầu vào và điều phối hoạt động giữa các ECU.
- Xây dựng ABS ECU sử dụng vi điều khiển STM32F103C8T6, driver XY-160D và hai động cơ DC GA25-370 tích hợp encoder nhằm mô phỏng hệ thống chống bó cứng phanh và thu thập dữ liệu tốc độ bánh xe.

- Xây dựng Airbag ECU sử dụng vi điều khiển STM32F103C8T6 kết hợp cảm biến gia tốc LIS331HH để phát hiện các tình huống va chạm và kích hoạt cơ chế cảnh báo bằng còi buzzer.
- Tích hợp các module giao tiếp CAN SN65HVD230 trên cả ba ECU để thực hiện truyền thông dữ liệu trên mạng CAN Bus.

Phát triển phần mềm:

- Xây dựng phần mềm điều khiển cho từng ECU bằng ngôn ngữ C trên nền tảng STM32CubeIDE.
- Triển khai các driver ngoại vi ở mức thanh ghi (Register Level Programming) cho các khối GPIO, ADC, UART, TIM, I2C và CAN.
- Xây dựng các lớp phần mềm theo kiến trúc AUTOSAR Classic thu gọn gồm MCAL, ECU Abstraction Layer, Service Layer, Runtime Environment (RTE) và Application Layer.
- Thiết kế các thông điệp CAN phục vụ trao đổi dữ liệu giữa Main ECU, ABS ECU và Airbag ECU.
- Xây dựng các chức năng ứng dụng bao gồm điều khiển động cơ, đọc encoder, giám sát tín hiệu biến trở, xử lý nút nhấn điều khiển, giám sát gia tốc và phát hiện va chạm.

Kiểm thử và đánh giá thực nghiệm:

- Kiểm tra khả năng truyền nhận dữ liệu giữa các ECU trên mạng CAN Bus ở nhiều điều kiện hoạt động khác nhau.
- Đánh giá khả năng điều khiển động cơ và thu thập dữ liệu tốc độ bánh xe từ encoder trên ABS ECU.
- Đánh giá khả năng phát hiện va chạm của Airbag ECU thông qua dữ liệu thu được từ cảm biến gia tốc LIS331HH.
- Kiểm tra hoạt động đồng bộ giữa các ECU thông qua các bản tin CAN và đánh giá độ ổn định của hệ thống.
- So sánh kết quả thực nghiệm với mục tiêu thiết kế ban đầu nhằm đánh giá tính khả thi của mô hình AUTOSAR Classic thu gọn trên nền tảng vi điều khiển STM32.

Thông qua quy trình nghiên cứu trên, đề tài hướng tới việc xây dựng thành công một hệ thống đa ECU hoạt động theo mô hình điều khiển phân tán, sử dụng mạng truyền thông CAN Bus và kiến trúc AUTOSAR Classic thu gọn. Kết quả nghiên cứu không chỉ giúp làm rõ nguyên lý hoạt động của các hệ thống điện tử trên ô tô hiện đại mà còn tạo nền tảng cho các nghiên cứu mở rộng về

AUTOSAR, chẩn đoán lỗi và các hệ thống điều khiển thông minh trong tương lai.

1.6 BỐ CỤC ĐỒ ÁN

Chương 1. TỔNG QUAN

Chương này trình bày tổng quan về đề tài, lý do chọn đề tài, mục tiêu thực hiện, phạm vi giới hạn, phương pháp nghiên cứu, đối tượng và phạm vi nghiên cứu.

Chương 2. CƠ SỞ LÝ THUYẾT

Trình bày các kiến thức liên quan đến mạng CAN, kiến trúc AUTOSAR Classic, vi điều khiển STM32, cảm biến gia tốc LIS331HH và các thành phần được sử dụng trong hệ thống.

Chương 3. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Chương này trình bày quá trình thiết kế và xây dựng hệ thống, bao gồm yêu cầu hệ thống, mô hình tổng thể, thiết kế phần cứng và thiết kế phần mềm.

Phần thiết kế phần cứng: trình bày sơ đồ khối, chức năng các khối của hệ thống, cơ sở lựa chọn linh kiện như động cơ, vi điều khiển STM32, cảm biến gia tốc LIS331HH, các module cần thiết khác, thiết kế từng khối, tính toán thông số linh kiện cho từng khối. Đồng thời mô tả kiến trúc kết nối giữa các thành phần.

Phần thiết kế phần mềm trình bày chi tiết yêu cầu, chức năng, quy trình phát triển hệ thống phần mềm, quá trình thu nhận dữ liệu cảm biến và cơ chế truyền thông CAN.

Chương 4. KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG

Trình bày kết quả triển khai hệ thống, kết quả truyền nhận CAN Bus, điều khiển động cơ, phát hiện va chạm và đánh giá hoạt động của hệ thống.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các nội dung đã thực hiện, đánh giá mức độ hoàn thành mục tiêu đề tài và nêu ra những kết quả đạt được của hệ thống. Bên cạnh đó, chương này đề xuất các hướng phát triển nhằm nâng cao độ ổn định, khả năng điều hướng và hiệu quả tối truyền dữ liệu và độ tin cậy trong các nghiên cứu tiếp theo.

CHƯƠNG 2

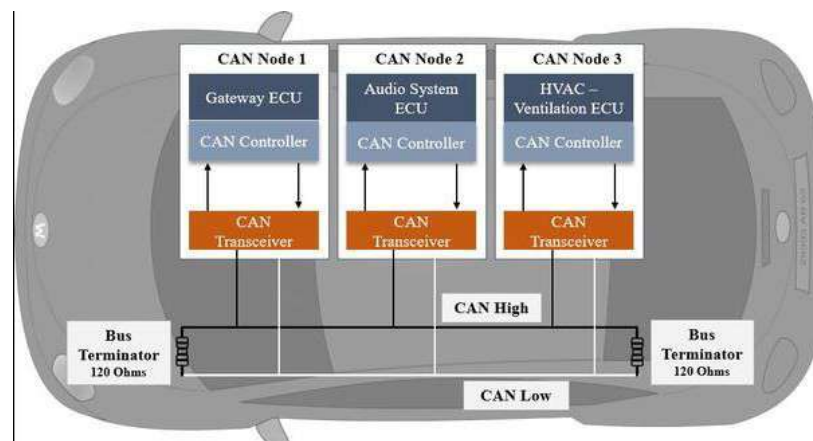
CƠ SỞ LÝ THUYẾT

2.1 GIAO THỨC CAN

2.1.1 Tổng quan về mạng CAN (Controller Area Network)

Giao thức CAN (Controller Area Network) là chuẩn truyền thông nối tiếp được phát triển bởi hãng Bosch vào năm 1986 nhằm đáp ứng nhu cầu trao đổi dữ liệu giữa các bộ điều khiển điện tử (Electronic Control Unit - ECU) trong ô tô mà không cần sử dụng máy tính trung tâm [2]. Nhờ khả năng truyền dữ liệu theo thời gian thực, độ tin cậy cao và khả năng chống nhiễu tốt, CAN đã trở thành một trong những chuẩn truyền thông phổ biến nhất trong lĩnh vực ô tô, công nghiệp tự động hóa và hệ thống nhúng [2], [14].

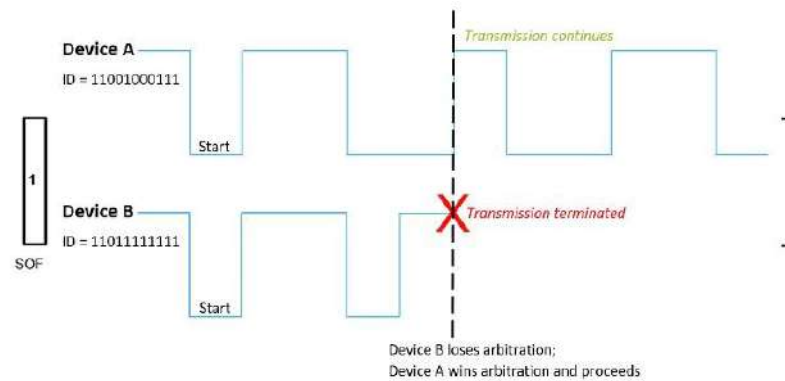
Trong mạng CAN, các ECU được kết nối chung trên một đường truyền vật lý gọi là CAN Bus. Dữ liệu được truyền theo cơ chế quảng bá (broadcast), nghĩa là mọi nút trên mạng đều có thể nhận được tất cả các khung dữ liệu (frame). Mỗi ECU sẽ quyết định xử lý hoặc bỏ qua dữ liệu dựa trên trường định danh (Identifier - ID) của frame nhận được [14]. Nhờ cơ chế này, việc mở rộng số lượng nút trong hệ thống trở nên linh hoạt mà không cần thay đổi cấu trúc mạng.



Hình 2.1 Mô hình kết nối nhiều ECU trên mạng CAN Bus.

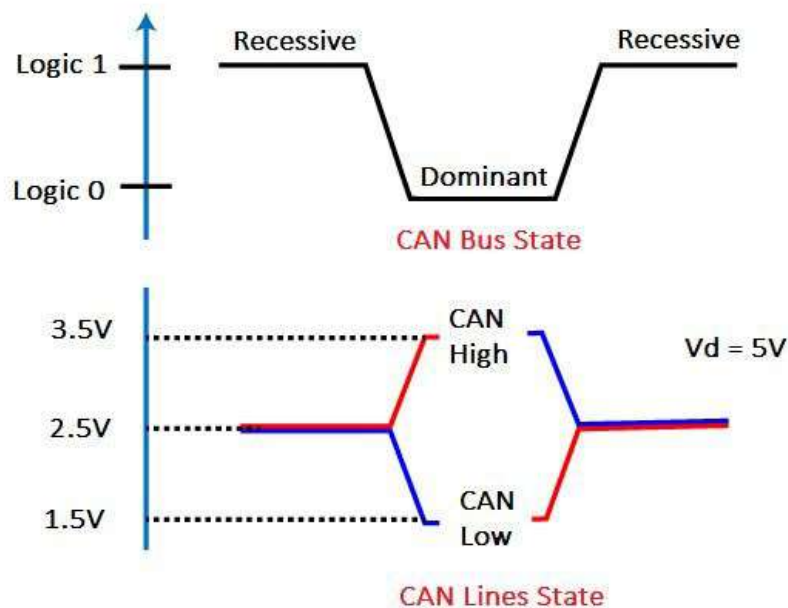
Một trong những đặc điểm nổi bật của CAN là cơ chế phân xử bus (Bus Arbitration) dựa trên độ ưu tiên của trường ID. Khi nhiều ECU cùng yêu cầu truyền dữ liệu tại một thời điểm, ECU có ID ưu tiên cao hơn (giá trị nhị phân nhỏ hơn) sẽ giành quyền truyền mà không làm mất dữ liệu của các ECU còn lại [2]. Cơ chế này được gọi là Non-Destructive Bitwise Arbitration và là yếu tố quan

trọng giúp CAN đáp ứng yêu cầu thời gian thực trong các hệ thống điều khiển phân tán.



Hình 2.2 Cơ chế phân xử ưu tiên trên mạng CAN Bus.

Bên cạnh đó, CAN sử dụng phương thức truyền tín hiệu vi sai thông qua hai đường tín hiệu CAN_H và CAN_L. Khi truyền dữ liệu, hiệu điện thế giữa hai đường tín hiệu thay đổi theo trạng thái Dominant và Recessive, giúp tăng khả năng chống nhiễu điện từ và đảm bảo độ tin cậy của hệ thống trong môi trường công nghiệp hoặc ô tô có nhiều nguồn nhiễu [14], [15].



Hình 2.3 Nguyên lý truyền tín hiệu vi sai CAN_H và CAN_L.

Ngoài khả năng chống nhiễu, giao thức CAN còn được trang bị nhiều cơ chế phát hiện và xử lý lỗi như Bit Monitoring, Bit Stuffing Check, Cyclic Redundancy Check (CRC), Frame Check và Acknowledgement Check. Các cơ chế này cho phép hệ thống phát hiện lỗi truyền thông và tự động cô lập các nút

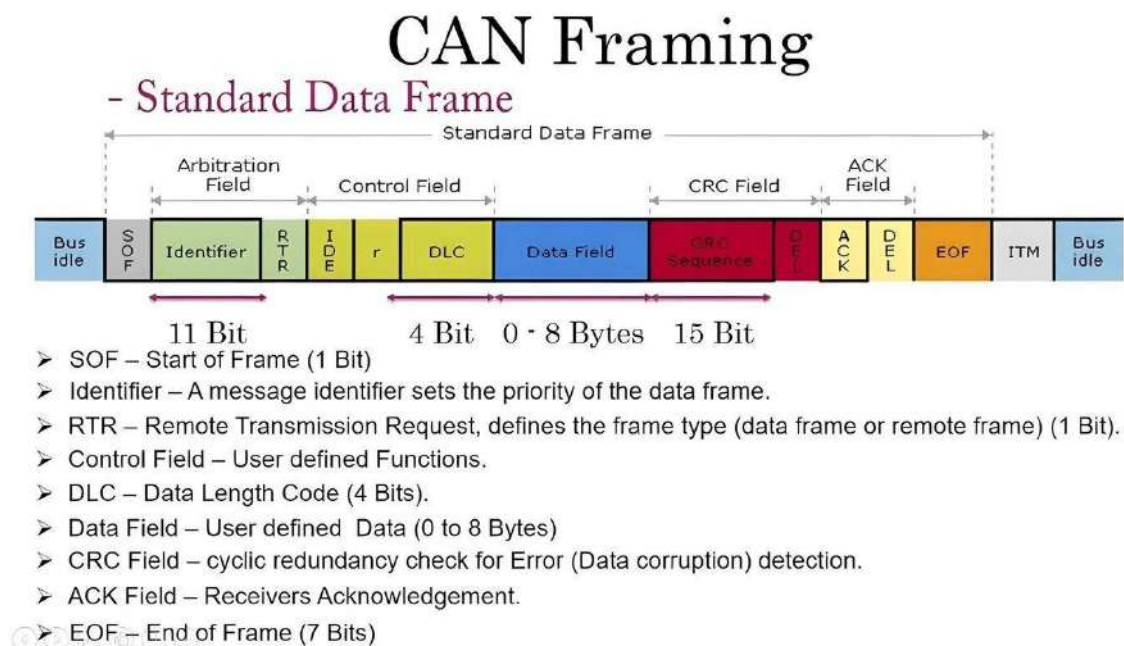
hoạt động không ổn định, từ đó nâng cao độ tin cậy của toàn bộ mạng CAN [14], [15].

Trong phạm vi đồ án này, giao thức CAN được sử dụng làm nền tảng truyền thông giữa ba ECU gồm ECU Main sử dụng vi điều khiển STM32F407 và hai ECU chức năng (ABS ECU, Airbag ECU) sử dụng vi điều khiển STM32F103C8T6. Các ECU trao đổi dữ liệu thông qua CAN Bus sử dụng transceiver SN65HVD230 theo kiến trúc AUTOSAR Classic thu gọn nhằm mô phỏng hệ thống mạng truyền thông trên ô tô ở quy mô nhỏ.

2.1.2 Cấu trúc khung dữ liệu trong CAN

Dữ liệu trong mạng CAN được truyền dưới dạng các khung dữ liệu (Frame), trong đó phổ biến nhất là Data Frame. Một khung CAN bao gồm nhiều trường thông tin khác nhau, mỗi trường đảm nhiệm một chức năng cụ thể trong quá trình truyền thông giữa các ECU trên mạng CAN [14], [2].

Một khung CAN tiêu chuẩn (Standard CAN Frame) bao gồm các thành phần chính như: trường bắt đầu khung (Start of Frame – SOF), trường định danh (Identifier), trường điều khiển (Control Field), trường dữ liệu (Data Field), trường kiểm tra lỗi (CRC Field), trường xác nhận (ACK Field) và trường kết thúc khung (End of Frame – EOF) như minh họa trong Hình 2.4 [2].



Hình 2.4 Cấu trúc khung dữ liệu chuẩn CAN 2.0A.

Bảng 2.1 Chức năng các trường trong CAN Frame

Trường	Chức năng
--------	-----------

SOF	Đánh dấu bắt đầu khung dữ liệu
Identifier	Định danh và xác định mức ưu tiên
RTR	Xác định loại khung dữ liệu
IDE	Xác định chuẩn 11 bit hay 29 bit
DLC	Xác định số byte dữ liệu
Data Field	Chứa dữ liệu ứng dụng
CRC	Kiểm tra lỗi truyền dữ liệu
ACK	Xác nhận dữ liệu đã được nhận
EOF	Đánh dấu kết thúc khung

Trường Identifier không chỉ dùng để nhận dạng nội dung dữ liệu mà còn quyết định mức độ ưu tiên của frame trong quá trình phân xử bus (Arbitration). Khi nhiều ECU cùng truyền dữ liệu, frame có Identifier ưu tiên cao hơn sẽ được quyền tiếp tục truyền mà không xảy ra va chạm dữ liệu [14].

Trường Data Field chứa dữ liệu ứng dụng được truyền giữa các ECU. Trong chuẩn CAN 2.0, trường dữ liệu có kích thước tối đa 8 byte. Trường CRC (Cyclic Redundancy Check) được sử dụng để kiểm tra tính toàn vẹn của dữ liệu trong quá trình truyền. Nếu phát hiện lỗi, frame sẽ bị loại bỏ và yêu cầu truyền lại. Trường ACK cho phép các node nhận xác nhận việc tiếp nhận dữ liệu thành công, giúp tăng độ tin cậy của hệ thống truyền thông [15], [2].

Ngoài khung dữ liệu chuẩn (Standard Frame) sử dụng Identifier 11 bit, CAN còn hỗ trợ khung dữ liệu mở rộng (Extended Frame) với Identifier 29 bit. Việc mở rộng độ dài Identifier cho phép tăng số lượng định danh trong hệ thống, phù hợp với các mạng CAN có quy mô lớn hoặc yêu cầu phân loại dữ liệu chi tiết hơn [14]. Tuy nhiên, nguyên lý hoạt động của hai loại khung dữ liệu này về cơ bản là tương tự nhau.

Đối với đề tài này, hệ thống sử dụng chuẩn CAN 2.0A với Identifier 11 bit để trao đổi dữ liệu giữa ba ECU gồm Main ECU, ABS ECU và Airbag ECU. Việc sử dụng Standard Frame giúp giảm tải truyền thông, đơn giản hóa quá trình xử lý dữ liệu và phù hợp với quy mô mô hình nghiên cứu.

2.1.3 Cơ chế phát hiện lỗi trong mạng CAN

Một trong những ưu điểm nổi bật của giao thức CAN là khả năng phát hiện và xử lý lỗi mạnh mẽ. Để đảm bảo độ tin cậy cao trong quá trình truyền dữ liệu, CAN được trang bị nhiều cơ chế phát hiện lỗi hoạt động song song. Các cơ chế

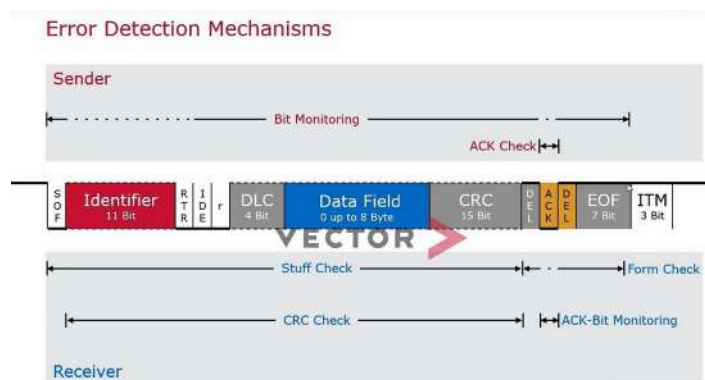
này cho phép phát hiện lỗi ngay khi phát sinh và thực hiện các biện pháp xử lý phù hợp nhằm duy trì tính toàn vẹn của dữ liệu trên mạng truyền thông [14], [2].

Theo tiêu chuẩn CAN 2.0, hệ thống sử dụng năm cơ chế phát hiện lỗi chính gồm: Bit Monitoring, Bit Stuffing Check, CRC Check, Form Check và ACK Check [2].

Cơ chế Bit Monitoring

Bit Monitoring là cơ chế trong đó node phát đồng thời giám sát mức tín hiệu xuất hiện trên đường truyền CAN Bus trong quá trình gửi dữ liệu. Nếu giá trị đọc được trên bus khác với giá trị mà node đang truyền, một lỗi bit (Bit Error) sẽ được phát hiện [14].

Cơ chế này giúp phát hiện các lỗi phát sinh do nhiễu điện từ, hỏng phần cứng hoặc sai lệch tín hiệu truyền trên đường CAN_H và CAN_L.

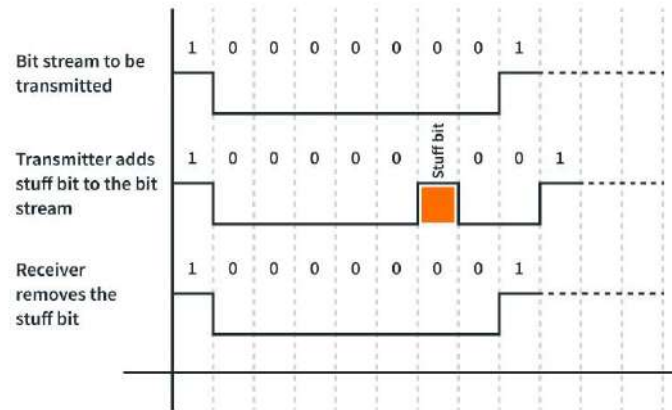


Hình 2.5 Nguyên lý phát hiện lỗi bằng cơ chế Bit Monitoring.

Cơ chế Bit Stuffing Check

Trong CAN, để đảm bảo đồng bộ hóa giữa các node, sau mỗi chuỗi năm bit liên tiếp có cùng giá trị, bộ phát sẽ tự động chèn thêm một bit có giá trị ngược lại. Quá trình này được gọi là Bit Stuffing [2].

Node nhận sẽ kiểm tra quy tắc này trong suốt quá trình nhận dữ liệu. Nếu phát hiện có sáu bit liên tiếp cùng giá trị mà không xuất hiện bit chèn, lỗi Stuff Error sẽ được phát hiện và truyền Error Frame lên mạng CAN [14].

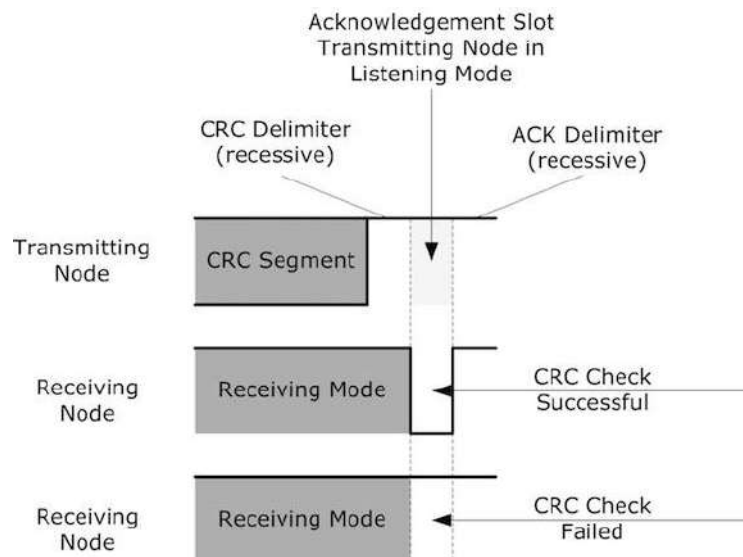


Hình 2.6 Minh họa cơ chế Bit Stuffing trong CAN.

Cơ chế CRC Check

CRC (Cyclic Redundancy Check) là phương pháp kiểm tra tính toàn vẹn dữ liệu được sử dụng phổ biến trong các hệ thống truyền thông số. Trước khi truyền frame, node phát sẽ tính toán mã CRC dựa trên nội dung dữ liệu và chèn giá trị này vào trường CRC của frame [15].

Khi nhận dữ liệu, node nhận thực hiện tính toán lại CRC và so sánh với giá trị nhận được. Nếu hai giá trị không khớp, hệ thống sẽ kết luận frame bị lỗi và yêu cầu truyền lại dữ liệu.



Hình 2.7 Nguyên lý kiểm tra lỗi CRC trong CAN.

Cơ chế Form Check

Form Check là cơ chế kiểm tra cấu trúc của các trường có định dạng cố định trong CAN Frame như:

- CRC Delimiter

- ACK Delimiter
- End Of Frame (EOF)
- Intermission

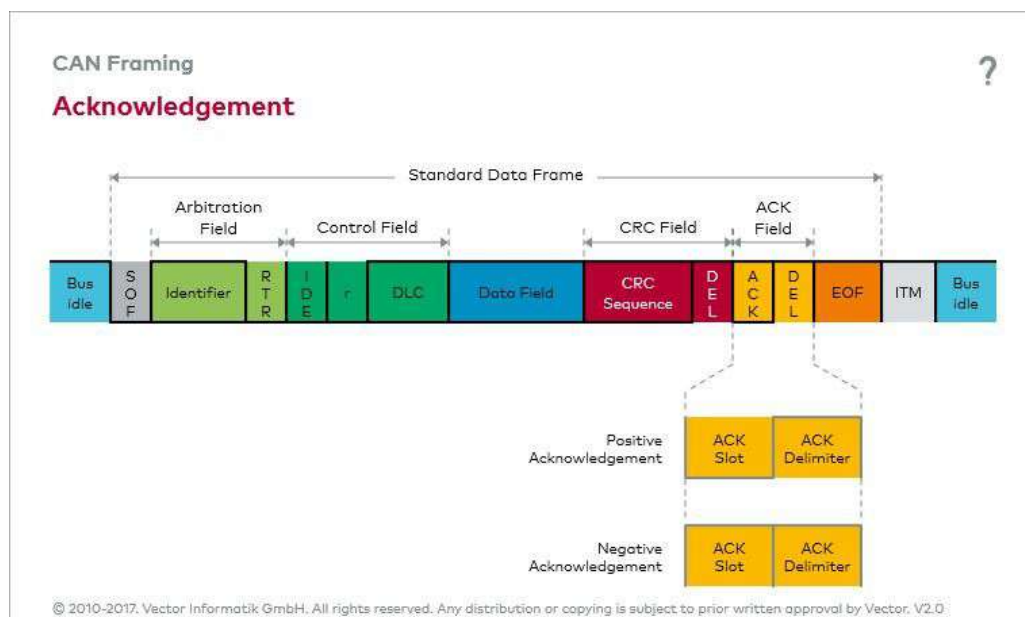
Nếu một trong các trường này không tuân theo định dạng được quy định trong tiêu chuẩn CAN, hệ thống sẽ phát hiện Form Error và gửi Error Frame lên mạng [14], [2].

Cơ chế này giúp đảm bảo cấu trúc khung dữ liệu luôn đúng chuẩn và tránh lỗi trong quá trình giải mã dữ liệu.

Cơ chế ACK Check

Sau khi truyền xong dữ liệu, node phát sẽ kiểm tra trường ACK để xác định xem có node nào trên mạng nhận được frame hay không.

Nếu không có bất kỳ node nào ghi nhận dữ liệu và phản hồi ACK, node phát sẽ phát hiện ACK Error và thực hiện truyền lại frame theo cơ chế tự động của CAN Controller [15].



Hình 2.8 Cơ chế xác nhận ACK trong mạng CAN.

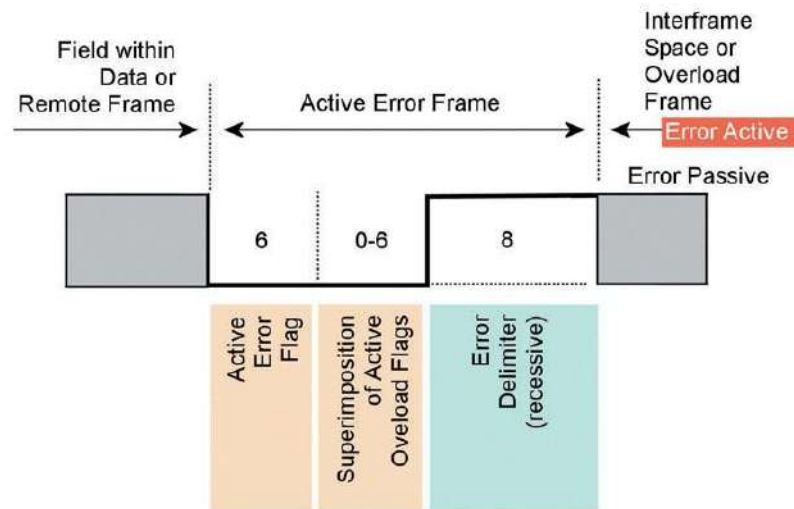
Cơ chế Error Frame

Khi bất kỳ lỗi nào được phát hiện, node sẽ phát một khung lỗi (Error Frame) lên mạng CAN nhằm thông báo cho tất cả các node khác biết frame hiện tại không hợp lệ [14].

Error Frame gồm hai phần:

- Error Flag
- Error Delimiter

Sau khi Error Frame được truyền, tất cả các node sẽ loại bỏ frame lỗi và thực hiện truyền lại dữ liệu nếu cần thiết.



Hình 2.9 Cấu trúc Error Frame trong giao thức CAN.

Bảng 2.2 Các cơ chế phát hiện lỗi trong CAN

Cơ chế	Chức năng
Bit Monitoring	Phát hiện sai khác giữa dữ liệu truyền và dữ liệu thực tế trên bus
Bit Stuffing Check	Kiểm tra quy tắc chèn bit đồng bộ
CRC Check	Kiểm tra tính toàn vẹn dữ liệu
Form Check	Kiểm tra cấu trúc các trường cố định trong frame
ACK Check	Kiểm tra việc xác nhận dữ liệu từ node nhận

2.1.4 Cơ chế xử lý lỗi và fault confinement

Bên cạnh các cơ chế phát hiện lỗi, giao thức CAN còn được trang bị cơ chế xử lý lỗi và cô lập lỗi (Fault Confinement) nhằm đảm bảo một node gặp sự cố không làm ảnh hưởng đến hoạt động của toàn bộ hệ thống. Đây là một trong những đặc điểm quan trọng giúp CAN đạt được độ tin cậy cao trong các ứng dụng ô tô và công nghiệp [14], [2].

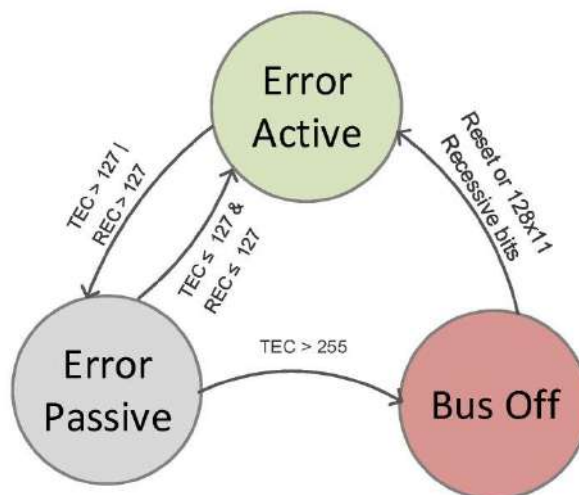
Khi một node phát hiện lỗi trong quá trình truyền hoặc nhận dữ liệu, node đó sẽ phát một Error Frame lên mạng CAN để thông báo cho các node khác biết

rằng frame hiện tại không hợp lệ. Sau đó, frame bị lỗi sẽ bị hủy bỏ và được truyền lại theo cơ chế tự động của CAN Controller [2].

Để đánh giá mức độ lỗi của từng node, giao thức CAN sử dụng hai bộ đếm lỗi:

- Transmit Error Counter (TEC): Bộ đếm lỗi phát.
- Receive Error Counter (REC): Bộ đếm lỗi nhận.

Mỗi khi xảy ra lỗi trong quá trình truyền hoặc nhận dữ liệu, các bộ đếm tương ứng sẽ được tăng lên theo quy định của chuẩn CAN. Ngược lại, khi truyền hoặc nhận dữ liệu thành công, giá trị của các bộ đếm sẽ giảm xuống [14].



Hình 2.10 Nguyên lý hoạt động của bộ đếm lỗi TEC và REC trong CAN.

Dựa trên giá trị của hai bộ đếm lỗi TEC và REC, mỗi node CAN có thể chuyển đổi giữa ba trạng thái hoạt động khác nhau gồm Error Active, Error Passive và Bus Off [15], [2].

Trạng thái Error Active

Đây là trạng thái hoạt động bình thường của node CAN. Khi số lượng lỗi còn thấp, node có đầy đủ quyền truy cập bus và có thể phát Active Error Frame khi phát hiện lỗi. Trong trạng thái này, node tham gia truyền thông như các node bình thường khác trên mạng [14].

Trạng thái Error Passive

Khi số lượng lỗi vượt quá ngưỡng cho phép nhưng chưa đến mức nghiêm trọng, node sẽ chuyển sang trạng thái Error Passive. Trong trạng thái này, node vẫn được phép truyền và nhận dữ liệu nhưng khả năng tác động lên bus bị hạn chế nhằm giảm ảnh hưởng đến các node khác trên hệ thống [2].

Trạng thái Bus Off

Khi bộ đếm lỗi TEC vượt quá ngưỡng quy định của tiêu chuẩn CAN ($TEC > 255$), node sẽ chuyển sang trạng thái Bus Off. Lúc này node bị ngắt hoàn toàn khỏi mạng CAN và không được phép truyền dữ liệu cho đến khi được khởi tạo lại [15].

Bảng 2.3 Trạng thái Fault Confinement trong CAN

Trạng thái	Điều kiện
Error Active	$TEC < 128$ và $REC < 128$
Error Passive	$TEC \geq 128$ hoặc $REC \geq 128$
Bus Off	$TEC > 255$

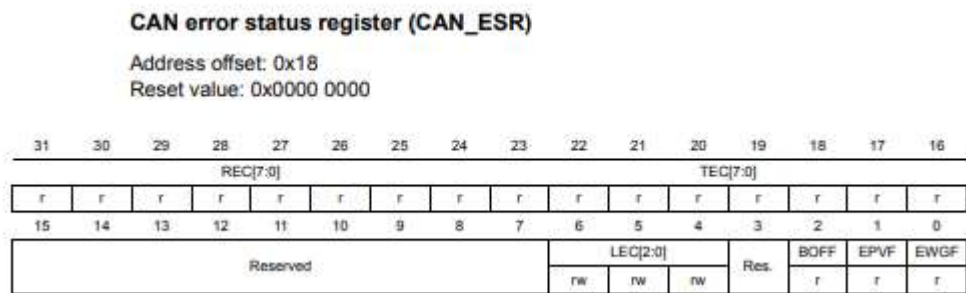
Nhờ cơ chế Fault Confinement, CAN có khả năng cô lập các node bị lỗi khỏi mạng truyền thông trước khi chúng gây ảnh hưởng đến toàn bộ hệ thống. Điều này giúp duy trì tính ổn định, độ tin cậy và khả năng hoạt động liên tục của mạng CAN trong các môi trường yêu cầu mức độ an toàn cao như hệ thống ABS, Airbag, động cơ và các ECU trên ô tô [14], [2].

2.1.5 Thanh ghi và cơ chế xử lý lỗi CAN trên vi điều khiển STM32

Trên các vi điều khiển STM32F103C8T6 và STM32F407VG được sử dụng trong đề tài, bộ điều khiển CAN (bxCAN) được tích hợp sẵn bên trong phần cứng vi điều khiển, cho phép thực hiện truyền nhận dữ liệu CAN mà không cần bộ điều khiển CAN ngoài. Bộ điều khiển bxCAN cung cấp hệ thống thanh ghi chuyên dụng phục vụ cấu hình, truyền nhận dữ liệu và giám sát trạng thái hoạt động của mạng CAN. Các thanh ghi này cho phép lập trình viên truy cập trực tiếp vào phần cứng để xây dựng giao thức truyền thông CAN ở mức thanh ghi (Register-Level Programming) [16].

Trong số các thanh ghi liên quan đến cơ chế xử lý lỗi, thanh ghi quan trọng nhất là Error Status Register (CAN_ESR). Thanh ghi này lưu trữ thông tin về trạng thái lỗi hiện tại của CAN Controller, cho phép phần mềm giám sát và phân tích các sự cố xảy ra trên mạng CAN. Theo tài liệu tham khảo của STMicroelectronics, thanh ghi ESR bao gồm các trường thông tin chính như [16]:

- LEC (Last Error Code): Mã lỗi gần nhất.
- EWGF (Error Warning Flag): Cờ cảnh báo lỗi.
- EPVF (Error Passive Flag): Cờ báo trạng thái Error Passive.
- BOFF (Bus-Off Flag): Cờ báo trạng thái Bus Off.



Hình 2.11 Thanh ghi CAN_ESR trong bộ điều khiển bxCAN của STM32.

Trường LEC (Last Error Code) cho biết loại lỗi gần nhất xảy ra trên mạng CAN. Dựa vào giá trị của trường này, hệ thống có thể xác định nguyên nhân gây lỗi như lỗi Bit Error, Stuff Error, CRC Error, Form Error hoặc ACK Error. Đây là thông tin quan trọng phục vụ quá trình phân tích và chẩn đoán lỗi của mạng CAN trong thực tế.

Bên cạnh đó, các cờ trạng thái EWGF, EPVF và BOFF phản ánh mức độ nghiêm trọng của lỗi trên node CAN.

Error Warning (EWGF)

Khi số lượng lỗi tích lũy vượt ngưỡng cảnh báo (Error Warning Limit), cờ EWGF được thiết lập. Trạng thái này cho biết node bắt đầu xuất hiện lỗi nhưng vẫn có khả năng tham gia truyền thông bình thường. Việc giám sát cờ EWGF giúp hệ thống phát hiện sớm các sự cố tiềm ẩn trên mạng CAN.

Error Passive (EPVF)

Khi bộ đếm lỗi TEC hoặc REC tăng đến ngưỡng Error Passive, cờ EPVF được kích hoạt. Trong trạng thái này node vẫn được phép truyền và nhận dữ liệu nhưng quyền tác động lên bus bị hạn chế nhằm giảm ảnh hưởng tới các node khác trên mạng.

Bus Off (BOFF)

Khi bộ đếm lỗi truyền TEC vượt quá giới hạn cho phép theo tiêu chuẩn CAN, node sẽ chuyển sang trạng thái Bus Off. Lúc này node bị ngắt hoàn toàn khỏi mạng CAN và không được phép truyền dữ liệu cho đến khi được khởi tạo lại. Cờ BOFF trong thanh ghi ESR cho phép phần mềm nhận biết trạng thái này để thực hiện các biện pháp phục hồi phù hợp.

Ngoài thanh ghi ESR, bộ điều khiển CAN trên STM32 còn cung cấp các thanh ghi khác như thanh ghi điều khiển (CAN_MCR), thanh ghi trạng thái (CAN_MSR), các mailbox truyền (TX Mailbox) và bộ đệm nhận (RX FIFO). Sự kết hợp của các thành phần này cho phép lập trình viên không chỉ thực hiện truyền nhận dữ liệu mà còn giám sát và xử lý lỗi một cách hiệu quả.

Bảng 2.4 Các thanh ghi CAN quan trọng trên STM32

Thanh ghi	Chức năng
CAN_MCR	Điều khiển và cấu hình bộ CAN
CAN_MSR	Theo dõi trạng thái hoạt động CAN
CAN_TSR	Quản lý trạng thái các Mailbox truyền
CAN_RF0R	Quản lý FIFO nhận 0
CAN_RF1R	Quản lý FIFO nhận 1
CAN_ESR	Theo dõi trạng thái lỗi
CAN_BTR	Cấu hình tốc độ baudrate CAN
CAN_TI0R, TDT0R, TDL0R, TDH0R	Thanh ghi Mailbox truyền
CAN_RI0R, RDT0R, RDL0R, RDH0R	Thanh ghi FIFO nhận

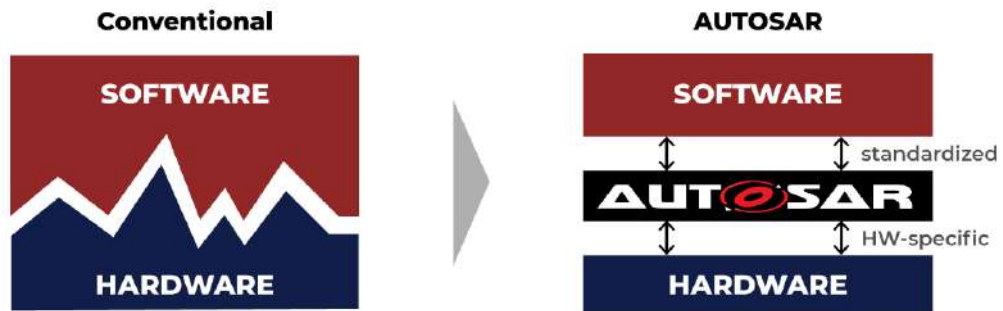
Việc khai thác các thanh ghi này trong đề tài giúp cho hệ thống thực nghiệm có khả năng ghi nhận trạng thái lỗi theo thời gian thực, từ đó phục vụ cho việc phân tích cơ chế fault confinement và hành vi của node CAN trong các điều kiện lỗi khác nhau.

2.2 TỔNG QUAN VỀ AUTOSAR CLASSIC

2.2.1 Khái niệm

AUTOSAR (AUTomotive Open System ARchitecture) là một tiêu chuẩn kiến trúc phần mềm mở dành cho các hệ thống điện tử ô tô, được phát triển bởi sự hợp tác giữa nhiều nhà sản xuất ô tô, nhà cung cấp linh kiện và các công ty phần mềm trên thế giới. Mục tiêu chính của AUTOSAR là xây dựng một nền tảng phần mềm chuẩn hóa nhằm tăng khả năng tái sử dụng, khả năng mở rộng và tính tương thích giữa các hệ thống điều khiển điện tử (Electronic Control Unit – ECU) khác nhau [20].

Trong các phương tiện hiện đại, số lượng ECU ngày càng gia tăng để thực hiện các chức năng như điều khiển động cơ, hệ thống phanh chống bó cứng (ABS), túi khí (Airbag), điều khiển thân xe và các hệ thống hỗ trợ lái. Việc phát triển phần mềm riêng biệt cho từng ECU dẫn đến khó khăn trong quá trình tích hợp, bảo trì và mở rộng hệ thống. Để giải quyết vấn đề này, AUTOSAR cung cấp một kiến trúc phần mềm phân lớp, cho phép tách biệt phần mềm ứng dụng khỏi phần cứng, đồng thời chuẩn hóa các giao diện giao tiếp giữa các thành phần trong hệ thống [19].



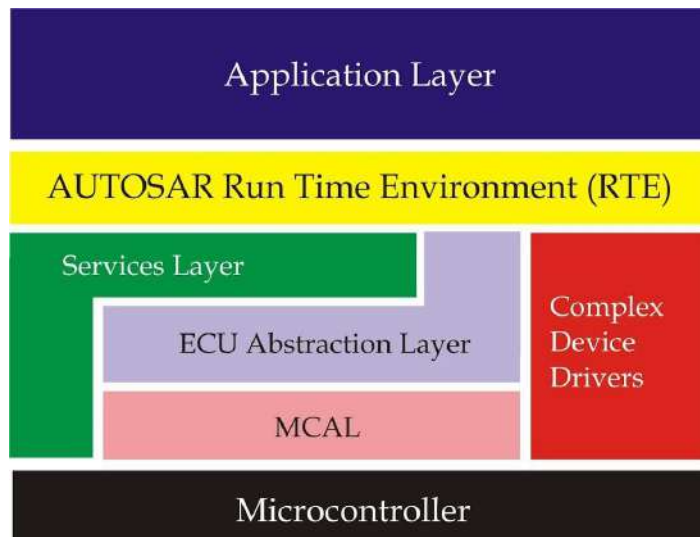
Hình 2.12 So sánh mô hình phát triển phần mềm truyền thống và kiến trúc AUTOSAR.

Kiến trúc AUTOSAR được xây dựng dựa trên nguyên tắc phân tách chức năng thành nhiều lớp khác nhau, bao gồm lớp ứng dụng (Application Layer), môi trường thực thi thời gian chạy (Runtime Environment – RTE) và phần mềm cơ sở (Basic Software – BSW). Cách tổ chức này giúp các nhà phát triển có thể thay đổi phần cứng hoặc mở rộng chức năng hệ thống mà không cần sửa đổi đáng kể phần mềm ứng dụng [17].

Nhờ khả năng chuẩn hóa và tái sử dụng phần mềm, AUTOSAR hiện đã trở thành tiêu chuẩn phổ biến trong ngành công nghiệp ô tô, được áp dụng rộng rãi trong quá trình phát triển các hệ thống điều khiển điện tử hiện đại. Việc nghiên cứu và áp dụng AUTOSAR không chỉ giúp nâng cao chất lượng phần mềm mà còn tạo điều kiện thuận lợi cho việc tích hợp các ECU trong các hệ thống ô tô phức tạp [18].

2.2.2 Kiến trúc phân lớp AUTOSAR Classic.

AUTOSAR Classic được xây dựng theo mô hình kiến trúc phân lớp nhằm tách biệt phần mềm ứng dụng khỏi phần cứng, đồng thời chuẩn hóa giao diện giữa các thành phần trong hệ thống. Cách tổ chức này giúp nâng cao khả năng tái sử dụng phần mềm, giảm sự phụ thuộc vào phần cứng và tạo điều kiện thuận lợi cho việc phát triển các hệ thống đa ECU có quy mô lớn [17].



Hình 2.13 Kiến trúc phân lớp AUTOSAR Classic Platform.

Trong kiến trúc AUTOSAR, lớp BSW được chia thành nhiều lớp con gồm Service Layer, ECU Abstraction Layer, MCAL và Complex Device Driver. Service Layer cung cấp các dịch vụ hệ thống, truyền thông và chẩn đoán; ECU Abstraction Layer che giấu sự khác biệt của phần cứng; MCAL cung cấp các driver ngoại vi mức thấp; còn Complex Device Driver được sử dụng cho các thiết bị đặc biệt yêu cầu truy cập trực tiếp đến phần cứng [17].

Application Layer là lớp cao nhất của kiến trúc AUTOSAR, chứa các thành phần phần mềm ứng dụng (Software Component – SWC) thực hiện các chức năng nghiệp vụ của hệ thống. Các thành phần này chỉ tập trung vào thuật toán và chức năng điều khiển mà không cần quan tâm đến chi tiết phần cứng bên dưới. Trong hệ thống ô tô, lớp ứng dụng có thể thực hiện các chức năng như điều khiển động cơ, giám sát tốc độ bánh xe, phát hiện va chạm hoặc điều khiển hệ thống túi khí [17].

Runtime Environment (RTE) đóng vai trò là lớp trung gian giữa Application Layer và Basic Software. RTE cung cấp các giao diện giao tiếp chuẩn hóa, cho phép các thành phần phần mềm ứng dụng trao đổi dữ liệu với nhau hoặc truy cập các dịch vụ từ lớp phần mềm cơ sở. Nhờ RTE, các thành phần ứng dụng không cần truy cập trực tiếp đến phần cứng, qua đó tăng tính độc lập và khả năng tái sử dụng của phần mềm [17].

Basic Software (BSW) là lớp phần mềm cơ sở cung cấp các dịch vụ cần thiết cho hệ thống. Lớp này bao gồm các module quản lý truyền thông, quản lý hệ thống, quản lý bộ nhớ và các dịch vụ hỗ trợ khác. BSW đóng vai trò cầu nối giữa RTE và các driver phần cứng, đồng thời đảm bảo các chức năng chung của ECU được thực hiện một cách thống nhất [17].

Microcontroller Abstraction Layer (MCAL) là lớp thấp nhất trong kiến trúc AUTOSAR, nằm ngay phía trên phần cứng vi điều khiển. MCAL cung cấp các driver chuẩn hóa cho các ngoại vi như CAN, ADC, GPIO, SPI và UART. Thông qua MCAL, các lớp phần mềm phía trên có thể truy cập tài nguyên phần cứng mà không cần phụ thuộc vào đặc điểm cụ thể của từng dòng vi điều khiển [18].

Nhờ cấu trúc phân lớp này, AUTOSAR Classic giúp tách biệt rõ ràng giữa phần mềm ứng dụng và phần cứng, tạo điều kiện thuận lợi cho việc phát triển, bảo trì và mở rộng hệ thống. Đây cũng là nền tảng quan trọng giúp các hệ thống ô tô hiện đại có thể tích hợp số lượng lớn ECU nhưng vẫn đảm bảo tính nhất quán và khả năng tái sử dụng phần mềm.

2.2.2.1 Runtime Environment (RTE)

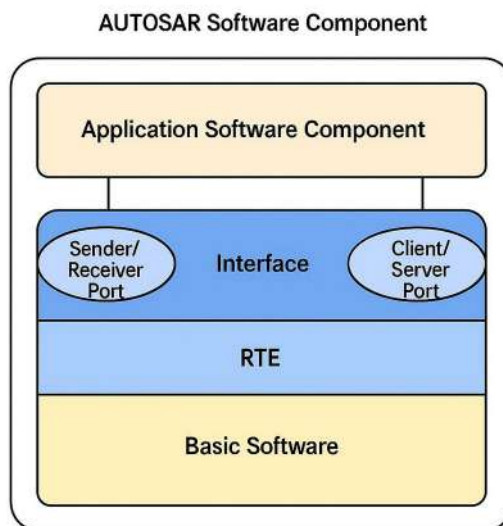
Runtime Environment (RTE) là lớp trung gian trong kiến trúc AUTOSAR Classic, đóng vai trò kết nối giữa lớp ứng dụng (Application Layer) và lớp phần mềm cơ sở (Basic Software – BSW). RTE cung cấp các giao diện truyền thông chuẩn hóa, cho phép các thành phần phần mềm ứng dụng trao đổi dữ liệu với nhau hoặc sử dụng các dịch vụ từ lớp bên dưới mà không cần truy cập trực tiếp đến phần cứng. Nhờ đó, RTE giúp tách biệt phần mềm ứng dụng khỏi nền tảng phần cứng và nâng cao khả năng tái sử dụng phần mềm giữa các ECU khác nhau [21], [18].

Một trong những chức năng quan trọng nhất của RTE là che giấu sự phức tạp của hệ thống bên dưới đối với các thành phần ứng dụng. Thông qua các giao diện chuẩn hóa, Software Component (SWC) có thể gửi hoặc nhận dữ liệu mà không cần quan tâm dữ liệu được truyền qua CAN, LIN hay các cơ chế truyền thông khác. Điều này giúp giảm sự phụ thuộc giữa các thành phần trong hệ thống và tăng tính linh hoạt trong quá trình phát triển phần mềm ô tô [21].

Trong AUTOSAR Classic, RTE hoạt động như một lớp trung gian điều phối dữ liệu giữa các Software Component và các dịch vụ hệ thống. Khi một thành phần ứng dụng cần gửi hoặc nhận dữ liệu, yêu cầu sẽ được chuyển đến RTE. Sau đó, RTE thực hiện việc truyền dữ liệu đến thành phần hoặc module tương ứng dựa trên cấu hình đã được định nghĩa trước. Nhờ cơ chế này, các thành phần ứng dụng có thể hoạt động độc lập mà không cần biết vị trí hoặc cách thức triển khai của các thành phần khác trong hệ thống [18].

Ngoài chức năng truyền dữ liệu, RTE còn hỗ trợ ánh xạ các cổng giao tiếp (Ports) giữa các Software Component, quản lý luồng dữ liệu và cung cấp các giao diện truy cập dịch vụ hệ thống. Điều này giúp đảm bảo tính nhất quán trong quá trình trao đổi dữ liệu và tạo điều kiện thuận lợi cho việc mở rộng hệ thống trong tương lai [22].

Trong AUTOSAR tiêu chuẩn, lớp RTE thường được sinh tự động từ các công cụ cấu hình dựa trên mô hình hệ thống. Tuy nhiên, trong các hệ thống nghiên cứu hoặc thực nghiệm quy mô nhỏ, RTE có thể được xây dựng thủ công nhằm đơn giản hóa kiến trúc và phù hợp với yêu cầu của đề tài. Mặc dù được đơn giản hóa, RTE vẫn giữ nguyên vai trò là cầu nối giữa phần mềm ứng dụng và các lớp phần mềm cơ sở, đảm bảo nguyên tắc phân lớp đặc trưng của kiến trúc AUTOSAR [21].



Hình 2.14 Cơ chế truyền dữ liệu thông qua RTE

Dữ liệu được truyền từ SWC nguồn đến SWC đích thông qua Runtime Environment. RTE chịu trách nhiệm ánh xạ dữ liệu, điều phối giao tiếp và cung cấp các dịch vụ cần thiết cho các thành phần ứng dụng.

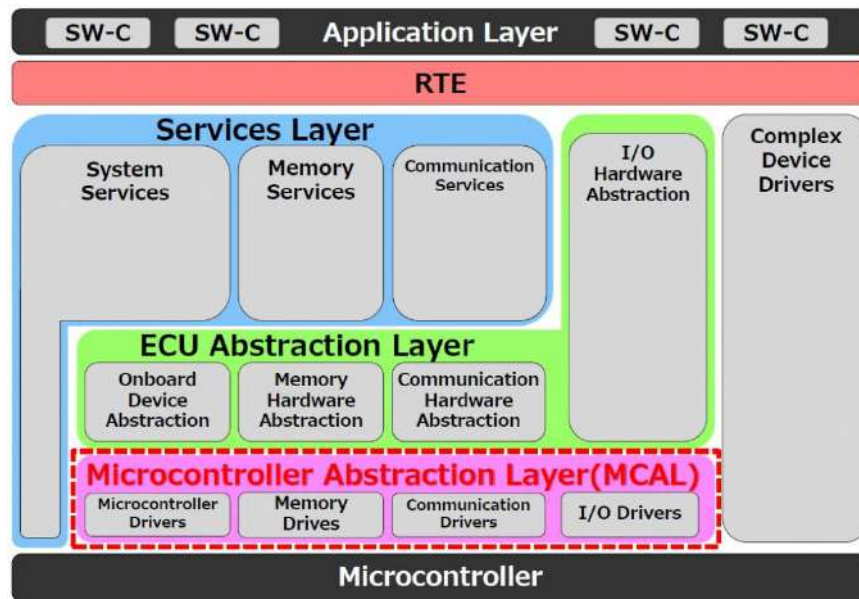
2.2.2.2 Basic Software (BSW)

Basic Software (BSW) là lớp phần mềm cơ sở trong kiến trúc AUTOSAR Classic, nằm giữa Runtime Environment (RTE) và lớp trừu tượng phần cứng vi điều khiển (Microcontroller Abstraction Layer – MCAL). BSW cung cấp các dịch vụ hệ thống cần thiết cho hoạt động của ECU, đồng thời tạo ra môi trường chuẩn hóa để các thành phần phần mềm ứng dụng có thể hoạt động độc lập với phần cứng bên dưới. Nhờ đó, BSW góp phần nâng cao khả năng tái sử dụng phần mềm, giảm sự phụ thuộc vào nền tảng phần cứng và tăng khả năng mở rộng của hệ thống [17], [18].

Vai trò chính của BSW là quản lý các chức năng hệ thống như truyền thông, quản lý trạng thái ECU, chẩn đoán lỗi, quản lý bộ nhớ và các dịch vụ hỗ trợ khác. Thông qua lớp này, các thành phần phần mềm ứng dụng có thể truy cập tài nguyên hệ thống mà không cần giao tiếp trực tiếp với phần cứng. Điều này giúp

giảm mức độ phụ thuộc giữa phần mềm ứng dụng và phần cứng, đồng thời tạo điều kiện thuận lợi cho việc bảo trì và phát triển các hệ thống ECU phức tạp [18].

Trong kiến trúc AUTOSAR Classic, BSW được tổ chức thành nhiều lớp chức năng khác nhau. Lớp Service Layer cung cấp các dịch vụ hệ thống như quản lý ECU, quản lý lỗi, quản lý bộ nhớ và các dịch vụ truyền thông. Lớp ECU Abstraction Layer cung cấp giao diện trừu tượng hóa phần cứng, giúp các lớp phần mềm phía trên không phụ thuộc vào đặc điểm cụ thể của vi điều khiển hoặc thiết bị ngoại vi. Ngoài ra, BSW còn bao gồm các module truyền thông đảm nhiệm việc trao đổi dữ liệu giữa các ECU thông qua các giao thức như CAN, LIN, FlexRay hoặc Ethernet [17], [23].



Hình 2.15 Cấu trúc lớp Basic Software trong AUTOSAR Classic.

Đối với các hệ thống AUTOSAR đầy đủ, BSW thường bao gồm nhiều module chức năng chuyên biệt. Tuy nhiên, trong các hệ thống thực nghiệm hoặc nghiên cứu quy mô nhỏ, chỉ những module cần thiết mới được triển khai nhằm giảm độ phức tạp và phù hợp với tài nguyên phần cứng. Mặc dù được rút gọn, các module này vẫn tuân thủ nguyên tắc phân lớp và cơ chế giao tiếp chuẩn hóa của AUTOSAR [18].

Nhờ đóng vai trò là lớp trung gian giữa Runtime Environment và MCAL, Basic Software giúp hệ thống phần mềm đạt được cấu trúc rõ ràng, dễ bảo trì và thuận lợi cho việc mở rộng chức năng trong tương lai. Đây là một thành phần quan trọng góp phần tạo nên tính chuẩn hóa và khả năng tái sử dụng phần mềm của kiến trúc AUTOSAR Classic [17].

Các thành phần chính của BSW

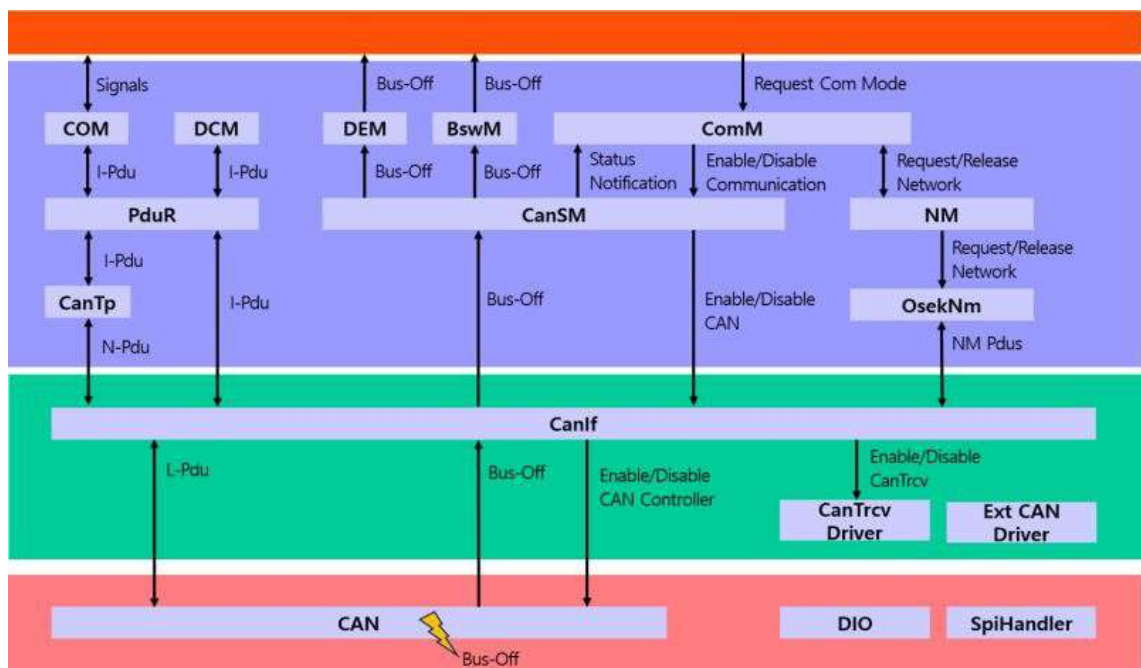
Service Layer là lớp cao nhất trong BSW, cung cấp các dịch vụ hệ thống dùng chung như:

- ECU State Manager (EcuM)
- Diagnostic Event Manager (Dem)
- Diagnostic Communication Manager (Dcm)
- Watchdog Manager (WdgM)
- Communication Services

Lớp này giúp quản lý trạng thái hoạt động của ECU, ghi nhận lỗi và hỗ trợ giao tiếp với các công cụ chẩn đoán bên ngoài [23].

ECU Abstraction Layer cung cấp giao diện chuẩn hóa giữa phần mềm và phần cứng ngoại vi. Thông qua lớp này, phần mềm phía trên có thể sử dụng cảm biến, bộ nhớ hoặc thiết bị truyền thông mà không cần quan tâm đến chi tiết triển khai phần cứng [17].

Communication Stack là thành phần đặc biệt quan trọng trong AUTOSAR vì chịu trách nhiệm truyền dữ liệu giữa các ECU.



Hình 2.16 Ngăn xếp truyền thông CAN trong AUTOSAR.

Các module chính bao gồm:

- COM (Communication Module)
- PduR (PDU Router)
- CanIf (CAN Interface)
- CanDrv (CAN Driver)

Trong đó:

- COM xử lý tín hiệu ứng dụng.
- PduR định tuyến dữ liệu giữa các giao thức.
- CanIf cung cấp giao diện chuẩn hóa cho CAN.
- CanDrv điều khiển trực tiếp phần cứng CAN của vi điều khiển.

Các module này giúp dữ liệu từ ứng dụng có thể được đóng gói và truyền xuống mạng CAN một cách chuẩn hóa [23].

2.2.2.3 Microcontroller Abstraction Layer (MCAL)

Microcontroller Abstraction Layer (MCAL) là lớp thấp nhất trong kiến trúc AUTOSAR Classic, nằm ngay phía trên phần cứng vi điều khiển. MCAL cung cấp các driver chuẩn hóa cho các ngoại vi như GPIO, ADC, Timer, SPI, UART và CAN, cho phép các lớp phần mềm phía trên truy cập tài nguyên phần cứng thông qua giao diện thống nhất mà không phụ thuộc vào loại vi điều khiển cụ thể [18], [24].

Mục tiêu chính của MCAL là tách biệt phần mềm ứng dụng khỏi phần cứng. Nhờ đó, khi thay đổi nền tảng vi điều khiển, các lớp phần mềm phía trên như Application Layer, Runtime Environment (RTE) và Basic Software (BSW) có thể được tái sử dụng mà không cần sửa đổi đáng kể. Điều này giúp giảm chi phí phát triển và nâng cao khả năng mở rộng của hệ thống [24].

Trong AUTOSAR Classic, MCAL bao gồm nhiều driver ngoại vi khác nhau như GPIO Driver, ADC Driver, PWM Driver, SPI Driver, UART Driver và CAN Driver. Các driver này chịu trách nhiệm cấu hình thanh ghi phần cứng, quản lý hoạt động của ngoại vi và cung cấp các hàm giao tiếp chuẩn hóa cho các lớp phần mềm bên trên [18].

Đối với các hệ thống sử dụng mạng CAN, CAN Driver là một trong những thành phần quan trọng nhất của MCAL. Driver này thực hiện khởi tạo bộ điều khiển CAN, cấu hình tốc độ truyền, thiết lập bộ lọc, quản lý mailbox truyền nhận và xử lý các sự kiện truyền thông CAN. Thông qua CAN Driver, các module truyền thông ở lớp BSW có thể trao đổi dữ liệu giữa các ECU mà không cần truy cập trực tiếp đến các thanh ghi phần cứng [16].

Trong phạm vi đề tài này, lớp MCAL được hiện thực bằng các chương trình điều khiển ngoại vi trên vi điều khiển STM32F103C8T6 và STM32F407VG. Các driver như CAN, GPIO, ADC và Timer được xây dựng trực tiếp dựa trên thanh ghi phần cứng của STM32. Đặc biệt, CAN Driver đóng vai trò quan trọng trong việc nghiên cứu cơ chế Fault Confinement và phân tích trạng thái lỗi của mạng CAN giữa các ECU thực nghiệm.

2.2.3 AUTOSAR Classic thu gọn

AUTOSAR Classic là một kiến trúc phần mềm toàn diện được phát triển nhằm đáp ứng các yêu cầu của các hệ thống điện tử ô tô hiện đại. Trong thực tế công nghiệp, một hệ thống AUTOSAR đầy đủ bao gồm nhiều module chức năng như hệ điều hành (Operating System), quản lý bộ nhớ (Memory Stack), chẩn đoán (Diagnostic Communication Manager – DCM), quản lý lỗi (Diagnostic Event Manager – DEM), quản lý trạng thái ECU và nhiều dịch vụ hệ thống khác. Tuy nhiên, việc triển khai đầy đủ các thành phần này thường đòi hỏi tài nguyên phần cứng lớn, công cụ cấu hình chuyên dụng và quy trình phát triển phức tạp.

Đối với các hệ thống nghiên cứu, đào tạo hoặc thực nghiệm quy mô nhỏ, việc áp dụng toàn bộ kiến trúc AUTOSAR là không thực sự cần thiết. Vì vậy, nhiều nghiên cứu lựa chọn cách tiếp cận AUTOSAR Classic thu gọn (Lightweight AUTOSAR Classic), trong đó chỉ giữ lại các thành phần cốt lõi của kiến trúc nhằm đảm bảo tính phân lớp và nguyên tắc thiết kế của AUTOSAR, đồng thời giảm độ phức tạp trong quá trình triển khai.

Trong mô hình AUTOSAR Classic thu gọn, các lớp chính bao gồm Application Layer, Runtime Environment (RTE), Basic Software (BSW) và Microcontroller Abstraction Layer (MCAL) vẫn được duy trì. Lớp Application chứa các chức năng điều khiển và xử lý nghiệp vụ của hệ thống. Lớp RTE đảm nhiệm vai trò trung gian trong quá trình trao đổi dữ liệu giữa ứng dụng và các dịch vụ hệ thống. Lớp BSW cung cấp các chức năng cơ sở như truyền thông và quản lý hệ thống, trong khi MCAL chịu trách nhiệm truy cập trực tiếp đến phần cứng thông qua các driver ngoại vi.

Ngược lại, các thành phần phức tạp như hệ điều hành AUTOSAR, các module chẩn đoán, quản lý bộ nhớ và các công cụ sinh mã tự động được lược bỏ nhằm giảm yêu cầu về tài nguyên hệ thống cũng như đơn giản hóa quá trình phát triển. Việc loại bỏ các thành phần này không làm thay đổi nguyên tắc phân lớp của AUTOSAR mà chỉ giới hạn phạm vi chức năng phù hợp với mục tiêu nghiên cứu và thực nghiệm.

Trong phạm vi đề tài này, kiến trúc AUTOSAR Classic thu gọn được lựa chọn để tổ chức phần mềm cho hệ thống đa ECU giao tiếp CAN. Cách tiếp cận này vừa đảm bảo tính trực quan trong việc triển khai và kiểm soát phần cứng ở mức thanh ghi, vừa duy trì cấu trúc phần mềm phân lớp theo định hướng của AUTOSAR. Nhờ đó, hệ thống có khả năng mở rộng, dễ bảo trì và phù hợp với các nghiên cứu về hệ thống nhúng và truyền thông ô tô.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CỦA HỆ THỐNG

Trong các hệ thống điện tử ô tô hiện đại, nhiều bộ điều khiển điện tử (ECU) cùng hoạt động và trao đổi dữ liệu với nhau thông qua mạng truyền thông CAN nhằm thực hiện các chức năng điều khiển và đảm bảo an toàn cho phương tiện. Để mô phỏng mô hình này, đề tài xây dựng một hệ thống đa ECU gồm Main ECU, ABS ECU và Airbag ECU được kết nối với nhau thông qua mạng CAN và tổ chức phần mềm theo kiến trúc AUTOSAR Classic thu gọn.

Hệ thống được thiết kế nhằm đáp ứng các yêu cầu kỹ thuật sau:

- Khả năng truyền thông CAN: Hệ thống phải cho phép các ECU trao đổi dữ liệu với nhau thông qua mạng CAN sử dụng module SN65HVD230. Quá trình truyền nhận dữ liệu phải đảm bảo ổn định, chính xác và đáp ứng yêu cầu thời gian thực..
- Triển khai CAN mức thanh ghi: CAN Driver phải được xây dựng bằng phương pháp truy cập trực tiếp thanh ghi của vi điều khiển STM32 thay vì sử dụng thư viện HAL nhằm làm rõ cơ chế hoạt động của bộ điều khiển CAN.
- Kiến trúc phần mềm phân lớp: Phần mềm phải được tổ chức theo kiến trúc AUTOSAR Classic thu gọn, bao gồm các lớp MCAL, BSW và Application Layer nhằm nâng cao tính cấu trúc, khả năng mở rộng và tái sử dụng phần mềm.
- Điều Giám sát trạng thái hệ thống: Main ECU phải có khả năng thu thập dữ liệu từ các ECU chức năng, xử lý thông tin và hiển thị trạng thái hoạt động của hệ thống trên màn hình LCD 16x2.
- Chức năng mô phỏng hệ thống ABS: ABS ECU phải có khả năng điều khiển hai động cơ DC GA25-370 thông qua mạch điều khiển động cơ XY-160D. Hệ thống cho phép thay đổi tốc độ động cơ bằng biến trở và điều khiển chiều quay thông qua các nút nhấn nhằm mô phỏng tốc độ bánh xe.
- Chức năng mô phỏng hệ thống Airbag: Airbag ECU phải có khả năng thu thập dữ liệu từ cảm biến gia tốc LIS331HH, xử lý tín hiệu gia tốc và phát hiện trạng thái va chạm dựa trên ngưỡng được thiết lập trước.

- Giao tiếp UART phục vụ Debug: Các ECU phải hỗ trợ giao tiếp UART nhằm theo dõi dữ liệu, giám sát trạng thái hoạt động và hỗ trợ quá trình kiểm tra, gỡ lỗi hệ thống.
- Khả năng hoạt động độc lập của các ECU: Mỗi ECU phải có khả năng thực hiện chức năng riêng của mình đồng thời phối hợp với các ECU khác thông qua mạng CAN.
- Tính ổn định và khả năng mở rộng: Hệ thống phải hoạt động ổn định trong điều kiện thực nghiệm, cho phép mở rộng thêm các ECU hoặc chức năng mới mà không làm thay đổi cấu trúc tổng thể của phần mềm.

Từ các yêu cầu trên, hệ thống được xây dựng dưới dạng mô hình thực nghiệm đa ECU gồm một Main ECU sử dụng vi điều khiển STM32F407 và hai ECU chức năng sử dụng vi điều khiển STM32F103C8T6. Các ECU được kết nối thông qua mạng CAN sử dụng transceiver SN65HVD230 và tổ chức phần mềm theo kiến trúc AUTOSAR Classic thu gọn. Trên cơ sở đó, hệ thống được phát triển với các chức năng chính như sau:

- Thu thập và xử lý dữ liệu từ cảm biến gia tốc LIS331HH.
- Điều khiển động cơ DC và mô phỏng tốc độ bánh xe của hệ thống ABS.
- Trao đổi dữ liệu giữa các ECU thông qua mạng CAN.
- Hiển thị trạng thái hệ thống trên màn hình LCD 16x2.
- Giám sát và gỡ lỗi thông qua giao tiếp UART.
- Tổ chức phần mềm theo mô hình AUTOSAR Classic thu gọn.
- Đánh giá khả năng hoạt động của hệ thống đa ECU trên phần cứng thực tế

Các chức năng trên là cơ sở để triển khai, kiểm tra và đánh giá hệ thống trong các phần tiếp theo của đề án

3.2 MÔ HÌNH HỆ THỐNG

3.2.1 Mô hình tổng thể của hệ thống

Hệ thống thực nghiệm được xây dựng nhằm nghiên cứu cơ chế truyền thông CAN Bus và phân tích cơ chế Fault Confinement trong mạng CAN sử dụng các vi điều khiển STM32. Hệ thống được thiết kế theo kiến trúc nhiều ECU tương tự các hệ thống điện – điện tử trên ô tô hiện đại, bao gồm Main ECU, ABS ECU và Airbag ECU được kết nối với nhau thông qua mạng CAN Bus.

Main ECU sử dụng vi điều khiển STM32F407VG đóng vai trò điều khiển trung tâm, thực hiện thu thập dữ liệu từ các thiết bị điều khiển, xử lý thông tin và điều phối hoạt động của toàn hệ thống. ABS ECU sử dụng vi điều khiển STM32F103C8T6 kết hợp với driver công suất và động cơ DC có encoder để mô phỏng hệ thống chống bó cứng phanh. Airbag ECU sử dụng vi điều khiển

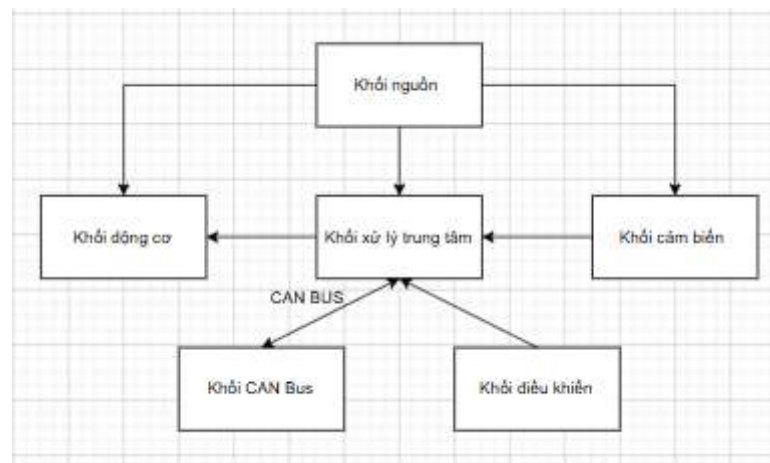
STM32F103C8T6 kết hợp với cảm biến gia tốc LIS331HH để mô phỏng chức năng phát hiện va chạm và kích hoạt cảnh báo.

Ba ECU được kết nối thông qua các transceiver CAN SN65HVD230 hoạt động ở mức điện áp 3.3 V. Hệ thống cho phép thực hiện các kịch bản truyền nhận dữ liệu, tạo lỗi truyền thông có kiểm soát và theo dõi sự thay đổi của các trạng thái Error Active, Error Passive và Bus-Off nhằm phục vụ nghiên cứu cơ chế Fault Confinement trong mạng CAN.

Mô hình được thiết kế theo hướng mở, cho phép dễ dàng thay đổi cấu hình hệ thống, bổ sung ECU hoặc xây dựng các kịch bản thử nghiệm khác nhau phục vụ quá trình đánh giá và phân tích mạng CAN.

3.2.2 Sơ đồ khối hệ thống

Hệ thống được xây dựng từ các khối chức năng chính gồm khối nguồn, khối xử lý trung tâm, khối động cơ, khối cảm biến, khối điều khiển và khối truyền thông CAN Bus. Các khối này phối hợp với nhau để mô phỏng hoạt động của một hệ thống nhiều ECU trên ô tô.



Hình thiết kế hệ thống.1 Sơ đồ khối tổng quát hệ thống

Trong hệ thống, khối xử lý trung tâm giữ vai trò điều phối hoạt động chung và trao đổi dữ liệu với các ECU thông qua mạng CAN. Khối động cơ và khối cảm biến cung cấp dữ liệu thực nghiệm để tạo ra các bản tin CAN phục vụ quá trình nghiên cứu. Khối điều khiển cho phép người sử dụng tác động trực tiếp đến trạng thái hoạt động của hệ thống. Khối nguồn đảm bảo cung cấp điện áp ổn định cho toàn bộ thiết bị phần cứng.

3.2.3 Chức năng các khối trong hệ thống

Khối nguồn có nhiệm vụ cung cấp điện áp cho toàn bộ hệ thống. Nguồn đầu vào 12 V được sử dụng để cấp cho driver động cơ XY-160D và các động cơ DC. Đồng thời, hệ thống sử dụng các mạch hạ áp LM2596 để tạo ra điện áp 5 V và 3.3 V cấp cho màn hình LCD, cảm biến và các module CAN.

Khối xử lý trung tâm sử dụng vi điều khiển STM32F407VG đóng vai trò Main ECU. Khối này thực hiện việc thu thập dữ liệu từ các thiết bị điều khiển, xử lý thông tin, giám sát trạng thái hệ thống và trao đổi dữ liệu với các ECU khác thông qua mạng CAN Bus.

Khối động cơ gồm driver công suất XY-160D và hai động cơ giảm tốc GA25-370 tích hợp encoder Hall. Khối này mô phỏng hoạt động của hệ thống ABS, cho phép tạo dữ liệu tốc độ bánh xe và thực hiện điều khiển chiều quay, tốc độ quay của động cơ.

Khối cảm biến sử dụng cảm biến gia tốc LIS331HH để đo gia tốc theo ba trục. Dữ liệu cảm biến được gửi đến Airbag ECU nhằm mô phỏng chức năng phát hiện va chạm và đánh giá trạng thái hoạt động của hệ thống.

Khối điều khiển bao gồm biến trở ADC, nút Brake và nút Direction. Các thiết bị này cho phép người sử dụng thay đổi các tham số hoạt động và tạo các kịch bản thử nghiệm khác nhau cho hệ thống.

Khối CAN Bus sử dụng các transceiver SN65HVD230 kết nối giữa Main ECU, ABS ECU và Airbag ECU. Khối này đảm nhiệm việc truyền nhận dữ liệu giữa các ECU và là đối tượng chính được nghiên cứu trong đề tài nhằm phân tích cơ chế phát hiện lỗi và Fault Confinement.

3.2.4 Nguyên lý hoạt động của hệ thống.

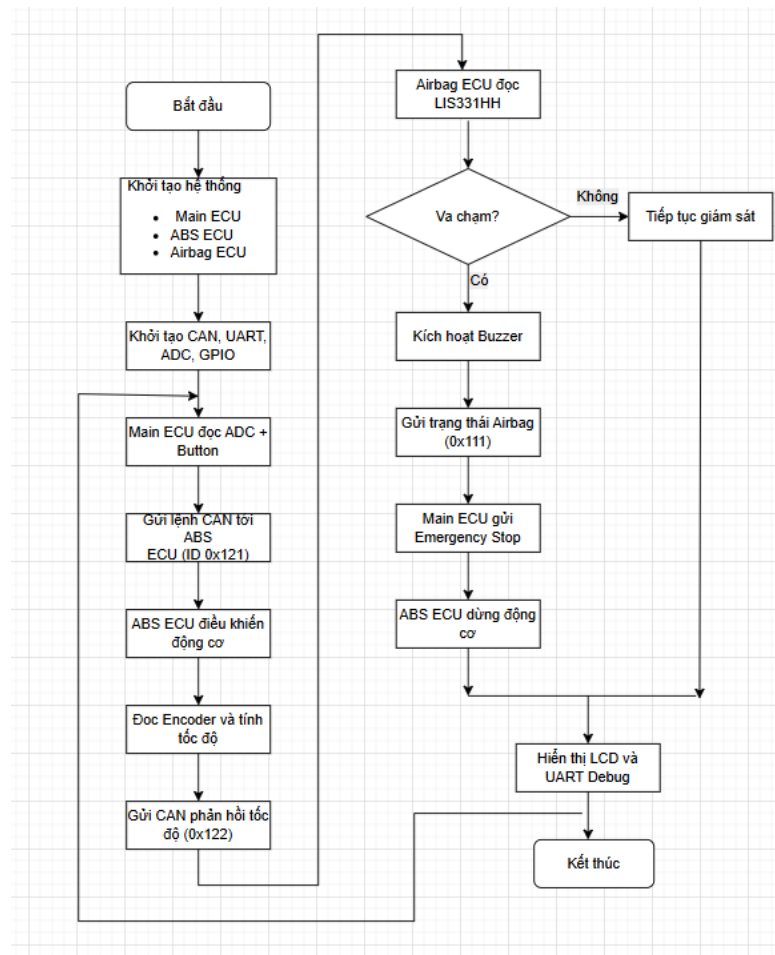
Khi hệ thống được cấp nguồn, các ECU tiến hành khởi tạo ngoại vi, cấu hình bộ điều khiển CAN và tham gia vào mạng truyền thông CAN Bus. Main ECU đóng vai trò trung tâm điều phối hoạt động của hệ thống.

Người sử dụng có thể thay đổi giá trị biến trở ADC hoặc tác động lên các nút Brake và Direction để tạo các trạng thái hoạt động khác nhau. Các thông tin này được Main ECU xử lý và truyền tới các ECU còn lại thông qua các bản tin CAN.

ABS ECU nhận dữ liệu điều khiển từ Main ECU, thực hiện điều khiển driver XY-160D và hai động cơ GA25-370. Đồng thời, dữ liệu encoder từ động cơ được thu thập và gửi ngược trở lại mạng CAN để các ECU khác có thể giám sát.

Airbag ECU liên tục đọc dữ liệu từ cảm biến gia tốc LIS331HH. Khi phát hiện giá trị gia tốc vượt ngưỡng cài đặt, ECU sẽ kích hoạt buzzer cảnh báo và truyền thông tin sự kiện lên mạng CAN.

Trong quá trình hoạt động, các ECU liên tục trao đổi dữ liệu thông qua mạng CAN Bus. Đề tài cho phép tạo các lỗi truyền thông có kiểm soát nhằm khảo sát cơ chế phát hiện lỗi của CAN. Thông qua việc theo dõi các bộ đếm lỗi TEC, REC và thanh ghi trạng thái lỗi của bộ điều khiển CAN trên STM32, hệ thống có thể đánh giá quá trình chuyển đổi giữa các trạng thái Error Active, Error Passive và Bus-Off. Kết quả thu được là cơ sở để phân tích cơ chế Fault Confinement và đánh giá độ tin cậy của mạng CAN trong các hệ thống nhiều ECU.



Hình thiết kế hệ thống.2 Lưu đồ giải thuật của hệ thống

3.3 THIẾT KẾ PHẦN CỨNG

3.3.1 Chức năng của phần cứng

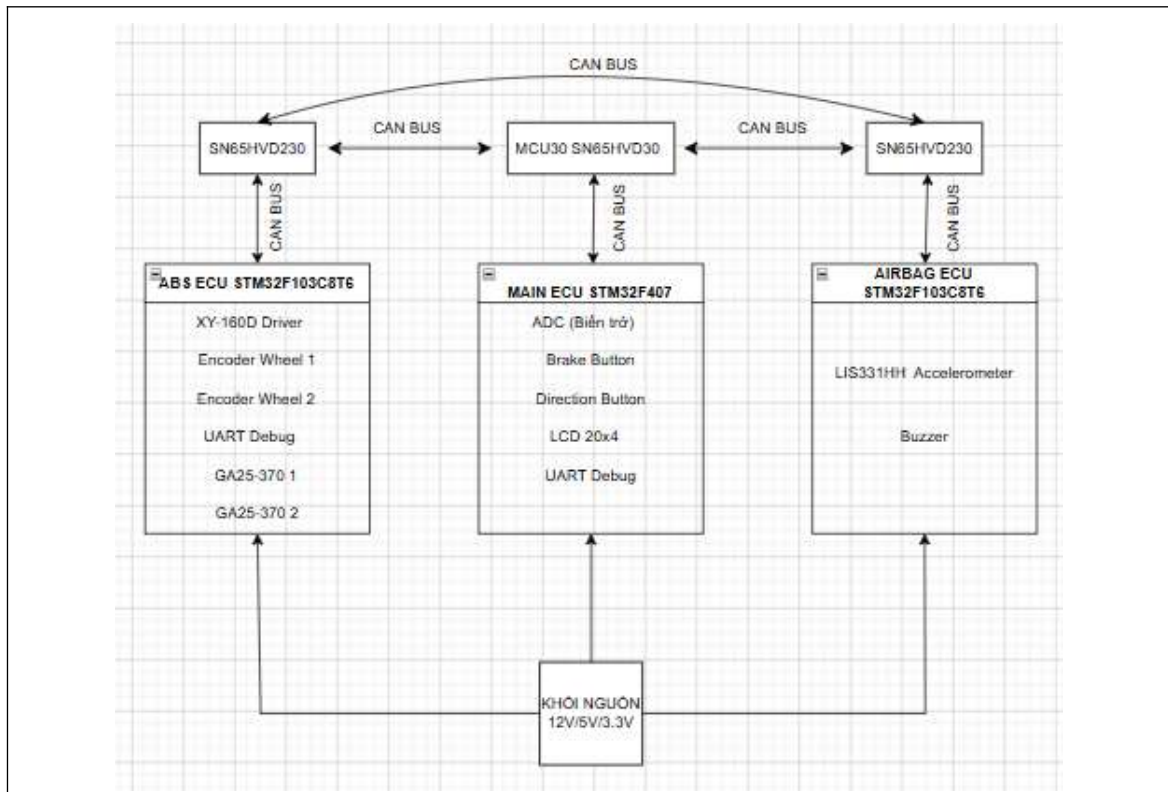
Hệ thống phần cứng được thiết kế nhằm xây dựng mô hình đa ECU giao tiếp CAN theo kiến trúc AUTOSAR Classic thu gọn. Các thành phần phần cứng có nhiệm vụ thu thập tín hiệu đầu vào, xử lý dữ liệu, điều khiển cơ cấu chấp hành, truyền thông giữa các ECU và cung cấp nguồn cho toàn hệ thống. Các chức năng chính của phần cứng bao gồm:

- Thu thập dữ liệu và tín hiệu điều khiển: Hệ thống tiếp nhận các tín hiệu điều khiển từ biến trở và các nút nhấn được kết nối với Main ECU. Biến trở được sử dụng để điều chỉnh tốc độ động cơ, trong khi các nút nhấn dùng để điều khiển phanh và thay đổi chiều quay của động cơ. chuyển.
- Xử lý và trao đổi dữ liệu giữa các ECU: Main ECU, ABS ECU và Airbag ECU thực hiện xử lý dữ liệu cục bộ và trao đổi thông tin thông qua mạng CAN. Các bản tin điều khiển và phản hồi được truyền giữa các ECU nhằm đảm bảo hoạt động đồng bộ của toàn hệ thống.
- Điều khiển động cơ và giám sát tốc độ: ABS ECU điều khiển hai động cơ DC giảm tốc GA25-370 thông qua mạch cầu H XY-160D. Tín hiệu encoder từ động cơ được sử dụng để tính toán tốc độ quay và phản hồi về Main ECU.
- Phát hiện va chạm và cảnh báo: Airbag ECU sử dụng cảm biến gia tốc LIS331HH để thu thập dữ liệu gia tốc theo ba trục. Khi phát hiện gia tốc vượt quá ngưỡng cài đặt, hệ thống sẽ kích hoạt còi cảnh báo và gửi trạng thái va chạm đến Main ECU.
- Hiển thị và giám sát hệ thống: Main ECU hiển thị các thông tin hoạt động của hệ thống trên màn hình LCD 16x2, bao gồm tốc độ động cơ, trạng thái phanh, chiều quay và trạng thái Airbag. Ngoài ra, giao tiếp UART được sử dụng để hỗ trợ quá trình giám sát và gỡ lỗi.
- Đảm bảo cung cấp nguồn cho hệ thống: Khối nguồn đảm bảo cung cấp các mức điện áp 12V, 5V và 3.3V cho toàn bộ hệ thống. Các mức điện áp này được sử dụng tương ứng cho động cơ, các module ngoại vi và vi điều khiển.

Từ các yêu cầu chức năng trên, hệ thống phần cứng được xây dựng gồm các khối chính là khối nguồn, Main ECU, ABS ECU, Airbag ECU, khối điều khiển động cơ, khối cảm biến gia tốc, khối truyền thông CAN và khối hiển thị LCD.

3.3.2 Sơ đồ khối phần cứng

Hệ thống thực nghiệm được thiết kế theo kiến trúc đa ECU, gồm ba bộ điều khiển điện tử (Electronic Control Unit – ECU) giao tiếp với nhau thông qua mạng truyền thông CAN Bus. Các ECU đảm nhiệm những chức năng riêng biệt và phối hợp với nhau để thực hiện việc điều khiển, giám sát và trao đổi dữ liệu trong hệ thống. Sơ đồ khối phần cứng chi tiết của hệ thống được trình bày trong Hình 3.3.



Hình thiết kế hệ thống.3 Sơ đồ khối phần cứng chi tiết

Trong hệ thống, **Main ECU** sử dụng vi điều khiển STM32F407 đóng vai trò bộ điều khiển trung tâm. ECU này thực hiện việc thu thập tín hiệu từ các nút nhấn điều khiển, đọc giá trị biến trở thông qua bộ chuyển đổi tương tự - số (ADC), hiển thị thông tin lên màn hình LCD và quản lý quá trình trao đổi dữ liệu trên mạng CAN. Ngoài ra, Main ECU còn thực hiện chức năng giám sát trạng thái hoạt động của các ECU còn lại và hiển thị kết quả phục vụ quá trình thực nghiệm.

ABS ECU được xây dựng trên nền tảng vi điều khiển STM32F103C8T6. ECU này kết nối với mạch điều khiển động cơ XY-160D để điều khiển hai động cơ giảm tốc GA25-370 có tích hợp encoder. Thông qua tín hiệu phản hồi từ encoder, ABS ECU có khả năng tính toán tốc độ quay của bánh xe và gửi dữ liệu vận hành lên mạng CAN. Khối này được sử dụng để mô phỏng ECU ABS trong hệ thống ô tô và nghiên cứu cơ chế truyền thông thời gian thực giữa các ECU.

Airbag ECU cũng sử dụng vi điều khiển STM32F103C8T6. ECU này kết nối với cảm biến gia tốc ba trục LIS331HH để thu thập dữ liệu gia tốc của hệ thống. Khi giá trị gia tốc vượt ngưỡng cài đặt, Airbag ECU sẽ kích hoạt còi cảnh báo (Buzzer) đồng thời truyền thông tin cảnh báo lên mạng CAN nhằm mô phỏng chức năng phát hiện va chạm và kích hoạt túi khí trên ô tô.

Ba ECU được kết nối với nhau thông qua các module giao tiếp CAN sử dụng transceiver SN65HVD230. Module này thực hiện chuyển đổi tín hiệu CAN mức logic của vi điều khiển sang tín hiệu vi sai CAN_H và CAN_L trên đường truyền vật lý CAN Bus. Việc sử dụng mạng CAN cho phép các ECU trao đổi dữ liệu với độ tin cậy cao, hỗ trợ cơ chế phát hiện lỗi và xử lý lỗi tích hợp trong giao thức.

Khối nguồn cung cấp các mức điện áp cần thiết cho toàn bộ hệ thống. Nguồn đầu vào 12 V được sử dụng để cấp cho driver động cơ và các tải công suất. Từ nguồn 12 V, các mạch hạ áp tạo ra mức điện áp 5 V và 3,3 V để cấp cho các module giao tiếp CAN, màn hình LCD, cảm biến gia tốc và các linh kiện ngoại vi khác. Việc phân tách các mức điện áp giúp hệ thống hoạt động ổn định và đảm bảo an toàn cho các thiết bị điện tử.

Thông qua cấu trúc phần cứng trên, hệ thống có thể mô phỏng một mạng ECU thu nhỏ trong ô tô với các chức năng điều khiển, giám sát và trao đổi dữ liệu theo thời gian thực. Đây là nền tảng để nghiên cứu cơ chế truyền thông CAN Bus, xử lý lỗi truyền thông và đánh giá hoạt động của các ECU trong các điều kiện vận hành khác nhau.

3.3.3 Thiết kế từng khối

3.3.3.1 Khối xử lý trung tâm Main ECU STM32F407

Khối xử lý trung tâm (Main ECU) là thành phần điều khiển chính của hệ thống thực nghiệm mạng CAN Bus. Trong đề tài này, vi điều khiển STM32F407VG được sử dụng làm bộ xử lý trung tâm nhằm thực hiện chức năng điều phối hoạt động giữa các ECU, quản lý truyền thông CAN Bus, thu thập tín hiệu điều khiển từ người dùng và hiển thị trạng thái hoạt động của hệ thống.

STM32F407 thuộc họ vi điều khiển hiệu năng cao STM32F4 của hãng STMicroelectronics, được trang bị nhân xử lý ARM Cortex-M4 hỗ trợ tập lệnh DSP và bộ xử lý dấu chấm động (FPU). Nhờ hiệu năng xử lý cao cùng khả năng tích hợp nhiều ngoại vi truyền thông, STM32F407 phù hợp cho các ứng dụng điều khiển thời gian thực và hệ thống mạng nhúng đa nút. [25].



Hình thiết kế hệ thống.4 Board STM32F407 Discovery

Để lựa chọn bộ điều khiển trung tâm cho hệ thống, nhóm đã khảo sát một số dòng vi điều khiển phổ biến như STM32F103, ESP32 và STM32F407.

Bảng 3.1 So sánh các vi điều khiển

Tiêu chí	STM32F103	ESP32	STM32F407
CPU	Cortex-M3 72MHz	Xtensa 240MHz	Cortex-M4 168MHz
CAN phần cứng	Có	Không	Có
ADC	12 bit	12 bit	12 bit
RAM	20KB	520KB	192KB
Flash	64KB	4MB	1MB
Hiệu năng thời gian thực	Trung bình	Khá	Cao
Ổn định công nghiệp	Cao	Trung bình	Cao

Mặc dù ESP32 có xung nhịp cao và tích hợp WiFi/Bluetooth, nhưng giao tiếp CAN trên ESP32 được triển khai thông qua bộ điều khiển TWAI và không được sử dụng phổ biến trong các hệ thống ECU công nghiệp.

STM32F103 có ưu điểm chi phí thấp nhưng hạn chế về bộ nhớ và hiệu năng xử lý khi phải thực hiện đồng thời nhiều tác vụ như quản lý CAN, xử lý ADC, hiển thị LCD và giám sát lỗi truyền thông.

STM32F407 cung cấp hiệu năng xử lý cao hơn, bộ nhớ lớn hơn và tích hợp bộ điều khiển bxCAN phần cứng hoàn chỉnh. Do đó STM32F407 được lựa chọn làm Main ECU nhằm đảm bảo khả năng xử lý ổn định cho hệ thống thực nghiệm..

Bảng thiết kế hệ thống.2 Thông số STM32F407VG

Thông số	Giá trị
CPU	ARM Cortex-M4
Tần số hoạt động	168 MHz
Flash	1 MB
SRAM	192 KB
ADC	12 bit
CAN	bxCAN
UART	6
SPI	3
I2C	3
GPIO	82
Điện áp hoạt động	1.8V – 3.6V

3.3.3.2 Khối truyền thông CAN Bus

Module giao tiếp CAN MCU230 SN65HVD230

Để thực hiện truyền thông CAN Bus giữa các ECU, hệ thống sử dụng module giao tiếp CAN MCU230 SN65HVD230. Module này sử dụng IC transceiver SN65HVD230 của hãng Texas Instruments, có nhiệm vụ chuyển đổi tín hiệu CAN mức logic từ vi điều khiển sang tín hiệu vi sai CAN_H và CAN_L trên đường truyền vật lý CAN Bus.

Trong hệ thống thực nghiệm, Main ECU STM32F407 sử dụng module 11898MCU230 SN65HVD230 để kết nối với mạng CAN và trao đổi dữ liệu với ABS ECU và Airbag ECU.

Để xây dựng mạng CAN Bus thực nghiệm, nhóm đã khảo sát một số transceiver CAN phổ biến như MCP2551, TJA1050 và SN65HVD230 [26].

Bảng 3.3 So sánh các transceiver CAN

Tiêu chí	MCP2551	TJA1050	SN65HVD230
Điện áp hoạt động	5V	5V	5V
Tương thích STM32 3.3V	Cần chuyển mức	Cần chuyển mức	Trực tiếp
Tốc độ CAN tối đa	1 Mbps	1 Mbps	1 Mbps
Công suất tiêu thụ	Trung bình	Trung bình	Thấp
Khả năng chống nhiễu	Tốt	Tốt	Tốt

SN65HVD230 được lựa chọn vì hoạt động trực tiếp ở mức điện áp 3.3V, tương thích hoàn toàn với các vi điều khiển STM32F407 và STM32F103 trong hệ thống mà không cần mạch chuyển mức điện áp. Điều này giúp đơn giản hóa thiết kế phần cứng, giảm chi phí và tăng độ ổn định của hệ thống.

**Hình thiết kế hệ thống.5 Module MCU230 SN65HVD230****Bảng thiết kế hệ thống.4 Thông số kỹ thuật module MCU230 SN65HVD230**

Thông số	Giá trị
IC transceiver	SN65HVD230
Điện áp hoạt động	3.0V – 3.6V
Điện áp danh định	3.3V
Tốc độ CAN tối đa	1 Mbps
Chuẩn CAN	ISO 11898
Chế độ hoạt động	High Speed CAN

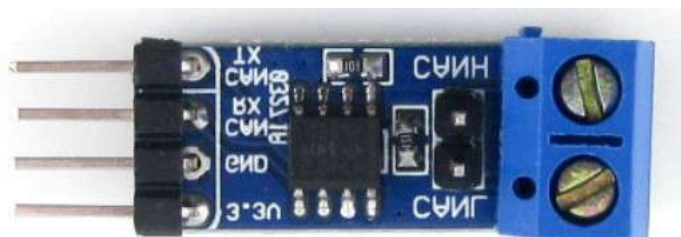
Module giao tiếp CAN SN65HVD230 của ABS ECU và Airbag ECU

ABS ECU và Airbag ECU sử dụng module giao tiếp CAN SN65HVD230 để kết nối với mạng CAN Bus. Các module này đảm nhiệm vai trò lớp vật lý của mạng CAN, cho phép các ECU truyền và nhận dữ liệu thông qua hai đường tín hiệu CAN_H và CAN_L.

Việc sử dụng cùng một loại transceiver trên cả ba ECU giúp đảm bảo tính đồng nhất của hệ thống, giảm sai khác điện áp và tăng độ tin cậy trong quá trình truyền thông.

Các ECU trong hệ thống đều sử dụng vi điều khiển STM32 hoạt động ở mức điện áp 3.3V. Do đó việc sử dụng SN65HVD230 cho tất cả các ECU giúp:

- Không cần mạch chuyển mức điện áp.
- Giảm số lượng linh kiện.
- Đơn giản hóa thiết kế PCB.
- Đảm bảo khả năng tương thích giữa các ECU.
- Thuận lợi cho việc nghiên cứu lỗi truyền thông CAN và cơ chế Fault Confinement.



Hình 3.6 Module SN65HVD230 sử dụng trên ABS ECU và Airbag ECU

Bảng 3.5 Thông số chính của module SN65HVD230

Thông số	Giá trị
Điện áp hoạt động	3.3V
Chuẩn CAN	ISO 11898
Tốc độ tối đa	1 Mbps
Tín hiệu vật lý	CAN_H, CAN_L
Chế độ truyền	Vi sai

3.3.3.3 Khối ECU chức năng STM32F103C8T6

Khối ECU chức năng trong hệ thống bao gồm ABS ECU và Airbag ECU, đều được xây dựng trên nền tảng vi điều khiển STM32F103C8T6. Hai ECU này đảm nhiệm việc thu thập dữ liệu từ các cảm biến, xử lý thông tin và trao đổi dữ liệu với Main ECU thông qua mạng CAN Bus.

Trong đó:

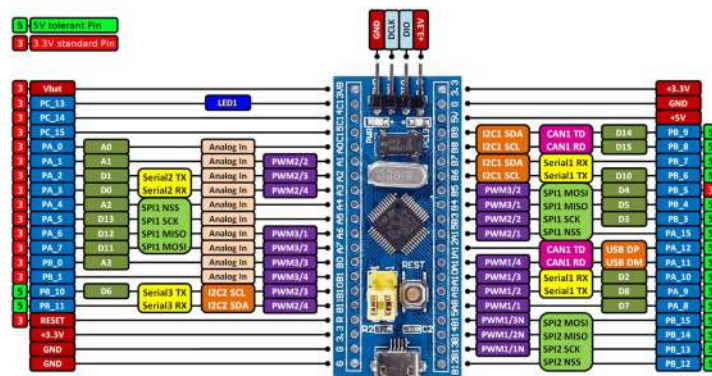
- ABS ECU thực hiện điều khiển động cơ và thu thập tín hiệu encoder để xác định tốc độ bánh xe.
- Airbag ECU thực hiện thu thập dữ liệu từ cảm biến gia tốc LIS331HH nhằm mô phỏng chức năng phát hiện va chạm.

Việc sử dụng cùng một loại vi điều khiển cho hai ECU giúp đơn giản hóa thiết kế phần cứng, giảm chi phí và thuận lợi trong quá trình phát triển phần mềm.

STM32F103C8T6 được lựa chọn do:

- Tích hợp CAN Controller (bxCAN).
- Hỗ trợ Timer Encoder Interface.
- Hỗ trợ I2C cho cảm biến LIS331HH.
- Chi phí thấp.
- Tương thích trực tiếp với SN65HVD230.

Những đặc điểm trên đáp ứng đầy đủ yêu cầu của cả ABS ECU và Airbag ECU.



Hình thiết kế hệ thống.7 Board STM32F103C8T6

Bảng thiết kế hệ thống.6 Thông số kỹ thuật board STM32F103C8T6

Thông số	Giá trị
Vi điều khiển	STM32F103C8T6
Nhân xử lý	ARM Cortex-M3
Tần số hoạt động	72 MHz
Flash	64 KB
SRAM	20 KB
Điện áp hoạt động	2.0V – 3.6V
ADC	12 bit
CAN	bxCAN
Timer	4
I2C	2
UART	3

3.3.3.4 Khối cảm biến

Trong hệ thống thực nghiệm, cảm biến gia tốc LIS331HH được sử dụng để mô phỏng chức năng phát hiện va chạm của hệ thống túi khí (Airbag). Cảm biến có khả năng đo gia tốc theo ba trục X, Y và Z, cho phép phát hiện các thay đổi đột ngột về gia tốc của hệ thống.

LIS331HH là cảm biến gia tốc MEMS 3 trục do hãng STMicroelectronics phát triển. Cảm biến hỗ trợ nhiều dải đo khác nhau và giao tiếp với vi điều khiển thông qua giao thức I2C hoặc SPI. Trong đề tài này, cảm biến được kết nối với Airbag ECU thông qua giao tiếp I2C để thu thập dữ liệu gia tốc theo thời gian thực [27].

Để lựa chọn cảm biến gia tốc cho Airbag ECU, nhóm đã khảo sát một số cảm biến phổ biến như MPU6050, ADXL345 và LIS331HH.

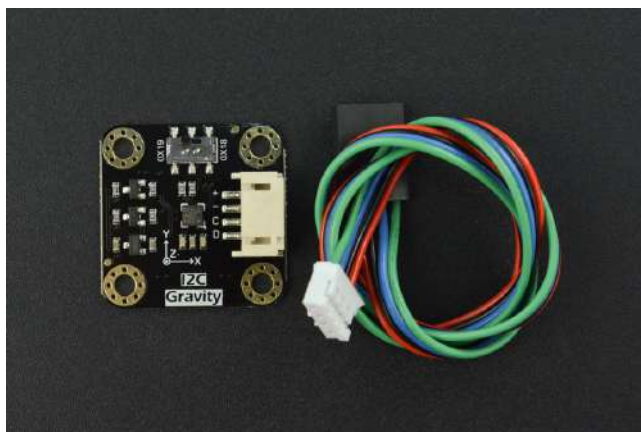
Bảng 3.7 So sánh các cảm biến gia tốc

Tiêu chí	MPU6050	ADXL345	LIS331HH
Số trục đo	6 trục	3 trục	3 trục

Dải đo tối đa	$\pm 16g$	$\pm 16g$	$\pm 24g$
Giao tiếp	I2C	I2C/SPI	I2C/SPI
Điện áp hoạt động	3.3V	3.3V	3.3V
Ứng dụng công nghiệp	Trung bình	Khá	Cao

Từ kết quả so sánh có thể thấy LIS331HH có dải đo lớn hơn so với MPU6050 và ADXL345, phù hợp với các ứng dụng phát hiện rung động mạnh hoặc mô phỏng va chạm. Ngoài ra, cảm biến được sản xuất bởi STMicroelectronics nên có khả năng tương thích tốt với nền tảng STM32 được sử dụng trong đề tài.

Do đó, LIS331HH được lựa chọn để xây dựng Airbag ECU trong hệ thống thực nghiệm.



Hình thiết kế hệ thống.8 Cảm biến LIS331HH

Bảng thiết kế hệ thống.8 Thông số kỹ thuật của LIS331HH

Thông số	Giá trị
Cảm biến	LIS331HH
Số trục đo	3 trục
Dải đo	$\pm 6g, \pm 12g, \pm 24g$
Điện áp hoạt động	2.16V – 3.6V
Giao tiếp	I2C, SPI
Tần số lấy mẫu tối đa	12 bit
Tần số lấy mẫu	Lên đến 1000 Hz

3.3.3.5 Khối động cơ và Encoder GA25-370

Trong hệ thống thực nghiệm, động cơ giảm tốc GA25-370 được sử dụng để mô phỏng chuyển động của bánh xe trong hệ thống ABS. Động cơ được tích hợp encoder Hall hai kênh A và B, cho phép vi điều khiển đo tốc độ quay và xác định chiều quay của động cơ.

Thông tin tốc độ thu được từ encoder được ABS ECU xử lý và truyền lên mạng CAN Bus để Main ECU giám sát và hiển thị. Việc sử dụng động cơ tích hợp encoder giúp hệ thống có khả năng mô phỏng các tín hiệu tốc độ bánh xe tương tự như trên các hệ thống ABS thực tế [28].

Để xây dựng mô hình ABS, nhóm đã khảo sát một số loại động cơ phổ biến như động cơ DC thường, động cơ Servo và động cơ GA25-370 tích hợp encoder.

Bảng 3.9 So sánh các loại động cơ

Tiêu chí	DC thường	Servo Motor	GA25-370 Encoder
Điều khiển tốc độ	Có	Có	Có
Phản hồi vị trí	Không	Có	Có
Encoder tích hợp	Không	Có	Có
Dễ đọc tốc độ	Không	Trung bình	Cao
Giá thành	Thấp	Cao	Trung bình
Phù hợp mô phỏng ABS	Thấp	Trung bình	Cao

GA25-370 được lựa chọn vì:

- Tích hợp encoder Hall hai kênh A/B.
- Dễ dàng đo tốc độ bằng Timer Encoder trên STM32.
- Kích thước nhỏ gọn.
- Điện áp hoạt động 12V phù hợp với hệ thống.
- Giá thành hợp lý.
- Thuận lợi cho việc mô phỏng tốc độ bánh xe trong nghiên cứu mạng CAN.



Hình thiết kế hệ thống.9 Động cơ giảm tốc GA25-370 tích hợp encoder.

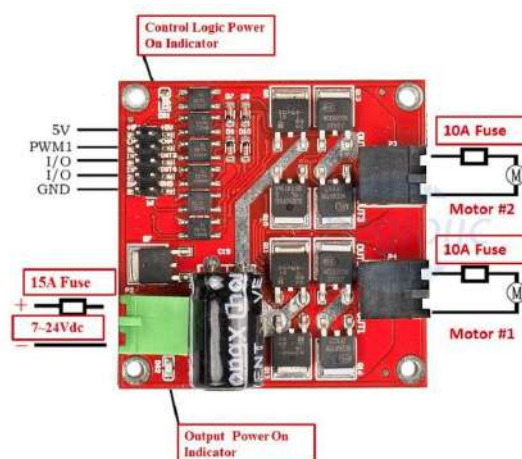
Bảng thiết kế hệ thống.10 Thông số kỹ thuật của động cơ GA25-370

Thông số	Giá trị
Model	GA25-370
Điện áp định mức	12V
Tốc độ không tải	280 RPM
Loại động cơ	DC Geared Motor
Encoder	Hall Encoder
Số kênh Encoder	2 (A, B)
Tín hiệu Encoder	Xung vuông

Mạch điều khiển động cơ XY-160D

Trong hệ thống thực nghiệm, mạch điều khiển động cơ XY-160D được sử dụng để điều khiển hai động cơ giảm tốc GA25-370 của ABS ECU. Mạch có chức năng khuếch đại công suất tín hiệu điều khiển từ vi điều khiển STM32F103C8T6, cho phép điều khiển tốc độ và chiều quay của động cơ thông qua tín hiệu PWM và tín hiệu điều khiển chiều.

XY-160D là module điều khiển động cơ DC sử dụng cấu trúc cầu H (H-Bridge), hỗ trợ điều khiển hai động cơ độc lập hoặc một động cơ công suất lớn. Module có khả năng làm việc với điện áp từ 6.5V đến 27V và đáp ứng tốt các ứng dụng điều khiển động cơ trong hệ thống nhúng và robot.



Hình 3.10 Mạch điều khiển động cơ XY-160D.

XY-160D được lựa chọn nhằm đáp ứng yêu cầu điều khiển hai động cơ đồng thời với chi phí hợp lý và độ ổn định cao.

Bảng 3.11 Thông số chính của XY-160D

Thông số	Giá trị
Model	XY-160D
Điện áp hoạt động	6.5V – 27V
Dòng tải tối đa	7A
Số kênh điều khiển	2
Điều khiển tốc độ	PWM
Điều khiển chiều quay	Có
Loại mạch	Cầu H MOSFET

3.3.3.6 Khối điều khiển và hiển thị

Khối giao diện người dùng giúp người vận hành theo dõi trạng thái hoạt động của hệ thống và thực hiện các thao tác điều khiển trong quá trình thực nghiệm. Trong đề tài này, khối giao diện bao gồm màn hình LCD 16x2 giao tiếp I2C và các nút nhấn điều khiển. Thông qua các thiết bị này, người dùng có thể quan sát dữ liệu hệ thống, thay đổi trạng thái hoạt động và tạo các kịch bản thử nghiệm phục vụ nghiên cứu cơ chế Fault Confinement trên mạng CAN.

Màn hình LCD 16x2 I2C

Màn hình LCD 16x2 I2C được sử dụng để hiển thị các thông tin hoạt động của hệ thống như tốc độ động cơ, trạng thái các ECU, dữ liệu CAN và các thông báo

lỗi. Màn hình có khả năng hiển thị 16 ký tự trên mỗi dòng và 2 dòng ký tự, đáp ứng tốt nhu cầu giám sát trong quá trình thực nghiệm.

LCD được tích hợp module chuyển đổi I2C sử dụng IC PCF8574, giúp giảm số lượng chân kết nối với vi điều khiển từ nhiều chân song song xuống chỉ còn hai đường tín hiệu SDA và SCL. Phần lớn các LCD 16x2 I2C hiện nay sử dụng bộ điều khiển HD44780 kết hợp với bộ chuyển đổi PCF8574.



Hình 3.11 Màn hình LCD 16x2 I2C.

Bảng 3.12 Thông số chính của LCD 16x2 I2C

Thông số	Giá trị
Kích thước hiển thị	16 x 2 ký tự
Điện áp hoạt động	5 V
Bộ điều khiển LCD	HD44780
IC chuyển đổi I2C	PCF8574
Giao tiếp	I2C
Màu nền	Vàng

Nút nhấn điều khiển

Nút nhấn được sử dụng để mô phỏng thao tác điều khiển của người sử dụng. Các tín hiệu từ nút nhấn được đưa trực tiếp vào các chân GPIO của vi điều khiển STM32F407 để thực hiện các chức năng điều khiển hệ thống.

Hai nút nhấn chính được sử dụng trong đề tài gồm:

- Nút Brake.

- Nút Direction.

So với các thiết bị nhập liệu khác như bàn phím ma trận hoặc màn hình cảm ứng, nút nhấn có cấu tạo đơn giản, chi phí thấp và dễ tích hợp vào hệ thống nhúng



Hình 3.12 Nút nhấn.

Biến trở điều chỉnh tốc độ động cơ

Biến trở $10\text{ k}\Omega$ được sử dụng để tạo tín hiệu điện áp tương tự (Analog Voltage) đưa vào bộ chuyển đổi tương tự – số (ADC) của vi điều khiển STM32F407. Giá trị điện áp thu được sẽ được chuyển đổi thành giá trị số và sử dụng để điều chỉnh tốc độ quay của động cơ trong quá trình thực nghiệm.

Việc sử dụng biến trở giúp người vận hành có thể thay đổi tốc độ động cơ một cách liên tục và trực quan, đồng thời tạo ra các mức dữ liệu khác nhau để truyền trên mạng CAN Bus..



Hình thiết kế hệ thống.13 Biến trở 10 k Ω .

3.3.4 Thiết kế mạch điều khiển

Để kết nối các thành phần phần cứng và xây dựng mô hình mạng CAN đa ECU phục vụ nghiên cứu cơ chế Fault Confinement, nhóm đã thiết kế các mạch điều khiển cho Main ECU, ABS ECU và Airbag ECU dựa trên nền tảng vi điều khiển STM32. Các mạch được thiết kế trên phần mềm KiCad nhằm đảm bảo tính ổn định, dễ dàng mở rộng và thuận tiện cho việc thực nghiệm.

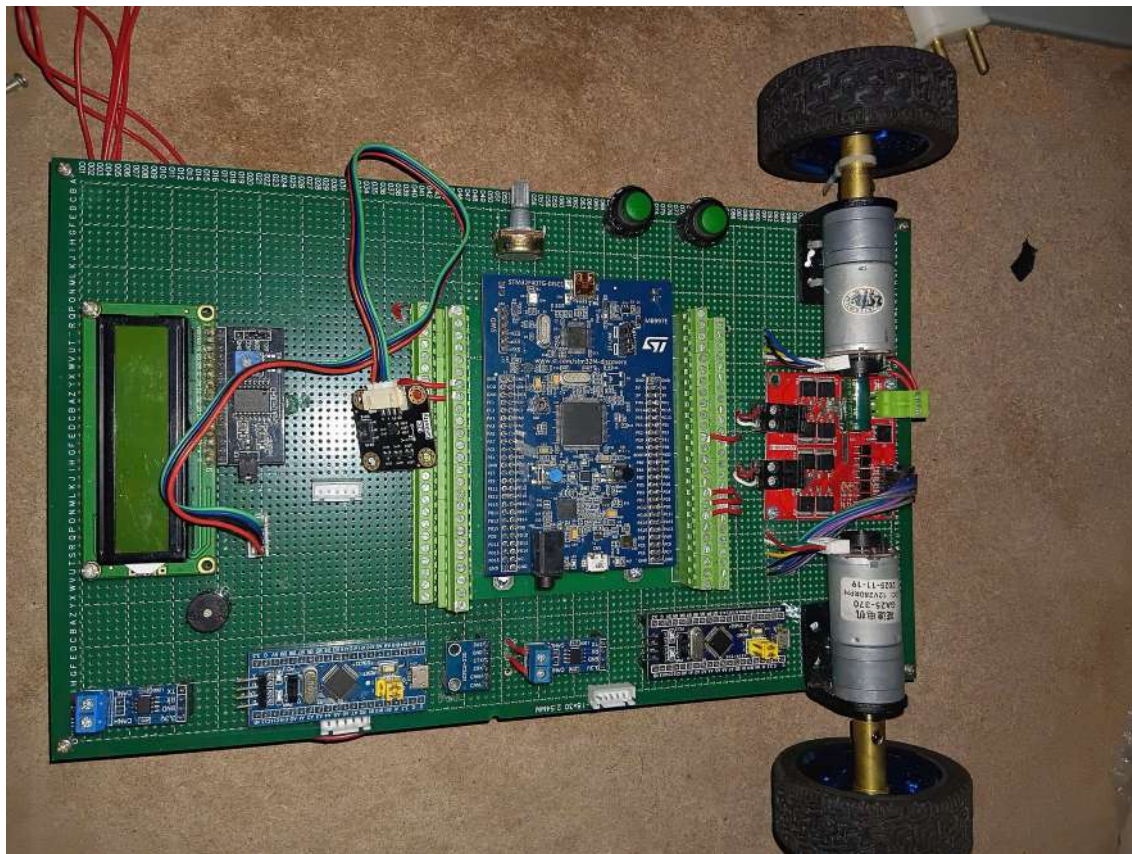
Mỗi ECU được xây dựng gồm các khối chức năng chính như vi điều khiển STM32, bộ chuyển đổi vật lý CAN SN65HVD230, mạch nguồn và các thiết bị ngoại vi tương ứng với từng chức năng. Main ECU sử dụng STM32F407VG làm bộ điều khiển trung tâm, kết nối với LCD 16x2, biến trở điều chỉnh tốc độ và các nút nhấn điều khiển. ABS ECU sử dụng STM32F103C8T6 kết hợp với driver XY-160D và động cơ encoder GA25-370 để mô phỏng hệ thống ABS. Airbag ECU sử dụng STM32F103C8T6 kết nối với cảm biến gia tốc LIS331HH và buzzer để mô phỏng hệ thống phát hiện va chạm.

Các ECU được kết nối với nhau thông qua mạng CAN Bus sử dụng transceiver SN65HVD230. Đường truyền CAN_H và CAN_L được bố trí theo cấu trúc bus nhằm mô phỏng kiến trúc truyền thông CAN trên các hệ thống điện tử ô tô thực tế. Việc sử dụng chuẩn truyền thông CAN giúp hệ thống có khả năng chống nhiễu tốt, hỗ trợ phát hiện lỗi và đảm bảo độ tin cậy trong quá trình truyền nhận dữ liệu.

Mạch điều khiển được thiết kế theo dạng PCB hai lớp. Các khối nguồn, vi điều khiển và khối truyền thông CAN được bố trí tách biệt nhằm giảm ảnh hưởng của nhiễu điện từ phát sinh trong quá trình hoạt động. Các đầu nối tín hiệu sử

dụng Header chuẩn 2.54 mm giúp thuận tiện trong việc lắp ráp, bảo trì và thay thế linh kiện khi thực hiện các thí nghiệm.

Thông qua thiết kế này, hệ thống không chỉ đáp ứng yêu cầu truyền thông CAN giữa nhiều ECU mà còn tạo điều kiện thuận lợi cho việc theo dõi các tham số lỗi như TEC, REC, LEC, EWGF, EPVF và BOFF nhằm phục vụ nghiên cứu cơ chế Fault Confinement trên mạng CAN.



Hình thiết kế hệ thống.14 Phần cứng hệ thống

3.4 THIẾT KẾ PHẦN MỀM

Bên cạnh phần cứng, phần mềm đóng vai trò quyết định đến khả năng vận hành và phối hợp giữa các ECU trong hệ thống. Trong đề tài này, phần mềm được xây dựng theo kiến trúc AUTOSAR Classic thu gọn nhằm tăng tính mô-đun, khả năng mở rộng và thuận lợi cho việc quản lý các chức năng của từng ECU.

Hệ thống phần mềm được triển khai trên ba ECU gồm Main ECU, ABS ECU và Airbag ECU. Các ECU thực hiện trao đổi dữ liệu thông qua mạng truyền thông CAN Bus, đồng thời phối hợp thực hiện các chức năng điều khiển, giám

sát và xử lý lỗi. Ngoài việc đảm bảo khả năng truyền nhận dữ liệu thời gian thực, phần mềm còn tích hợp các thuật toán xử lý tín hiệu cảm biến, tính toán tốc độ bánh xe, phát hiện va chạm, giám sát trạng thái truyền thông và đánh giá cơ chế Fault Confinement của giao thức CAN.

Nội dung phần này tập trung trình bày mục tiêu thiết kế phần mềm, kiến trúc hệ thống phần mềm, nguyên lý hoạt động của từng ECU, cấu trúc truyền thông CAN cũng như các thuật toán chính được triển khai trong hệ thống..

3.4.1 Mục tiêu thiết kế phần mềm

Phần mềm của hệ thống được thiết kế nhằm xây dựng một mô hình mạng CAN đa ECU hoạt động theo thời gian thực, phục vụ nghiên cứu cơ chế truyền thông và xử lý lỗi trong mạng CAN trên nền tảng vi điều khiển STM32.

Các mục tiêu chính của quá trình thiết kế phần mềm bao gồm:

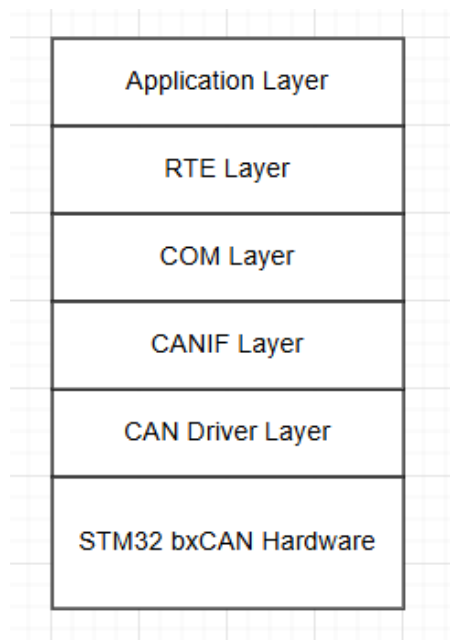
- Xây dựng kiến trúc phần mềm đa ECU theo hướng AUTOSAR Classic thu gọn, cho phép phân tách rõ ràng giữa các tầng ứng dụng và tầng truyền thông.
- Thiết lập cơ chế truyền thông CAN giữa Main ECU, ABS ECU và Airbag ECU thông qua các bản tin CAN được định nghĩa trước.
- Thu thập và xử lý dữ liệu từ các thiết bị ngoại vi như biến trở điều chỉnh tốc độ, nút nhấn điều khiển, encoder động cơ và cảm biến gia tốc LIS331HH.
- Xây dựng thuật toán điều khiển động cơ tại ABS ECU dựa trên tín hiệu điều khiển nhận được từ Main ECU.
- Xây dựng thuật toán tính toán tốc độ bánh xe từ tín hiệu encoder nhằm mô phỏng chức năng của hệ thống ABS trên ô tô.
- Xây dựng thuật toán phát hiện va chạm dựa trên dữ liệu gia tốc ba trục từ cảm biến LIS331HH nhằm mô phỏng chức năng của Airbag ECU.
- Thực hiện cơ chế giám sát truyền thông thông qua CAN Timeout, Sequence Counter và các bản tin chẩn đoán nhằm phát hiện các bất thường trong quá trình trao đổi dữ liệu.
- Theo dõi và đánh giá các trạng thái lỗi của bộ điều khiển CAN thông qua các tham số TEC, REC, LEC, EWGF, EPVF và BOFF để nghiên cứu cơ chế Fault Confinement theo tiêu chuẩn CAN.
- Đảm bảo hệ thống hoạt động ổn định, có khả năng mở rộng và thuận lợi cho việc thực hiện các kịch bản thực nghiệm liên quan đến mạng CAN và hệ thống đa ECU.

Thông qua các mục tiêu trên, phần mềm được xây dựng không chỉ đáp ứng yêu cầu vận hành của mô hình thực nghiệm mà còn tạo nền tảng cho việc nghiên cứu, phân tích và đánh giá hoạt động của mạng CAN trong các hệ thống nhúng và điện tử ô tô hiện đại..

3.4.2 Cấu trúc hệ thống phần mềm

Để tăng tính mô-đun, khả năng mở rộng và thuận lợi cho việc phát triển phần mềm trên nhiều ECU, hệ thống được xây dựng theo kiến trúc AUTOSAR Classic thu gọn. Kiến trúc này được đơn giản hóa từ mô hình AUTOSAR truyền thống nhưng vẫn giữ lại các thành phần quan trọng như tầng ứng dụng, tầng trung gian và tầng truyền thông.

Việc áp dụng kiến trúc phân lớp giúp tách biệt giữa phần mềm ứng dụng và phần cứng, đồng thời tạo điều kiện thuận lợi cho việc bảo trì, mở rộng và tái sử dụng mã nguồn trong các hệ thống nhúng nhiều ECU.



Hình 3.15 Kiến trúc phần mềm tổng thể của hệ thống

Trong kiến trúc trên, dữ liệu được truyền từ tầng ứng dụng xuống tầng phần cứng khi gửi bản tin CAN và truyền ngược lại từ phần cứng lên tầng ứng dụng khi nhận bản tin CAN. Mỗi tầng đảm nhận một chức năng riêng biệt nhằm giảm sự phụ thuộc giữa các thành phần phần mềm.

Tầng Application Layer

Đây là tầng cao nhất của hệ thống và chứa toàn bộ các chức năng nghiệp vụ của từng ECU.

Trong đề tài, tầng Application Layer bao gồm:

- Main ECU Application
- ABS ECU Application
- Airbag ECU Application

Tầng này không làm việc trực tiếp với phần cứng CAN mà sử dụng các dịch vụ từ các tầng bên dưới.

Tầng RTE (Runtime Environment)

RTE đóng vai trò là tầng trung gian giữa Application Layer và COM Layer. Tầng này cung cấp các hàm giao tiếp chuẩn hóa giúp các ứng dụng không cần truy cập trực tiếp vào các tầng truyền thông.

Trong hệ thống, RTE được sử dụng để:

- Trao đổi dữ liệu giữa các Software Component.
- Truy xuất dữ liệu cảm biến.
- Gửi dữ liệu điều khiển.
- Nhận dữ liệu từ các ECU khác.

Việc sử dụng RTE giúp tăng tính độc lập giữa phần mềm ứng dụng và phần cứng bên dưới.

Tầng COM Layer

COM Layer thực hiện nhiệm vụ quản lý dữ liệu truyền thông giữa các ECU.

Các chức năng chính bao gồm:

- Đóng gói dữ liệu ứng dụng thành CAN Frame.
- Giải mã dữ liệu nhận được từ CAN Frame.
- Quản lý CAN ID.
- Quản lý Sequence Counter.
- Quản lý các bản tin chẩn đoán.

Trong đề tài, COM Layer đóng vai trò quan trọng trong việc trao đổi dữ liệu giữa Main ECU, ABS ECU và Airbag ECU.

Tầng CANIF Layer

CANIF (CAN Interface) là tầng trung gian giữa COM Layer và CAN Driver.

Nhiệm vụ chính của tầng này gồm:

- Chuyển đổi dữ liệu từ COM Layer sang CAN Driver.
- Quản lý Mailbox truyền nhận.
- Quản lý Filter.
- Điều phối các bản tin CAN.

Tầng CANIF giúp các tầng phía trên không phụ thuộc vào phần cứng CAN cụ thể của vi điều khiển.

Tầng CAN Driver Layer

CAN Driver là tầng điều khiển trực tiếp bộ điều khiển CAN tích hợp bên trong vi điều khiển STM32.

Các chức năng chính:

- Khởi tạo CAN.
- Cấu hình Bit Timing.
- Cấu hình Filter.
- Quản lý Mailbox.
- Gửi và nhận dữ liệu CAN.
- Đọc trạng thái lỗi CAN.

Trong đề tài, CAN Driver được xây dựng ở mức thanh ghi nhằm khai thác trực tiếp bộ điều khiển bxCAN của STM32F407 và STM32F103.

Tầng phần cứng bxCAN

Đây là tầng thấp nhất trong hệ thống, bao gồm bộ điều khiển CAN tích hợp bên trong vi điều khiển STM32.

Bộ điều khiển bxCAN chịu trách nhiệm:

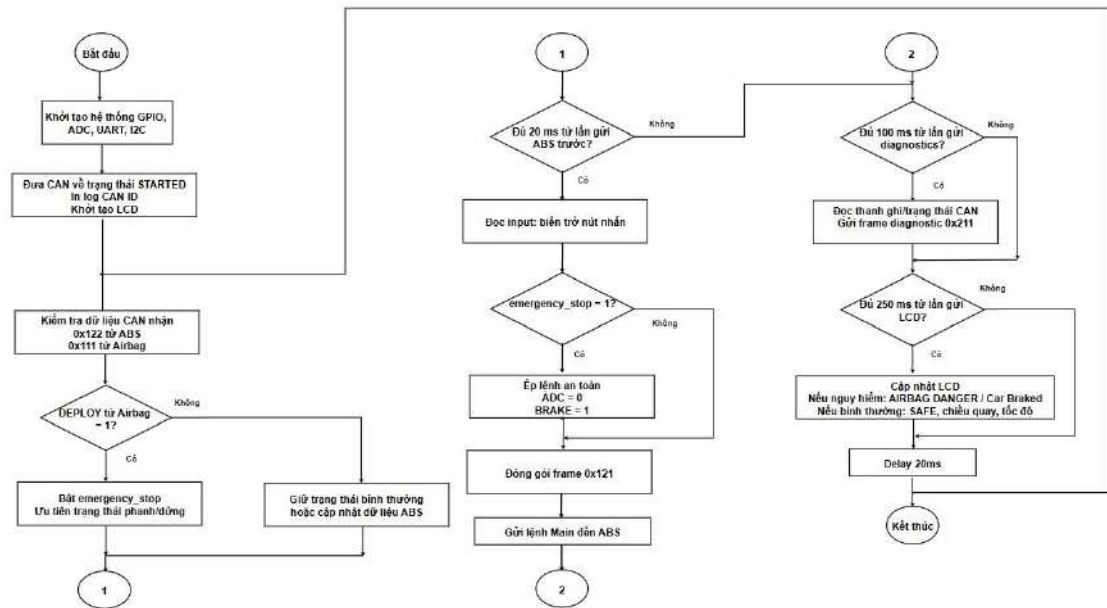
- Tạo khung CAN.
- Arbitration.
- CRC.
- Bit Stuffing.
- ACK.
- Error Detection.
- Fault Confinement.

Các trạng thái lỗi như Error Active, Error Passive và Bus-Off đều được bộ điều khiển bxCAN quản lý theo tiêu chuẩn CAN.

Thông qua kiến trúc phân lớp trên, hệ thống phần mềm đạt được tính mô-đun cao, thuận lợi cho việc phát triển nhiều ECU độc lập và đáp ứng yêu cầu nghiên cứu cơ chế truyền thông CAN cũng như Fault Confinement trong hệ thống đa ECU..

3.4.3 Thiết kế phần mềm Main ECU

Main ECU là bộ điều khiển trung tâm của hệ thống, chịu trách nhiệm thu thập dữ liệu từ người dùng, quản lý quá trình truyền thông CAN giữa các ECU và giám sát trạng thái hoạt động của toàn hệ thống. Phần mềm Main ECU được xây dựng theo mô hình xử lý tuần hoàn (Cyclic Executive), trong đó các tác vụ được thực hiện liên tục theo chu kỳ xác định nhằm đảm bảo tính thời gian thực của hệ thống.



Hình 3.16 Lưu đồ thuật toán của Main ECU.

Các chức năng chính của Main ECU bao gồm:

- Đọc tín hiệu ADC từ biến trở điều chỉnh tốc độ.
- Đọc trạng thái nút nhấn Brake.
- Đọc trạng thái nút nhấn Direction.
- Đóng gói dữ liệu điều khiển và gửi tới ABS ECU.
- Nhận dữ liệu tốc độ bánh xe từ ABS ECU.
- Nhận dữ liệu va chạm từ Airbag ECU.
- Hiển thị thông tin hệ thống trên LCD.
- Giám sát trạng thái truyền thông CAN.
- Thực hiện chức năng Emergency Stop khi phát hiện va chạm.

Thu thập tín hiệu điều khiển

Main ECU liên tục đọc tín hiệu từ biến trở thông qua bộ chuyển đổi ADC 12 bit của STM32F407. Giá trị ADC thu được có dải từ 0 đến 4095 và được sử dụng để xác định tốc độ mong muốn của động cơ.

Bên cạnh đó, Main ECU còn đọc trạng thái của hai nút nhấn gồm Brake và Direction. Các tín hiệu này được sử dụng để xác định chế độ điều khiển của ABS ECU.

Dữ liệu điều khiển được lưu trữ vào bộ nhớ đệm trước khi đóng gói thành bản tin CAN.

Đóng gói và truyền bản tin CAN

Sau khi thu thập dữ liệu điều khiển, Main ECU thực hiện đóng gói dữ liệu thành CAN Frame có ID 0x121 và gửi đến ABS ECU.

Cấu trúc bản tin gồm:

- Giá trị ADC.
- Trạng thái Brake.
- Trạng thái Direction.
- Sequence Counter.

Việc sử dụng Sequence Counter giúp phát hiện hiện tượng mất bản tin trong quá trình truyền thông.

Thuật toán Sequence Counter

Sequence Counter là bộ đếm tăng dần được gắn vào mỗi bản tin CAN do Main ECU truyền đi.

Tại phía nhận, ABS ECU sẽ kiểm tra giá trị Sequence Counter hiện tại với giá trị mong đợi. Nếu xuất hiện sự sai lệch giữa hai giá trị này, hệ thống xác định đã xảy ra hiện tượng mất bản tin.

Thuật toán này giúp giám sát độ tin cậy của quá trình truyền thông CAN và hỗ trợ đánh giá hoạt động của mạng trong các thử nghiệm Fault Confinement.

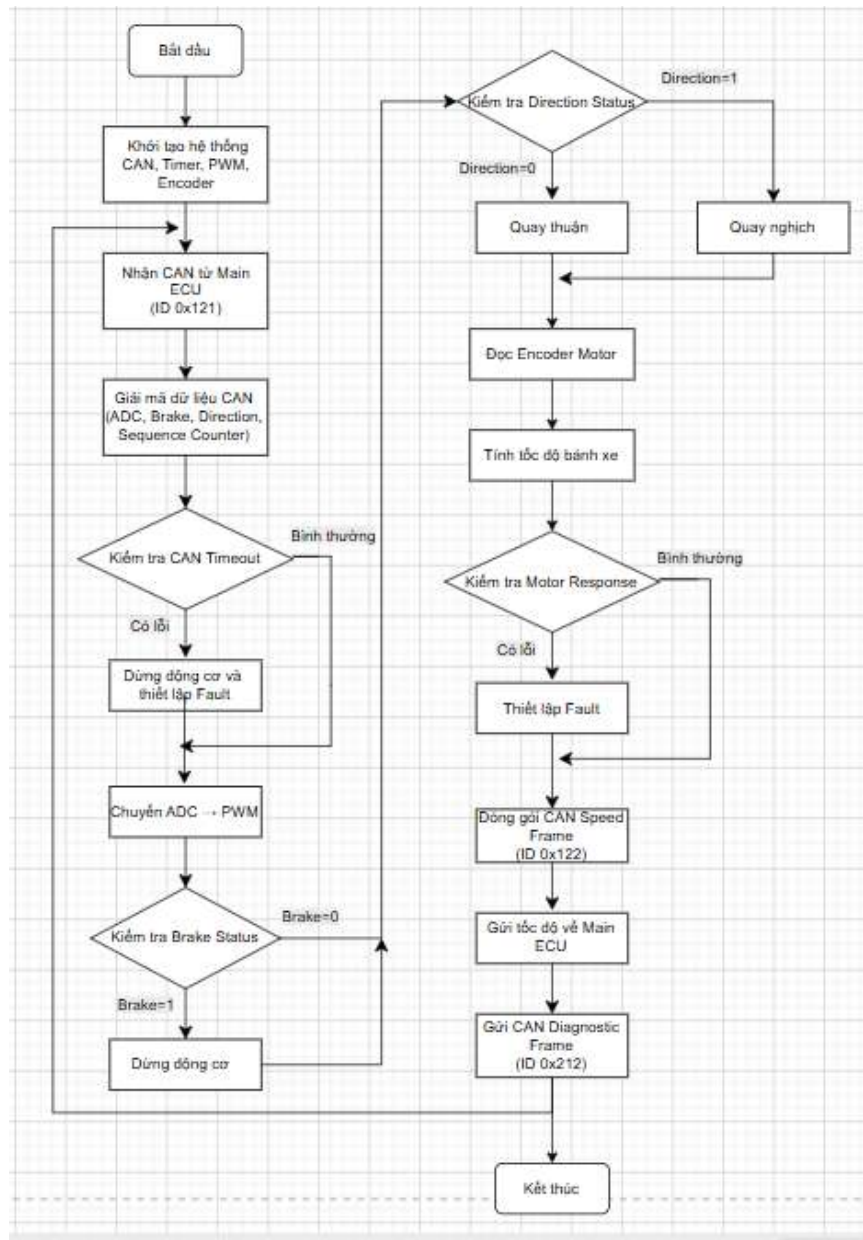
3.4.4 Thiết kế phần mềm ABS ECU

ABS ECU có nhiệm vụ nhận lệnh điều khiển từ Main ECU thông qua mạng CAN, điều khiển tốc độ và chiều quay của động cơ, đồng thời tính toán tốc độ bánh xe từ tín hiệu encoder và truyền dữ liệu phản hồi về Main ECU. Ngoài ra, ABS ECU còn thực hiện các chức năng giám sát an toàn nhằm phát hiện các lỗi trong quá trình vận hành.

Các chức năng chính của ABS ECU bao gồm:

- Nhận dữ liệu điều khiển từ Main ECU thông qua CAN Bus.

- Chuyển đổi giá trị ADC thành tín hiệu PWM điều khiển động cơ.
- Điều khiển chiều quay và trạng thái phanh của động cơ.
- Đọc tín hiệu encoder từ động cơ GA25-370.
- Tính toán tốc độ bánh xe.
- Truyền dữ liệu tốc độ về Main ECU.
- Giám sát trạng thái truyền thông CAN.
- Phát hiện lỗi động cơ không phản hồi.



Hình thiết kế hệ thống.17 Lưu đồ thuật toán ABS ECU

Nhận lệnh điều khiển từ Main ECU

ABS ECU liên tục lắng nghe các bản tin CAN có ID 0x121 được gửi từ Main ECU. Sau khi nhận bản tin, ECU thực hiện giải mã các trường dữ liệu gồm:

Giá trị ADC điều khiển tốc độ.

- Trạng thái Brake.
- Trạng thái Direction.
- Sequence Counter.

Thuật toán chuyển đổi ADC sang PWM

Giá trị ADC nhận từ Main ECU có dải từ 0 đến 4095. Để điều khiển động cơ, giá trị này được ánh xạ sang độ rộng xung PWM.

Thuật toán ánh xạ được thực hiện theo dạng công thức (3.1):

$PWM = \frac{ADC \times ADC \times PWM_{max}}{4095}$	(3.1)
--	-------

Trong đó:

- ADC: giá trị nhận từ Main ECU.
- PWM: giá trị CCR xuất ra Timer PWM.
- PWM_max: giá trị PWM lớn nhất.

Kết quả thu được là tín hiệu PWM điều khiển module công suất XY-160D nhằm thay đổi tốc độ động cơ.

Thuật toán điều khiển động cơ

Trạng thái hoạt động của động cơ được xác định dựa trên tín hiệu Brake và Direction.

Bảng 3.13 Trạng thái điều khiển động cơ

Brake	Direction	Trạng thái
1	X	Dừng động cơ
0	0	Quay thuận
0	1	Quay nghịch

Khi Brake được kích hoạt, động cơ sẽ dừng bất kể trạng thái Direction.

Thuật toán tính tốc độ bánh xe

ABS ECU sử dụng encoder Hall tích hợp trên động cơ GA25-370 để xác định tốc độ quay.

Trong mỗi chu kỳ lấy mẫu, hệ thống xác định số xung encoder thay đổi:

$\Delta \text{Count} = \text{Count_current} - \text{Count_previous}$	(3.2)
--	-------

Do bộ đếm encoder hoạt động ở chế độ 16 bit nên thuật toán có bổ sung cơ chế xử lý tràn bộ đếm nhằm đảm bảo độ chính xác khi tính toán.

Số vòng quay bánh xe được xác định theo công thức (3.3):

$Rev = \frac{Delta\ Count}{CountsPerRev}$	(3.3)
---	-------

Trong đó:

- Rev: số vòng quay bánh xe.
- CountsPerRev: số xung encoder trên một vòng quay.

Quãng đường di chuyển của bánh xe: $Distance = Rev \times C$

Trong đó: $C = \pi D$

- D: đường kính bánh xe.
- C: chu vi bánh xe.

Tốc độ tuyến tính của bánh xe: $Speed = \frac{Distance}{T}$

Trong đó:

- Distance: quãng đường di chuyển.
- T: chu kỳ lấy mẫu.

Kết quả cuối cùng được chuyển đổi sang đơn vị cm/s trước khi gửi về Main ECU..

Thuật toán truyền dữ liệu tốc độ

Sau khi tính toán tốc độ bánh xe, ABS ECU đóng gói dữ liệu thành CAN Frame có ID 0x122 và gửi tới Main ECU.

Dữ liệu truyền bao gồm:

- Tốc độ bánh xe 1.
- Tốc độ bánh xe 2.
- Sequence Counter.
- Trạng thái hệ thống.

Thông tin này được Main ECU sử dụng để hiển thị và giám sát hoạt động của hệ thống.

Thuật toán CAN Timeout Detection

Để phát hiện mất kết nối truyền thông, ABS ECU sử dụng cơ chế CAN Timeout. Sau mỗi lần nhận bản tin từ Main ECU, thời gian nhận cuối cùng được cập nhật. Nếu khoảng thời gian không nhận được dữ liệu vượt quá ngưỡng cho phép, hệ thống xác định đã xảy ra lỗi truyền thông.

Điều kiện phát hiện lỗi: **CurrentTime - LastRxTime > TimeoutThreshold**

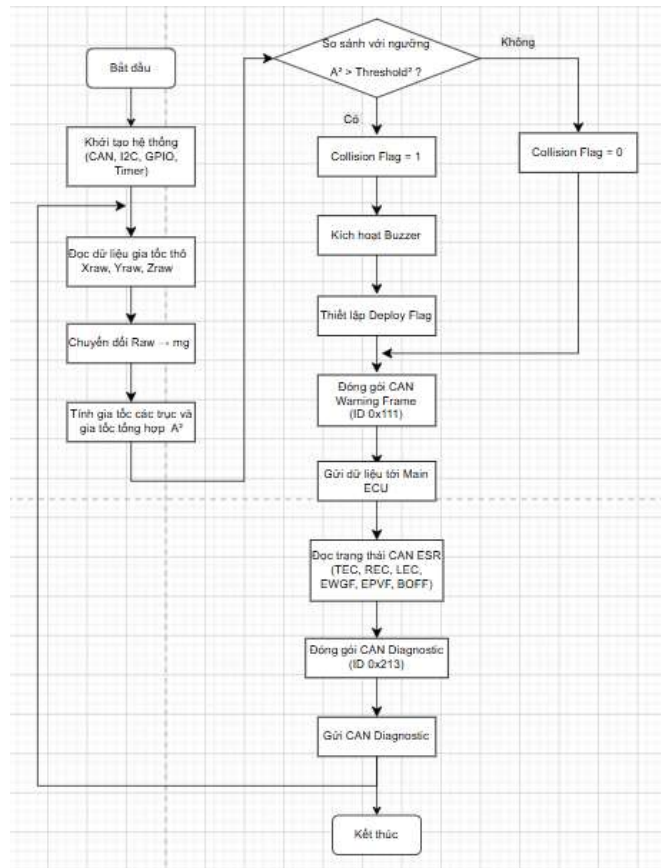
Khi lỗi xảy ra, ABS ECU sẽ đưa động cơ về trạng thái an toàn và ghi nhận trạng thái lỗi.

3.4.5 Thiết kế phần mềm Airbag ECU

Airbag ECU có nhiệm vụ thu thập dữ liệu gia tốc từ cảm biến LIS331HH, xử lý và đánh giá trạng thái va chạm của hệ thống. Khi phát hiện gia tốc vượt quá ngưỡng cho phép, ECU sẽ kích hoạt cảnh báo và gửi thông tin sự kiện lên mạng CAN để Main ECU thực hiện các biện pháp xử lý an toàn. Ngoài ra, Airbag ECU còn thực hiện chức năng giám sát trạng thái truyền thông nhằm đảm bảo dữ liệu được trao đổi ổn định trong hệ thống.

Các chức năng chính của Airbag ECU bao gồm:

- Thu thập dữ liệu từ cảm biến gia tốc LIS331HH.
- Chuyển đổi dữ liệu cảm biến sang đơn vị gia tốc.
- Tính toán gia tốc tổng hợp theo ba trục.
- Phát hiện trạng thái va chạm.
- Kích hoạt còi cảnh báo (Buzzer).
- Gửi dữ liệu cảnh báo lên Main ECU thông qua CAN Bus.
- Giám sát trạng thái hoạt động của cảm biến.
- Giám sát lỗi truyền thông CAN.



Hình thiết kế hệ thống.18 Lưu đồ thuật toán của Airbag ECU.

Thuật toán thu thập dữ liệu gia tốc

Airbag ECU giao tiếp với cảm biến LIS331HH thông qua giao thức I2C. Dữ liệu gia tốc được đọc theo ba trục X, Y và Z.

Các giá trị nhận được từ thanh ghi cảm biến là dữ liệu thô (Raw Data). Để sử dụng trong quá trình tính toán, các giá trị này được chuyển đổi sang đơn vị gia tốc dựa trên hệ số nhạy (Sensitivity) của cảm biến.

Kết quả thu được là các giá trị gia tốc theo ba trục:

- Gia tốc trục X.
- Gia tốc trục Y.
- Gia tốc trục Z.

Các giá trị này được sử dụng làm đầu vào cho thuật toán phát hiện va chạm..

Thuật toán tính gia tốc tổng hợp

Do va chạm có thể xảy ra theo nhiều hướng khác nhau, việc chỉ theo dõi riêng từng trục sẽ không phản ánh chính xác mức độ tác động lên hệ thống.

Để giải quyết vấn đề này, Airbag ECU thực hiện tính toán độ lớn vector gia tốc tổng hợp từ ba thành phần gia tốc X, Y và Z.

Công thức tổng quát:

$A = \sqrt{X^2 + Y^2 + Z^2}$	(3.4)
------------------------------	-------

Tuy nhiên, để giảm khối lượng tính toán trên vi điều khiển, hệ thống sử dụng giá trị bình phương của độ lớn gia tốc:

$A^2 = X^2 + Y^2 + Z^2$	(3.5)
-------------------------	-------

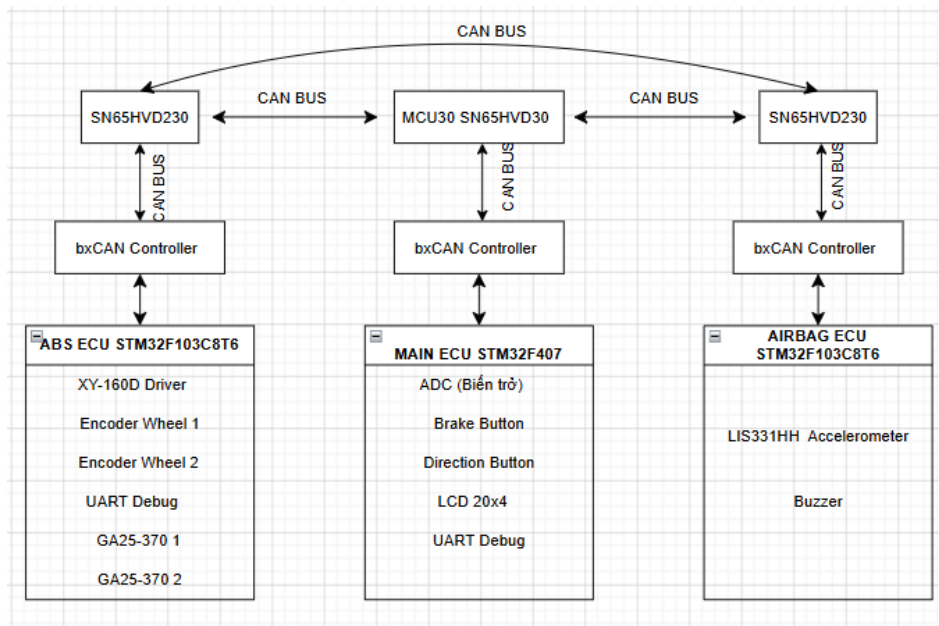
Phương pháp này cho phép loại bỏ phép tính căn bậc hai nhưng vẫn đảm bảo kết quả so sánh với ngưỡng va chạm là chính xác..

Airbag ECU thực hiện chức năng mô phỏng hệ thống phát hiện va chạm trên ô tô thông qua cảm biến gia tốc LIS331HH. Bằng việc áp dụng thuật toán tính gia tốc tổng hợp và cơ chế phát hiện ngưỡng va chạm, ECU có khả năng phát hiện các tác động bất thường và truyền thông tin cảnh báo tới Main ECU thông qua mạng CAN. Điều này góp phần xây dựng môi trường thực nghiệm phục vụ nghiên cứu truyền thông CAN và cơ chế Fault Confinement trong hệ thống đa ECU.

3.4.6 Thiết kế truyền thông CAN

Trong hệ thống thực nghiệm, mạng CAN Bus được sử dụng làm môi trường truyền thông chính giữa Main ECU, ABS ECU và Airbag ECU. Các ECU trao đổi dữ liệu theo mô hình phân tán, trong đó mỗi ECU thực hiện một chức năng riêng biệt nhưng vẫn có khả năng phối hợp thông qua các bản tin CAN.

Việc sử dụng CAN Bus giúp hệ thống giảm số lượng dây kết nối, tăng khả năng chống nhiễu và hỗ trợ cơ chế phát hiện lỗi.



Hình thiết kế hệ thống.29 Kiến trúc truyền thông CAN của hệ thống

Các ECU trao đổi dữ liệu thông qua mạng CAN tốc độ cao sử dụng chuẩn CAN 2.0A với định danh chuẩn 11 bit. Để đảm bảo khả năng truyền nhận dữ liệu giữa các ECU, hệ thống sử dụng nhiều bản tin CAN với các định danh khác nhau.

Bảng 3.14 Danh sách CAN ID sử dụng trong hệ thống

CAN ID	Nguồn gửi	Đích nhận	Chức năng
0x121	Main ECU	ABS ECU	Điều khiển động cơ
0x122	ABS ECU	Main ECU	Gửi tốc độ bánh xe
0x111	Airbag ECU	Main ECU	Gửi trạng thái va chạm
0x211	Main ECU	Toàn mạng	Chẩn đoán CAN
0x212	ABS ECU	Toàn mạng	Chẩn đoán CAN
0x213	Airbag ECU	Toàn mạng	Chẩn đoán CAN

Main ECU sử dụng CAN ID 0x121 để gửi dữ liệu điều khiển tới ABS ECU.

Bảng 3.15 Cấu trúc bản tin CAN ID 0x121

Byte	Nội dung
Byte 0	ADC Low Byte
Byte 1	ADC High Byte

Byte 2	Brake Status
Byte 3	Direction Status
Byte 4	Sequence Counter

Trong đó:

- ADC biểu diễn giá trị điều khiển tốc độ.
- Brake xác định trạng thái phanh.
- Direction xác định chiều quay.
- Sequence Counter dùng để phát hiện mất bản tin.

ABS ECU sử dụng CAN ID 0x122 để truyền dữ liệu tốc độ về Main ECU để hiển thị trên LCD và phục vụ các chức năng giám sát .

Bảng 3.16 Cấu trúc bản tin CAN ID 0x122

Byte	Nội dung
Byte 0	Wheel Speed 1
Byte 1	Wheel Speed 2
Byte 2	Sequence Counter
Byte 3	Diagnostic Status

Airbag ECU sử dụng CAN ID 0x111 để truyền trạng thái va chạm. Khi Collision Flag được thiết lập, Main ECU sẽ kích hoạt cơ chế Emergency Stop.

Bảng 3.17 Cấu trúc bản tin CAN ID 0x111

Byte	Nội dung
Byte 0	Collision Flag
Byte 1	Acceleration Magnitude
Byte 2	Sequence Counter
Byte 3	Diagnostic Status

Thuật toán Sequence Counter

Để nâng cao độ tin cậy của hệ thống, các bản tin CAN đều được bổ sung trường Sequence Counter.

Sau mỗi lần truyền thành công, bộ đếm được tăng lên một đơn vị:

$$\text{Sequence} = \text{Sequence} + 1$$

Tại ECU nhận, giá trị Sequence hiện tại được so sánh với giá trị mong đợi.

Nếu: $\text{Sequence_received} \neq \text{Sequence_expected}$

Hệ thống xác định đã xảy ra hiện tượng mất bản tin hoặc nhận sai thứ tự bản tin. Thuật toán này giúp đánh giá độ ổn định của quá trình truyền thông CAN trong các thử nghiệm thực tế..

Cấu hình tốc độ truyền CAN

Các ECU trong hệ thống được cấu hình cùng một tốc độ truyền dữ liệu nhằm đảm bảo khả năng giao tiếp.

Tốc độ truyền được xác định bởi công thức (3.6):

$\text{Bitrate} = \frac{F_{CAN}}{\text{Prescaler} \times (1 + \text{BS1} + \text{BS2})}$	(3.6)
--	--------------

Trong đó:

- F_{CAN} : tần số clock của bộ điều khiển CAN.
- Prescaler: hệ số chia tần số.
- BS1: Bit Segment 1.
- BS2: Bit Segment 2.

Thông qua việc lựa chọn các tham số phù hợp, hệ thống được cấu hình hoạt động ở tốc độ 500 kbps..

Bộ lọc CAN (CAN Filter)

Để giảm tải cho CPU, mỗi ECU chỉ tiếp nhận các bản tin liên quan đến chức năng của mình.

Bộ lọc CAN được cấu hình nhằm:

- Loại bỏ các bản tin không cần thiết.
- Giảm số lần xử lý ngắt.
- Tăng hiệu suất hệ thống.

Nhờ đó, ABS ECU chỉ tiếp nhận các bản tin điều khiển từ Main ECU trong khi Airbag ECU chỉ xử lý các bản tin liên quan đến chức năng giám sát của mình.

Hệ thống truyền thông CAN được thiết kế dựa trên mô hình đa ECU, trong đó mỗi ECU đảm nhiệm một chức năng riêng và trao đổi dữ liệu thông qua các bản

tin CAN được định nghĩa trước. Việc bổ sung Sequence Counter, cơ chế lọc CAN và cấu hình Bit Timing phù hợp giúp nâng cao độ tin cậy của hệ thống, đồng thời tạo nền tảng cho việc nghiên cứu cơ chế Fault Confinement được trình bày trong phần tiếp theo.

3.4.7 Thiết kế cơ chế Fault Confinement

Một trong những đặc điểm nổi bật của giao thức CAN là khả năng tự phát hiện và cô lập các nút mạng bị lỗi nhằm đảm bảo tính ổn định của toàn bộ hệ thống. Cơ chế này được gọi là Fault Confinement.

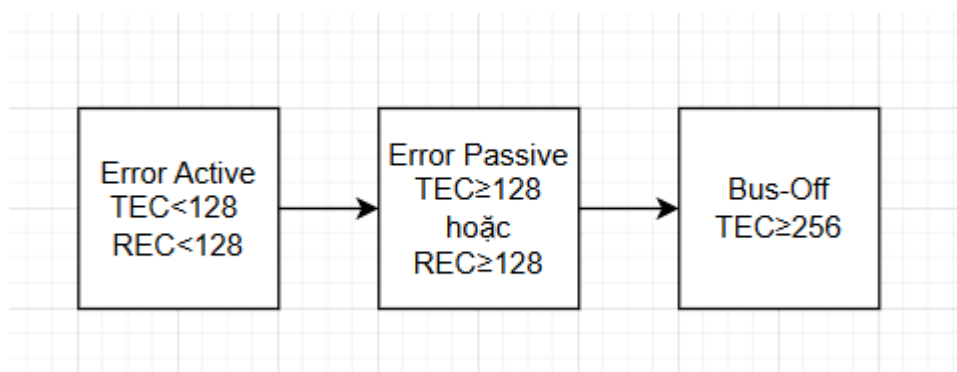
Trong đề tài này, cơ chế Fault Confinement được triển khai nhằm giám sát trạng thái hoạt động của các ECU trên mạng CAN và phục vụ quá trình nghiên cứu hành vi của giao thức CAN khi xuất hiện lỗi truyền thông.

Fault Confinement là cơ chế cho phép bộ điều khiển CAN tự động đánh giá mức độ lỗi của mỗi nút mạng thông qua các bộ đếm lỗi.

Mỗi ECU trên mạng CAN sẽ liên tục theo dõi:

- Số lỗi truyền (Transmit Error Counter - TEC).
- Số lỗi nhận (Receive Error Counter - REC).
- Mã lỗi gần nhất (Last Error Code - LEC).

Dựa trên các giá trị này, bộ điều khiển CAN sẽ tự động chuyển đổi giữa các trạng thái hoạt động khác nhau nhằm hạn chế ảnh hưởng của nút lỗi đến toàn bộ mạng.



Hình 3.20 Sơ đồ trạng thái Fault Confinement

Trong đề tài, trạng thái của bộ điều khiển CAN được theo dõi thông qua thanh ghi CAN_ESR của STM32.

Các tham số được giám sát bao gồm: Last Error Code (LEC)

LEC cho biết loại lỗi gần nhất xảy ra trên mạng CAN.

Bảng 3.18 Một số giá trị LEC thường gặp:

Giá trị	Ý nghĩa
---------	---------

0	Không có lỗi
1	Stuff Error
2	Form Error
3	Acknowledgment Error
4	Bit Recessive Error
5	Bit Dominant Error
6	CRC Error

Thông tin LEC được sử dụng để xác định nguyên nhân gây lỗi trong quá trình thực nghiệm.

Transmit Error Counter (TEC) là bộ đếm lỗi truyền. Giá trị TEC tăng khi ECU phát hiện lỗi trong quá trình truyền dữ liệu CAN. TEC được sử dụng để đánh giá mức độ nghiêm trọng của lỗi truyền.

Receive Error Counter (REC) là bộ đếm lỗi nhận. Giá trị REC tăng khi ECU phát hiện lỗi trong quá trình nhận dữ liệu. Thông số này phản ánh chất lượng dữ liệu nhận được từ mạng CAN.

Giám sát trạng thái CAN

Trong quá trình vận hành, mỗi ECU định kỳ đọc thanh ghi CAN_ESR để cập nhật trạng thái hiện tại của bộ điều khiển CAN.

Các thông tin thu thập bao gồm:

- TEC.
- REC.
- LEC.
- EWGF.
- EPVF.
- BOFF.

Dữ liệu được đóng gói thành các bản tin chẩn đoán và truyền lên mạng CAN nhằm phục vụ việc giám sát và đánh giá hệ thống..

Trạng thái Error Active

Đây là trạng thái hoạt động bình thường của bộ điều khiển CAN.

Điều kiện: $TEC < 128$ và $REC < 128$

Trong trạng thái này:

- ECU được phép truyền dữ liệu bình thường.

- ECU được phép phát Error Frame chủ động.
- Mạng CAN hoạt động ổn định.

Đây là trạng thái mà hệ thống hướng tới trong quá trình vận hành thông thường.

Trạng thái Error Passive

Khi số lượng lỗi tăng vượt ngưỡng cho phép, bộ điều khiển CAN sẽ chuyển sang trạng thái Error Passive.

Điều kiện: $TEC \geq 128$ hoặc $REC \geq 128$

Trong trạng thái này:

- ECU vẫn được phép truyền dữ liệu.
- ECU không còn phát Active Error Frame.
- Khả năng ảnh hưởng đến mạng được giảm xuống.

Trạng thái Error Passive cho phép hệ thống tiếp tục hoạt động trong khi hạn chế tác động của ECU đang gặp lỗi.

Trạng thái Bus-Off

Bus-Off là mức lỗi nghiêm trọng nhất của bộ điều khiển CAN.

Điều kiện: $TEC \geq 256$

Khi xảy ra Bus-Off:

- ECU bị tách khỏi mạng CAN.
- Không được phép truyền dữ liệu.
- Không được phép nhận dữ liệu.
- Toàn bộ quá trình giao tiếp bị dừng.

Mục tiêu của trạng thái Bus-Off là ngăn chặn một ECU lỗi làm ảnh hưởng đến các ECU còn lại trên mạng.

Việc triển khai Fault Confinement cho phép hệ thống:

- Theo dõi trạng thái hoạt động của từng ECU.
- Phát hiện lỗi truyền thông CAN.
- Đánh giá ảnh hưởng của lỗi đến toàn mạng.
- Quan sát quá trình chuyển đổi giữa Error Active, Error Passive và Bus-Off.
- Phục vụ nghiên cứu cơ chế tự bảo vệ của giao thức CAN.

Thông qua việc giám sát các bộ đếm lỗi và các cờ trạng thái của bộ điều khiển CAN, đề tài đã xây dựng được môi trường thực nghiệm cho phép nghiên cứu trực quan cơ chế Fault Confinement trên mạng CAN đa ECU.

3.4.8 Kết luận chương

Trong chương này, đề tài đã trình bày quá trình thiết kế và xây dựng hệ thống phần mềm cho mô hình mạng CAN đa ECU dựa trên nền tảng vi điều khiển STM32. Hệ thống phần mềm được phát triển theo kiến trúc AUTOSAR Classic thu gọn, bao gồm các tầng Application, RTE, COM, CANIF và CAN Driver nhằm đảm bảo tính mô-đun, khả năng mở rộng và thuận lợi cho việc phát triển các ECU độc lập.

Đối với Main ECU, phần mềm được xây dựng để thực hiện chức năng thu thập tín hiệu điều khiển từ người dùng, quản lý truyền thông CAN, hiển thị thông tin hệ thống và giám sát trạng thái hoạt động của toàn mạng. Các thuật toán như Sequence Counter, Moving Average Filter và Emergency Stop đã được triển khai nhằm nâng cao độ tin cậy và tính an toàn của hệ thống.

Đối với ABS ECU, phần mềm thực hiện các chức năng nhận lệnh điều khiển từ Main ECU, điều khiển động cơ thông qua tín hiệu PWM, tính toán tốc độ bánh xe từ encoder và giám sát trạng thái hoạt động của động cơ. Các thuật toán CAN Timeout Detection và Motor No Response Detection được sử dụng để phát hiện các tình huống bất thường trong quá trình vận hành.

Đối với Airbag ECU, phần mềm được xây dựng để thu thập dữ liệu từ cảm biến gia tốc LIS331HH, tính toán gia tốc tổng hợp theo ba trục và phát hiện các trạng thái va chạm dựa trên ngưỡng gia tốc được thiết lập trước. Khi phát hiện va chạm, ECU sẽ gửi bản tin cảnh báo lên mạng CAN và kích hoạt cơ chế xử lý an toàn của hệ thống.

Bên cạnh đó, hệ thống truyền thông CAN đã được thiết kế với các bản tin chức năng, cơ chế Sequence Counter và cấu hình Bit Timing phù hợp nhằm đảm bảo quá trình trao đổi dữ liệu giữa các ECU diễn ra ổn định. Đề tài cũng triển khai cơ chế Fault Confinement thông qua việc giám sát các tham số TEC, REC, LEC cùng các cờ trạng thái EWGF, EPVF và BOFF của bộ điều khiển CAN.

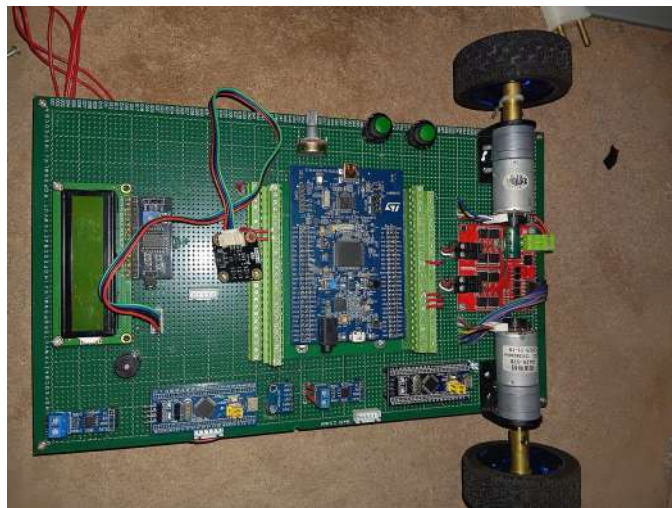
Những nội dung được trình bày trong chương này là cơ sở để tiến hành xây dựng mô hình thực nghiệm, thực hiện các kịch bản kiểm thử và đánh giá hoạt động của mạng CAN cũng như cơ chế Fault Confinement được trình bày trong chương tiếp theo.

CHƯƠNG 4

KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH

4.1 KẾT QUẢ MÔ HÌNH THI CÔNG

Hệ thống thực nghiệm đã được thi công và lắp ráp hoàn chỉnh gồm Main ECU, ABS ECU và Airbag ECU kết nối qua mạng CAN Bus. Mô hình cho phép thực hiện các chức năng điều khiển động cơ, đo tốc độ bánh xe, phát hiện va chạm và giám sát trạng thái truyền thông CAN. Phần này trình bày kết quả lắp ráp phần cứng và mô hình thực tế được sử dụng trong quá trình thử nghiệm và đánh giá hệ thống.



Hình KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.3 Mô hình hoàn thiện

4.2 KẾT QUẢ MÔ PHỎNG

4.2.1 Mục tiêu thực nghiệm

Mục tiêu của chương này là kiểm chứng hoạt động của hệ thống đa ECU giao tiếp CAN đã được thiết kế ở Chương 3. Các thực nghiệm tập trung đánh giá khả năng truyền nhận dữ liệu giữa ECU Main, ECU ABS và ECU Airbag; khả năng điều khiển động cơ của ECU ABS; khả năng phát hiện va chạm của ECU Airbag;

đồng thời đánh giá cơ chế giám sát lỗi CAN thông qua các giá trị đọc được từ bộ điều khiển CAN ở mức thanh ghi.

Các nội dung thực nghiệm được xây dựng nhằm kiểm tra tính đúng đắn của thiết kế, khả năng phối hợp giữa các ECU và hiệu quả của việc triển khai CAN mức thanh ghi kết hợp với kiến trúc AUTOSAR Classic thu gọn. Điều này phù hợp với mục tiêu của đề tài là xây dựng các kịch bản thực nghiệm để kiểm chứng hoạt động của hệ thống trong điều kiện vận hành thực tế.

4.2.2 Phương pháp thu thập dữ liệu thực nghiệm

Trong quá trình thực nghiệm, dữ liệu được thu thập thông qua ba phương pháp chính. Thứ nhất, USB-CAN tool được sử dụng để quan sát trực tiếp các frame xuất hiện trên bus CAN, bao gồm CAN ID, độ dài dữ liệu DLC và payload của từng frame. Phương pháp này giúp kiểm chứng rằng các frame chức năng và frame diagnostic thật sự được truyền trên bus CAN.

Thứ hai, UART log của từng ECU được sử dụng để kiểm tra quá trình xử lý nội bộ của hệ thống. Thông qua UART log, có thể quan sát được ECU Main gửi lệnh điều khiển đến ECU ABS, ECU ABS nhận lệnh và điều khiển motor, ECU Airbag đọc dữ liệu cảm biến và phát hiện va chạm, cũng như trạng thái diagnostic của từng node CAN.

Thứ ba, phản ứng vật lý của hệ thống được quan sát trực tiếp thông qua hoạt động của motor ABS và buzzer Airbag. Việc kết hợp giữa USB-CAN tool, UART log và quan sát thực tế giúp đánh giá hệ thống ở cả ba mức: dữ liệu trên bus CAN, xử lý nội bộ trong từng ECU và phản ứng vật lý của mô hình thực nghiệm.

Bảng KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.1 Các đối tượng cần kiểm tra

STT	Mục tiêu thực nghiệm	Đối tượng kiểm tra	Phương pháp quan sát	Kết quả cần thu thập
1	Kiểm tra truyền lệnh Main đến ABS	Frame 0x121	UART log trên Main và ABS	ADC, Brake, Direction, SEQ

2	Kiểm tra phản hồi từ ABS về Main	Frame 0x122	UART log trên ABS và Main	W1, W2, STATE, SEQ
3	Kiểm tra cảnh báo Airbag	Frame 0x111	UART log, buzzer, LCD Main	DEPLOY, SEQ, trạng thái buzzer
4	Kiểm tra nhiều ECU hoạt động đồng thời	Bus CAN 3 node	Log xen kẽ nhiều ID	0x111, 0x121, 0x122
5	Kiểm tra diagnostic CAN	Frame 0x211, 0x212, 0x213	UART log hoặc USB-CAN	LEC, TEC, REC, EWGF, EPVF, BOFF
6	Kiểm tra lỗi ACK	Node phát không nhận ACK	Log diagnostic	LEC = ACK Error, TEC tăng
7	Kiểm tra lỗi đường truyền CAN	CAN_H/CAN_L bất thường	Log diagnostic	LEC/TEC/REC thay đổi
8	Kiểm tra timeout ABS	ABS mất lệnh từ Main	Log ABS, motor	Motor dừng

4.3 QUY ƯỚC DỮ LIỆU SỬ DỤNG TRONG THỰC NGHIỆM

4.3.1 Quy ước CAN ID

Trong quá trình thực nghiệm, mỗi loại dữ liệu được gán một CAN ID riêng để phân biệt chức năng và hướng truyền. ECU Main gửi lệnh điều khiển đến ECU ABS thông qua ID 0x121. ECU ABS phản hồi trạng thái động cơ về Main thông qua ID 0x122. ECU Airbag gửi trạng thái va chạm về Main thông qua ID 0x111.

Ngoài các frame chức năng, mỗi node còn có frame diagnostic riêng để phục vụ giám sát lỗi CAN.

Bảng 4.2 Danh sách CAN ID sử dụng trong thực nghiệm

CAN ID	Node phát	Node nhận	DLC	Chức năng
0x111	Airbag	Main	2	Gửi trạng thái va chạm
0x121	Main	ABS	5	Gửi lệnh điều khiển ABS
0x122	ABS	Main	4	Phản hồi trạng thái ABS
0x211	Main	Bus CAN/PC	8	Diagnostic CAN của Main
0x212	ABS	Bus CAN/PC	8	Diagnostic CAN của ABS
0x212	Airbag	Bus CAN/PC	8	Diagnostic CAN của Airbag

Qua bảng phân bổ CAN ID, có thể thấy các frame được tách riêng theo từng chức năng. Việc sử dụng ID riêng giúp ECU Main phân biệt dữ liệu phản hồi từ ABS và Airbag, đồng thời giúp ECU ABS chỉ xử lý đúng frame điều khiển từ Main. Điều này làm giảm khả năng nhầm lẫn dữ liệu khi nhiều ECU cùng hoạt động trên một bus CAN.

4.3.2 Quy ước payload frame Main đến ABS

Frame 0x121 được ECU Main sử dụng để gửi lệnh điều khiển đến ECU ABS. Trong code hiện tại, frame này có DLC = 5 byte.

Bảng 4.3 Cấu trúc frame 0x121 Main đến ABS

Byte	Tên dữ liệu	Ý nghĩa
Byte 0	ADC_L	8 bit thấp của giá trị ADC
Byte 1	ADC_H	4 bit cao của giá trị ADC
Byte 2	BRAKE	Trạng thái phanh/dừng
Byte 3	DIR	Chiều quay motor
Byte 4	SEQ	Bộ đếm thứ tự frame

4.3.3 Quy ước payload frame ABS đến Main

Frame 0x122 được ECU ABS sử dụng để gửi trạng thái motor về ECU Main. Frame này có DLC = 4 byte.

Bảng 4.4 Cấu trúc frame 0x122 ABS → Main

Byte	Tên dữ liệu	Ý nghĩa
Byte 0	W1	Tốc độ bánh/motor 1, đơn vị cm/s

Byte 1	W2	Tốc độ bánh/motor 2, đơn vị cm/s
Byte 2	STATE	Trạng thái motor
Byte 3	SEQ	Bộ đếm thứ tự frame

Bảng 4.5 Quy ước trạng thái motor trong frame 0x122

Giá trị STATE	Ý nghĩa
0	STOP
1	FORWARD
2	REVERSE
3	BRAKE
4	FAULT

4.3.4 Quy ước payload frame Airbag đến Main

Frame 0x111 được ECU Airbag sử dụng để gửi trạng thái va chạm về ECU Main. Frame này có DLC = 2 byte.

Bảng 4.6 Cấu trúc frame 0x111 Airbag đến Main

Byte	Tên dữ liệu	Ý nghĩa
Byte 0	DEPLOY	Trạng thái phát hiện va chạm
Byte 1	SEQ	Bộ đếm thứ tự frame

Bảng 4.7 Quy ước trạng thái DEPLOY

DEPLOY	Ý nghĩa
0	Không phát hiện va chạm
1	Phát hiện va chạm, kích hoạt buzzer

4.3.5 Quy ước payload frame diagnostic CAN

Các frame diagnostic được dùng để theo dõi trạng thái lỗi của từng node CAN. Dữ liệu diagnostic gồm mã lỗi gần nhất LEC, bộ đếm lỗi truyền TEC, bộ đếm lỗi nhận REC và các cờ trạng thái lỗi. Trong Chương 2, bạn đã trình bày thành ghi ESR chứa các thông tin như LEC, EWGF, EPVF và BOFF để phản ánh trạng thái lỗi của bộ điều khiển CAN. Vì vậy, Chương 4 cần dùng chính các trường này làm số liệu thực nghiệm.

Bảng 4.8 Cấu trúc frame diagnostic CAN

Byte	Tên dữ liệu	Ý nghĩa
Byte 0	LEC	Last Error Code, mã lỗi gần nhất
Byte 1	TEC	Transmit Error Counter
Byte 2	REC	Receive Error Counter
Byte 3	EWGF	Error Warning Flag
Byte 4	EPVF	Error Passive Flag
Byte 5	BOFF	Bus-Off Flag
Byte 6	ErrorState	Trạng thái lỗi tổng quát
Byte 7	ControllerMode	Trạng thái bộ điều khiển CAN

Bảng 4.9 Quy ước mã LEC

LEC	Ý nghĩa
0	No Error
1	Stuff Error
2	Form Error
3	ACK Error
4	Bit Error - Recessive
5	Bit Error - Dominant
6	CRC Error
7	No Change

Bảng 4.10 Quy ước ErrorState và ControllerMode

Trường	Giá trị	Ý nghĩa
ErrorState	0	Error Active
ErrorState	1	Error Passive
ErrorState	2	Bus-Off
ControllerMode	0	Stopped
ControllerMode	1	Started
ControllerMode	2	Sleep
ControllerMode	3	Uninit

4.4 KẾT QUẢ THỰC NGHIỆM

4.4.1 Thực nghiệm 1: Kiểm tra khởi động hệ thống và trạng thái CAN bình thường

Mục tiêu: Kiểm tra các ECU khởi động đúng, CAN được cấu hình thành công và các node bắt đầu truyền nhận dữ liệu bình thường.

Sau khi cấp nguồn cho toàn bộ hệ thống, các ECU lần lượt thực hiện quá trình khởi tạo ngoại vi, cấu hình CAN và đưa bộ điều khiển CAN về trạng thái hoạt động. Quá trình khởi động được quan sát thông qua UART log của từng ECU nhằm kiểm tra trạng thái sẵn sàng của hệ thống trước khi thực hiện các kịch bản truyền nhận dữ liệu. Kết quả kiểm tra khởi động của các ECU được trình bày trong Bảng 4.11.

Bảng 4.11 Kết quả khởi động hệ thống

ECU	Log khởi động mong đợi	Kết quả thực tế	Đánh giá
Main	Hiển thị CAN IDs, bắt đầu gửi 0x121	Hoạt động tốt	Đạt
ABS	ABS sẵn sàng nhận ID 0x121, truyền ID 0x122	Hoạt động tốt	Đạt
Airbag	Hiển thị STATUS truyền ID 0x111, DIAG 0x213	Hoạt động tốt	Đạt

```
[MAIN] CAN IDs: TX_ABS=0x121 RX_AIRBAG=0x111 RX_ABS=0x122 DIAG=0x211
BOOT OK
UART2 OK
CAN Filter Config OK
CAN Start OK
CAN Notification OK

[BOOT] NODE=MAIN | STATUS=OK | TX=0x121 | RX=0x122, 0x111 | DIAG=0x211

MAIN READY | TX_ABS: 0x121 | RX_AIRBAG: 0x111 | RX_ABS: 0x122 | CAN_DIAG: 0x211

[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=0 | RESULT=OK | CNT=1 | RAW=FF OF 01 00 00
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=1 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=1 | RESULT=OK | CNT=2 | RAW=FF OF 01 00 01
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=2 | RESULT=OK | CNT=3 | RAW=FF OF 01 00 02
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=3 | RESULT=OK | CNT=4 | RAW=FF OF 01 00 03
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=2 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=4 | RESULT=OK | CNT=5 | RAW=FF OF 01 00 04
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=5 | RESULT=OK | CNT=6 | RAW=FF OF 01 00 05
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=3 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=6 | RESULT=OK | CNT=7 | RAW=FF OF 01 00 06
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=7 | RESULT=OK | CNT=8 | RAW=FF OF 01 00 07
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=8 | RESULT=OK | CNT=9 | RAW=FF OF 01 00 08
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=4 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=9 | RESULT=OK | CNT=10 | RAW=FF OF 01 00 09
[CAN RX] NODE=MAIN | SRC=ABS | ID=0x122 | DLC=4 | W1=0 cm/s | W2=0 cm/s | STATE=3 | SEQ=226 | CNT=4 | LOST=0
[CAN RX] NODE=MAIN | SRC=AIRBAG | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=79 | CNT=4 | LOST=0
```

Hình KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.2 Log khởi động ECU Main

```
[ABS] CAN IDs: RX_MAIN=0x121 TX_MAIN=0x122 DIAG=0x212
BOOT OK
UART2 OK
CAN Filter Config OK
CAN Start OK
CAN Notification OK

[BOOT] NODE=ABS | STATUS=OK | RX=0x121 | TX=0x122 | DIAG=0x212

[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=46 | CNT=1 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=47 | CNT=2 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[ENC] NODE=ABS | D1=0 | D2=8 | V1=0 cm/s | V2=2 cm/s
[CAN TX] NODE=ABS | DST=MAIN | ID=0x122 | DLC=4 | W1=0 cm/s | W2=2 cm/s | STATE=3 | SEQ=0 | RESULT=OK | CNT=1 | FAIL=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=48 | CNT=3 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=49 | CNT=4 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=50 | CNT=5 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[ENC] NODE=ABS | D1=0 | D2=2 | V1=0 cm/s | V2=0 cm/s
[CAN TX] NODE=ABS | DST=MAIN | ID=0x122 | DLC=4 | W1=0 cm/s | W2=0 cm/s | STATE=3 | SEQ=1 | RESULT=OK | CNT=2 | FAIL=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=51 | CNT=6 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=1 | DIR=0 | SEQ=52 | CNT=7 | LOST=46
[APP] NODE=ABS | MOTOR | ADC=4095 | PWM=28796 | BRAKE=1 | DIR=0
```

Hình KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.3 Log khởi động ECU ABS

```
[S331HH INIT OK | WHO_AM_I=0x32
Threshold = 2500 mg
[AIRBAG] CAN IDs: STATUS=0x111 DIAG=0x213

[BOOT] NODE=AIRBAG | STATUS=OK | TX=0x111 | DIAG=0x213

[SENSOR] NODE=AIRBAG | RAW_X=1 | RAW_Y=-163 | RAW_Z=26 | MG_X=6 | MG_Y=-976 | MG_Z=156 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=1 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=2 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=5 | RAW_Y=-167 | RAW_Z=30 | MG_X=30 | MG_Y=-1002 | MG_Z=180 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=3 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=4 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=4 | RAW_Y=-165 | RAW_Z=26 | MG_X=24 | MG_Y=-990 | MG_Z=156 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=5 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=6 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=2 | RAW_Y=-166 | RAW_Z=28 | MG_X=12 | MG_Y=-996 | MG_Z=168 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=7 | TX_FAIL=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=7 | RESULT=OK | CNT=7 | FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=8 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=2 | RAW_Y=-167 | RAW_Z=27 | MG_X=12 | MG_Y=-1002 | MG_Z=162 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=9 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=10 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=4 | RAW_Y=-163 | RAW_Z=29 | MG_X=24 | MG_Y=-978 | MG_Z=174 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=11 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=12 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=4 | RAW_Y=-164 | RAW_Z=25 | MG_X=24 | MG_Y=-984 | MG_Z=150 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=13 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0 (NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=14 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=3 | RAW_Y=-164 | RAW_Z=25 | MG_X=18 | MG_Y=-984 | MG_Z=150 | DEPLOY=0
```

Hình KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH.4 Log khởi động ECU Airbag

Kết quả khởi động cho thấy các ECU đã được nạp chương trình và khởi tạo thành công. ECU Main bắt đầu gửi lệnh điều khiển đến ECU ABS thông qua frame 0x121. ECU ABS nhận frame điều khiển và phản hồi trạng thái về Main thông qua frame 0x122. ECU Airbag gửi trạng thái va chạm về Main thông qua frame 0x111. Trong điều kiện bình thường, các giá trị lỗi như TEC, REC và các cờ EWGF, EPVF, BOFF duy trì ở mức thấp hoặc bằng 0, cho thấy bus CAN hoạt động ổn định.

4.4.2 Thực nghiệm 2: Truyền lệnh từ ECU Main đến ECU ABS

Mục tiêu: Kiểm tra ECU Main có gửi đúng frame điều khiển 0x121 đến ECU ABS hay không, đồng thời kiểm tra ECU ABS có giải mã đúng ADC, Brake, Direction và Sequence hay không.

Trong thực nghiệm này, ECU Main đọc giá trị điều khiển từ biến trở ADC và trạng thái các nút nhấn, sau đó đóng gói dữ liệu thành frame CAN có ID 0x121 để gửi đến ECU ABS. Việc truyền frame được kiểm tra thông qua UART log và công cụ USB-CAN nhằm xác nhận đúng CAN ID, đúng độ dài dữ liệu và đúng cấu trúc payload. Kết quả quan sát frame 0x121 được thể hiện trong Hình 4.4.

No	Direction	Time scale	Frame Format	Frame Type	Frame Id	Data Length	Data(Hex)
0	Received	20:55:52 324	Data frame	Standard frame	0x00000121	5	a4 03 01 00 d8
1	Received	20:55:52 360	Data frame	Standard frame	0x00000121	5	9d 03 01 00 d9
2	Received	20:55:52 388	Data frame	Standard frame	0x00000121	5	9d 03 01 00 da
3	Received	20:55:52 437	Data frame	Standard frame	0x00000121	5	98 03 01 00 db
4	Received	20:55:52 472	Data frame	Standard frame	0x00000121	5	9e 03 01 00 dc
5	Received	20:55:52 534	Data frame	Standard frame	0x00000121	5	a1 03 01 00 dd
6	Received	20:55:52 576	Data frame	Standard frame	0x00000121	5	9f 03 01 00 de
7	Received	20:55:52 611	Data frame	Standard frame	0x00000121	5	9f 03 01 00 df
8	Received	20:55:52 640	Data frame	Standard frame	0x00000121	5	a6 03 01 00 e0
9	Received	20:55:52 685	Data frame	Standard frame	0x00000121	5	a0 03 01 00 e1

Hình 4.5 MAIN gửi frame 0x121

Kết quả quan sát trên USB-CAN tool cho thấy ECU Main đã truyền thành công các frame điều khiển có CAN ID 0x121 lên bus CAN. Các frame thu được đều thuộc loại Data frame, sử dụng Standard frame và có độ dài dữ liệu DLC = 5 byte, phù hợp với cấu trúc frame Main gửi đến ABS đã thiết kế.

Dữ liệu payload của các frame có dạng như: A4 03 01 00 D8, 9D 03 01 00 D9, 9D 03 01 00 DA, 98 03 01 00 DB,... Theo quy ước payload của frame 0x121, hai byte đầu tiên là giá trị ADC, byte thứ ba là trạng thái BRAKE, byte thứ tư là trạng thái DIR và byte cuối cùng là bộ đếm SEQ. Từ các payload quan sát được, có thể thấy giá trị ADC thay đổi theo tín hiệu điều khiển từ biến trở, trạng thái BRAKE đang bằng 1, trạng thái DIR bằng 0 và giá trị SEQ tăng dần qua từng frame.

Việc bộ đếm SEQ tăng liên tục từ D8 đến E1 cho thấy ECU Main đang gửi frame điều khiển theo chu kỳ và các frame được truyền liên tiếp trên bus CAN. Đồng thời, việc CAN ID luôn giữ giá trị 0x121 và DLC luôn bằng 5 chứng minh quá trình đóng gói dữ liệu điều khiển của ECU Main đã đúng với quy ước truyền thông đã xây dựng.

Kết quả quan sát cho thấy ECU Main đã gửi đúng frame điều khiển 0x121 đến ECU ABS. Payload của frame chứa đầy đủ các trường dữ liệu gồm ADC, BRAKE, DIR và SEQ. Điều này chứng minh quá trình đọc tín hiệu đầu vào, đóng gói dữ liệu và truyền lệnh từ Main đến ABS hoạt động đúng theo thiết kế.

4.4.3 Thực nghiệm 3: Điều khiển động cơ trên ECU ABS

Mục tiêu: Kiểm tra ECU ABS có chuyển đổi giá trị ADC nhận từ ECU Main thành PWM điều khiển motor đúng theo thiết kế hay không.

Trong thực nghiệm này, giá trị ADC được thay đổi từ mức thấp đến mức cao để quan sát sự thay đổi của tín hiệu PWM trên ECU ABS. ECU ABS nhận dữ liệu điều khiển từ ECU Main thông qua frame 0x121, sau đó xử lý giá trị ADC và tạo tín hiệu PWM tương ứng để điều khiển motor. Các giá trị đo được giữa ADC, PWM và trạng thái motor được tổng hợp trong Bảng 4.12.

Bảng 4.12 Sự thay đổi của ADC và PWM trên ECU ABS

Lần đo	ADC nhận từ Main	PWM trên ABS	Tỷ lệ PWM xấp xỉ	BRAKE	DIR	Trạng thái motor	Nhận xét
1	0	0	0%	0	0	STOP	ADC nằm trong vùng chết, motor không quay
2	224	18562	51.6%	0	0	FORWARD	ADC vượt vùng chết, PWM đạt mức tối thiểu nên motor bắt đầu quay
3	1130	20957	58.2%	0	0	FORWARD	PWM tăng theo ADC, motor quay ổn định
4	1903	23001	63.9%	0	0	FORWARD	Tốc độ phản hồi tăng so với mức ADC thấp
5	2428	24389	67.8%	0	0	FORWARD	Motor tiếp tục duy trì trạng thái quay thuận
6	2860	25531	70.9%	0	0	FORWARD	PWM tăng, tốc độ phản hồi tương đối ổn

							định
7	3486	27186	75.5%	0	0	FORWARD	Motor quay nhanh hơn khi ADC tăng
8	3931	28362	78.8%	0	0	FORWARD	PWM gần mức giới hạn trên
9	4095	28796	80.0%	0	0	FORWARD	ADC đạt cực đại, PWM đạt mức giới hạn 80%

Từ bảng 4.12 có thể thấy ECU ABS đã nhận được giá trị ADC từ ECU Main thông qua frame 0x121 và chuyển đổi thành tín hiệu PWM để điều khiển motor. Khi ADC bằng 0 hoặc nằm trong vùng chết, PWM được đưa về 0 nên motor không quay. Điều này giúp hạn chế hiện tượng motor bị giật khi tín hiệu điều khiển ở gần mức 0 hoặc có dao động nhỏ do nhiễu ADC.

Khi ADC vượt khỏi vùng chết, ECU ABS bắt đầu tạo tín hiệu PWM để điều khiển motor. Ở mức ADC khoảng 224, PWM đạt 18562, tương ứng khoảng 51,6% chu kỳ timer. Đây là mức PWM tối thiểu giúp motor có đủ lực để bắt đầu quay. Khi ADC tiếp tục tăng, PWM cũng tăng tương ứng từ khoảng 58,2% đến 80%. Với giá trị ADC cực đại 4095, PWM đạt 28796, tương ứng xấp xỉ 80% với $ARR = 35995$. Kết quả này phù hợp với thiết kế phần mềm, trong đó PWM được giới hạn trong khoảng từ 50% đến 80% khi ADC vượt vùng chết.

Ngoài ra, các giá trị BRAKE và DIR trong quá trình thực nghiệm đều bằng 0, cho thấy motor đang hoạt động ở trạng thái quay thuận bình thường. Khi PWM bằng 0, trạng thái motor là STOP. Khi PWM tăng theo ADC, trạng thái motor chuyển sang FORWARD. Kết quả này chứng minh ECU ABS đã chuyển đổi đúng dữ liệu điều khiển từ ECU Main thành tín hiệu PWM điều khiển motor.



Hình 4.6 Trạng thái động khi điều chỉnh ADC về 0



Hình 4.7 Trạng thái động khi điều chỉnh biến trở ADC

Kết quả thực nghiệm cho thấy ECU ABS chuyển đổi giá trị ADC nhận từ ECU Main thành tín hiệu PWM điều khiển motor. Khi ADC nằm trong vùng chết, PWM được đưa về 0 nên motor không quay. Khi ADC tăng vượt ngưỡng, PWM tăng dần và motor bắt đầu quay ổn định. Tốc độ phản hồi từ encoder cũng thay đổi theo trạng thái PWM. Khi BRAKE được kích hoạt, ECU ABS ưu tiên đưa motor về trạng thái dừng/phanh, chứng minh logic điều khiển motor hoạt động đúng theo thiết kế.

4.4.4 Thực nghiệm 4: Phản hồi trạng thái từ ECU ABS về ECU Main

Mục tiêu: Kiểm tra ECU ABS có gửi đúng frame phản hồi 0x122 về ECU Main sau khi nhận và xử lý lệnh điều khiển từ Main hay không. Thực nghiệm này nhằm xác nhận rằng ECU ABS không chỉ nhận frame 0x121 từ ECU Main mà còn có khả năng phản hồi lại trạng thái hoạt động của motor, bao gồm W1, W2, trạng thái motor STATE và bộ đếm thứ tự frame SEQ.

Sau khi ECU ABS nhận và xử lý frame điều khiển từ ECU Main, trạng thái hoạt động của motor được đóng gói thành frame phản hồi có ID 0x122. Frame này được gửi ngược về ECU Main để phục vụ việc giám sát và hiển thị trạng thái hệ thống. Quá trình phản hồi được kiểm tra đồng thời trên USB-CAN tool, UART log của ECU ABS và UART log của ECU Main. Kết quả quan sát được thể hiện từ Hình 4.7 đến Hình 4.11

No	Direction	Time scale	Frame Format	Frame Type	Frame Id	Data Length	Data(Hex)
617	Received	09:59:08 620	Data frame	Standard frame	0x00000122	4	00 00 03 2c
618	Received	09:59:08 732	Data frame	Standard frame	0x00000122	4	00 00 03 2d
619	Received	09:59:08 829	Data frame	Standard frame	0x00000122	4	00 00 03 2e
620	Received	09:59:08 951	Data frame	Standard frame	0x00000122	4	00 00 03 2f
621	Received	09:59:09 056	Data frame	Standard frame	0x00000122	4	00 00 03 30
622	Received	09:59:09 161	Data frame	Standard frame	0x00000122	4	00 00 03 31
623	Received	09:59:09 278	Data frame	Standard frame	0x00000122	4	00 00 03 32
624	Received	09:59:09 394	Data frame	Standard frame	0x00000122	4	00 00 03 33
625	Received	09:59:09 502	Data frame	Standard frame	0x00000122	4	00 00 03 34

Hình 4.8 Frame 0x122 từ ECU ABS về ECU Main trên USB-CAN tool.

```
[CAN TX] NODE=ABS | DST=MAIN | ID=0x122 | DLC=4 | W1=70 cm/s | W2=0 cm/s | STATE=1 | SEQ=248 | RESULT=OK | CNT=1017 | FAIL=0
[CAN DIAG] NODE=ABS | ID=0x212 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=211 | TX_FAIL=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=1021 | BRAKE=0 | DIR=0 | SEQ=30 | CNT=728 | LOST=327
[APP] NODE=ABS | MOTOR | ADC=1021 | PWM=20669 | BRAKE=0 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=1018 | BRAKE=0 | DIR=0 | SEQ=31 | CNT=729 | LOST=327
[APP] NODE=ABS | MOTOR | ADC=1018 | PWM=20661 | BRAKE=0 | DIR=0
[ENC] NODE=ABS | D1=316 | D2=0 | V1=69 cm/s | V2=0 cm/s
[CAN TX] NODE=ABS | DST=MAIN | ID=0x122 | DLC=4 | W1=69 cm/s | W2=0 cm/s | STATE=1 | SEQ=249 | RESULT=OK | CNT=1018 | FAIL=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=1018 | BRAKE=0 | DIR=0 | SEQ=32 | CNT=730 | LOST=327
[APP] NODE=ABS | MOTOR | ADC=1018 | PWM=20661 | BRAKE=0 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=1014 | BRAKE=0 | DIR=0 | SEQ=33 | CNT=731 | LOST=327
[APP] NODE=ABS | MOTOR | ADC=1014 | PWM=20651 | BRAKE=0 | DIR=0
[CAN RX] NODE=ABS | SRC=MAIN | ID=0x121 | DLC=5 | ADC=1013 | BRAKE=0 | DIR=0 | SEQ=34 | CNT=732 | LOST=327
[APP] NODE=ABS | MOTOR | ADC=1013 | PWM=20648 | BRAKE=0 | DIR=0
```

Hình 4.9 Log UART ECU ABS khi gửi frame phản hồi 0x122.

```
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=630 | TX_FAIL=0
[CAN RX] NODE=MAIN | SRC=ABS | ID=0x122 | DLC=4 | W1=44 cm/s | W2=0 cm/s | STATE=1 | SEQ=236 | CNT=717 | LOST=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=24 | RESULT=OK | CNT=1817 | RAW=FF 0F 00 00 18
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=25 | RESULT=OK | CNT=1818 | RAW=FF 0F 00 00 19
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=26 | RESULT=OK | CNT=1819 | RAW=FF 0F 00 00 1A
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=631 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=27 | RESULT=OK | CNT=1820 | RAW=FF 0F 00 00 1B
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=28 | RESULT=OK | CNT=1821 | RAW=FF 0F 00 00 1C
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=4095 | BRAKE=0 | DIR=0 | SEQ=29 | RESULT=OK | CNT=1822 | RAW=FF 0F 00 00 1D
```

Hình 4.10 Log UART Main khi nhận frame phản hồi 0x122 từ ABS.

```
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=1645 | TX_FAIL=0
[CAN RX] NODE=MAIN | SRC=ABS | ID=0x122 | DLC=4 | W1=35 cm/s | W2=0 cm/s | STATE=2 | SEQ=144 | CNT=1905 | LOST=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1631 | BRAKE=0 | DIR=1 | SEQ=231 | RESULT=OK | CNT=4840 | RAW=5F 06 00 01 E7
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1637 | BRAKE=0 | DIR=1 | SEQ=232 | RESULT=OK | CNT=4841 | RAW=65 06 00 01 E8
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1627 | BRAKE=0 | DIR=1 | SEQ=233 | RESULT=OK | CNT=4842 | RAW=5B 06 00 01 E9
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=1646 | TX_FAIL=0
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1629 | BRAKE=0 | DIR=1 | SEQ=234 | RESULT=OK | CNT=4843 | RAW=5D 06 00 01 EA
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1632 | BRAKE=0 | DIR=1 | SEQ=235 | RESULT=OK | CNT=4844 | RAW=60 06 00 01 EB
[CAN TX] NODE=MAIN | DST=ABS | ID=0x121 | DLC=5 | ADC=1646 | BRAKE=0 | DIR=1 | SEQ=236 | RESULT=OK | CNT=4845 | RAW=6E 06 00 01 EC
[CAN DIAG] NODE=MAIN | ID=0x211 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=1647 | TX_FAIL=0
```

Hình 4.11 Log UART Main khi nhận trạng thái ABS thay đổi chiều quay.

[CAN DIAG] NODE=MAIN	ID=0x211	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=2030	TX_FAIL=0
[CAN RX] NODE=MAIN	SRC=ABS	ID=0x122	DLC=4	W1=0 cm/s	W2=0 cm/s	STATE=3	SEQ=83	CNT=2356	LOST=0		
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1576	BRAKE=1	DIR=1	SEQ=96	RESULT=OK	CNT=5985	RAW=28 06 01 01 60	
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1637	BRAKE=1	DIR=1	SEQ=97	RESULT=OK	CNT=5986	RAW=65 06 01 01 61	
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1659	BRAKE=1	DIR=1	SEQ=98	RESULT=OK	CNT=5987	RAW=7B 06 01 01 62	
[CAN DIAG] NODE=MAIN	ID=0x211	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=2031	TX_FAIL=0
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1641	BRAKE=1	DIR=1	SEQ=99	RESULT=OK	CNT=5988	RAW=69 06 01 01 63	
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1632	BRAKE=1	DIR=1	SEQ=100	RESULT=OK	CNT=5989	RAW=60 06 01 01 64	
[CAN TX] NODE=MAIN	DST=ABS	ID=0x121	DLC=5	ADC=1633	BRAKE=1	DIR=1	SEQ=101	RESULT=OK	CNT=5990	RAW=61 06 01 01 65	

Hình 4.12 Log UART ECU Main khi nhận trạng thái ABS dừng motor.

Kết quả thực nghiệm

Kết quả thực nghiệm cho thấy ECU ABS đã gửi được frame phản hồi 0x122 về ECU Main sau khi nhận và xử lý frame điều khiển 0x121. Trên USB-CAN tool, frame 0x122 xuất hiện với định dạng Standard frame và có độ dài dữ liệu DLC bằng 4 byte, phù hợp với cấu trúc frame phản hồi đã thiết kế.

Trong UART log của ECU ABS, frame phản hồi có dạng:

[CAN TX] NODE=ABS | DST=MAIN | ID=0x122 | DLC=4 | W1=... | W2=... | STATE=... | SEQ=... | RESULT=OK. Dòng log này cho thấy ECU ABS đã đóng gói các dữ liệu phản hồi gồm W1, W2, STATE và SEQ rồi gửi về ECU Main. Trường RESULT=OK thể hiện frame phản hồi đã được truyền thành công.

Ở phía ECU Main, UART log cũng ghi nhận frame 0x122 từ ECU ABS. Điều này chứng minh quá trình truyền thông hai chiều giữa ECU Main và ECU ABS đã hoạt động đúng theo thiết kế. ECU Main không chỉ gửi lệnh điều khiển đến ABS thông qua frame 0x121 mà còn nhận được trạng thái phản hồi từ ABS thông qua frame 0x122.

Khi motor hoạt động, trường STATE trong frame 0x122 chuyển sang giá trị 1, tương ứng với trạng thái FORWARD. Khi motor dừng, STATE chuyển về giá trị 0, tương ứng với trạng thái STOP. Bộ đếm SEQ thay đổi qua từng frame, cho thấy ECU ABS gửi dữ liệu phản hồi định kỳ về Main. Kết quả này chứng minh frame 0x122 phản ánh được trạng thái hoạt động của ECU ABS trong quá trình thực nghiệm.

Từ kết quả trên, có thể kết luận rằng ECU ABS đã thực hiện đúng chức năng phản hồi trạng thái về ECU Main. Việc phản hồi frame 0x122 giúp ECU Main giám sát được trạng thái hoạt động của motor ABS, đồng thời chứng minh quá trình truyền nhận giữa ECU Main và ECU ABS hoạt động hai chiều và ổn định.

4.4.5 Thực nghiệm 5: Phát hiện va chạm trên ECU Airbag

Mục tiêu: Kiểm tra ECU Airbag có phát hiện va chạm dựa trên dữ liệu cảm biến gia tốc và gửi cảnh báo về ECU Main thông qua frame 0x111 hay không.

Trong thực nghiệm này, ECU Airbag được kiểm tra ở ba điều kiện khác nhau gồm trạng thái đứng yên, tác động nhẹ và tác động mạnh. Dữ liệu gia tốc theo ba trục X, Y và Z được thu thập từ cảm biến LIS331HH để xác định trạng thái va chạm. Khi giá trị gia tốc vượt ngưỡng thiết kế, ECU Airbag kích hoạt buzzer và gửi trạng thái DEPLOY về ECU Main thông qua frame 0x111. Kết quả thực nghiệm được tổng hợp trong Bảng 4.13.

Bảng 4.13 Kết quả thực nghiệm Airbag

Lần thử	Điều kiện tác động	X mg	Y mg	Z mg	DEPLOY Airbag	Buzzer	DEPLOY Main nhận	Đánh giá
1	Đứng yên	246	-534	-690	0	Tắt	0	Đạt
2	Lắc nhẹ	234	792	-612	0	Tắt	0	Đạt
3	Tác động mạnh	1020	3006	2430	1	Bật	0	Đạt

No	Directon	Time scale	Frame Format	Frame Type	Frame Id	Data Length	Data(Hex)
0	Received	20:59:42 267	Data frame	Standard frame	0x00000111	2	00 0e
1	Received	20:59:42 327	Data frame	Standard frame	0x00000111	2	00 0f
2	Received	20:59:42 382	Data frame	Standard frame	0x00000111	2	00 10
3	Received	20:59:42 834	Data frame	Standard frame	0x00000111	2	00 00
4	Received	20:59:42 902	Data frame	Standard frame	0x00000111	2	00 01
5	Received	20:59:42 950	Data frame	Standard frame	0x00000111	2	00 02
6	Received	20:59:43 013	Data frame	Standard frame	0x00000111	2	00 03
7	Received	20:59:43 075	Data frame	Standard frame	0x00000111	2	00 04

Hình 4.13 Airbag ở trạng thái bình thường

Hình thể hiện kết quả giám sát frame trạng thái Airbag trên bus CAN bằng USB-CAN tool trong điều kiện hoạt động bình thường. Các frame thu được đều có CAN ID 0x111, thuộc loại Standard frame và có độ dài dữ liệu DLC bằng 2 byte, phù hợp với cấu trúc frame Airbag đến Main đã thiết kế.

Theo quy ước payload của frame 0x111, byte đầu tiên biểu diễn trạng thái va chạm, trong đó giá trị 0 tương ứng với trạng thái chưa phát hiện va chạm và giá trị 1 tương ứng với trạng thái đã phát hiện va chạm. Byte thứ hai là bộ đếm thứ tự frame SEQ, được sử dụng để theo dõi sự thay đổi của các frame được truyền liên tiếp.

Từ kết quả quan sát, các payload lần lượt có dạng 00 0E, 00 0F, 00 10, 00 00, 00 01,... Trong các payload này, byte đầu tiên đều bằng 00, cho thấy ECU Airbag chưa phát hiện va chạm tại thời điểm thực nghiệm. Byte thứ hai thay đổi theo từng frame, thể hiện bộ đếm SEQ đang tăng dần và ECU Airbag đang gửi dữ liệu trạng thái định kỳ lên bus CAN.

Kết quả này chứng minh rằng ECU Airbag đã truyền thành công frame trạng thái 0x111 lên bus CAN trong điều kiện bình thường. Đồng thời, việc frame có đúng CAN ID, đúng DLC và payload đúng cấu trúc cho thấy quá trình đóng gói và truyền dữ liệu từ ECU Airbag đến ECU Main hoạt động đúng theo thiết kế.

No	Direction	Time scale	Frame Format	Frame Type	Frame Id	Data Length	Data(Hex)
254	Received	21:01:13 639	Data frame	Standard frame	0x00000111	2	01 fc
255	Received	21:01:13 703	Data frame	Standard frame	0x00000111	2	01 fd
256	Received	21:01:13 754	Data frame	Standard frame	0x00000111	2	01 fe
257	Received	21:01:13 811	Data frame	Standard frame	0x00000111	2	01 ff
258	Received	21:01:13 868	Data frame	Standard frame	0x00000111	2	01 00
259	Received	21:01:13 927	Data frame	Standard frame	0x00000111	2	01 01
260	Received	21:01:13 980	Data frame	Standard frame	0x00000111	2	01 02
261	Received	21:01:14 044	Data frame	Standard frame	0x00000111	2	01 03

Hình 4.14 Airbag ở trạng thái phát hiện va chạm

Từ kết quả quan sát, các payload có dạng 01 FC, 01 FD, 01 FE, 01 FF, 01 00, 01 01, 01 02 và 01 03. Trong các payload này, byte đầu tiên đều bằng 01, cho thấy ECU Airbag đã chuyển sang trạng thái phát hiện va chạm. Byte thứ hai thay đổi liên tục từ FC đến FF, sau đó quay về 00 và tiếp tục tăng lên 01, 02, 03. Điều này thể hiện bộ đếm SEQ hoạt động đúng và có hiện tượng quay vòng sau khi đạt giá trị 0xFF.

Kết quả này chứng minh rằng khi phát hiện va chạm, ECU Airbag đã gửi thành công frame cảnh báo 0x111 lên bus CAN với trạng thái DEPLOY=1. Việc frame có đúng CAN ID, đúng DLC và payload phản ánh đúng trạng thái va chạm cho thấy chức năng phát hiện và truyền cảnh báo của ECU Airbag hoạt động đúng theo thiết kế.

```
[CAN TX]  NODE=AIRBAG, DST=MAIN, ID=0x111, DLC=2 DEPLOY=1 SEQ=189 RESULT=OK CNT=445 FAIL=0
[CAN DIAG] NODE=AIRBAG, ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 EPVF=0 BOFF=0 STATE=ACTIVE MODE=STARTED TX_OK=224 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 REC=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=225 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 REC=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=226 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 ENGf=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=227 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 REC=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=229 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 ENGf=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=230 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 ENGf=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=231 TX_FAIL=0
[CAN DIAG] NODE=AIRBAG ID=0x213 LEC=0(NO ERROR) TEC=0 REC=0 ENGf=0 EPVF=0 STATE=ACTIVE MODE=STARTED TX_OK=232 TX_FAIL=0
```

Hình 4.15 Log Airbag khi phát hiện va chạm

Bảng 4.14 Chuỗi hoạt động đa ECU trong một chu kỳ thực nghiệm

Thứ tự	Node phát	Node nhận	CAN ID	Nội dung	Kết quả quan sát
1	Main	ABS	0x121	Lệnh điều khiển motor	Motor hoạt động ổn định theo lệnh từ main
2	ABS	Main	0x122	Phản hồi tốc độ/trạng thái	LCD Main hiển thị trạng thái ABS gửi
3	Airbag	Main	0x111	Trạng thái va chạm	LCD Main hiển thị cảnh báo và motor dừng
4	Main	Bus	0x211	Diagnostic Main	Hiển thị được trên terminal UART và PC
5	ABS	Bus	0x212	Diagnostic ABS	Hiển thị được trên terminal UART và PC
6	Airbag	Bus	0x213	Diagnostic Airbag	Hiển thị được trên terminal UART và PC

Total:	1000	☐ Display receive only	☐ Overlay display	☒ Pause	Clear	Save	☐ Auto Save
No	Direction	Time scale	Frame Format	Frame Type	Frame Id	Data Length	Data(Hex)
0	Received	19:21:41 153	Data frame	Standard frame	0x00000111	2	00 81
1	Received	19:21:41 163	Data frame	Standard frame	0x00000121	5	37 03 00 00 82
2	Received	19:21:41 174	Data frame	Standard frame	0x00000211	8	00 00 00 00 00 00 00 01
3	Received	19:21:41 187	Data frame	Standard frame	0x00000122	4	40 01 01 af
4	Received	19:21:41 208	Data frame	Standard frame	0x00000121	5	38 03 00 00 83
5	Received	19:21:41 229	Data frame	Standard frame	0x00000111	2	00 82
6	Received	19:21:41 229	Data frame	Standard frame	0x00000213	8	00 00 00 00 00 00 00 01
7	Received	19:21:41 229	Data frame	Standard frame	0x00000212	8	00 00 00 00 00 00 00 01
8	Received	19:21:41 240	Data frame	Standard frame	0x00000121	5	36 03 00 00 84
9	Received	19:21:41 270	Data frame	Standard frame	0x00000121	5	31 03 00 00 85
10	Received	19:21:41 284	Data frame	Standard frame	0x00000211	8	00 00 00 00 00 00 00 01
11	Received	19:21:41 294	Data frame	Standard frame	0x00000111	2	00 83
12	Received	19:21:41 305	Data frame	Standard frame	0x00000122	4	42 00 01 b0
13	Received	19:21:41 314	Data frame	Standard frame	0x00000121	5	37 03 00 00 86
14	Received	19:21:41 338	Data frame	Standard frame	0x00000111	2	00 84
15	Received	19:21:41 340	Data frame	Standard frame	0x00000213	8	00 00 00 00 00 00 00 01
16	Received	19:21:41 351	Data frame	Standard frame	0x00000121	5	38 03 00 00 87
17	Received	19:21:41 381	Data frame	Standard frame	0x00000121	5	41 03 00 00 88
18	Received	19:21:41 394	Data frame	Standard frame	0x00000211	8	00 00 00 00 00 00 00 01

Hình 4.18 Các frame được truyền trên USB-CAN

Kết quả quan sát trên USB-CAN tool cho thấy nhiều frame CAN với các ID khác nhau xuất hiện xen kẽ trên cùng một bus CAN. Các frame chức năng gồm 0x121, 0x122 và 0x111 được truyền đúng theo hướng dữ liệu đã thiết kế. Trong đó, ECU Main gửi frame 0x121 để điều khiển ECU ABS, ECU ABS phản hồi frame 0x122 về ECU Main, còn ECU Airbag gửi trạng thái va chạm thông qua frame 0x111.

Ngoài các frame chức năng, các frame diagnostic 0x211, 0x212 và 0x213 cũng xuất hiện trên bus CAN. Điều này cho thấy trong khi hệ thống đang thực hiện truyền nhận dữ liệu điều khiển, các ECU vẫn có thể gửi thông tin chẩn đoán để phục vụ giám sát lỗi CAN. Việc các frame có thể xuất hiện xen kẽ nhưng vẫn giữ đúng CAN ID, đúng chức năng và đúng node phát cho thấy quá trình phân bổ ID và tổ chức truyền thông trên bus CAN là phù hợp.

Trong quá trình thực nghiệm, ECU Main vẫn nhận được phản hồi trạng thái từ ECU ABS và cảnh báo từ ECU Airbag. Đồng thời, motor ABS hoạt động theo lệnh điều khiển từ Main và có thể dừng khi hệ thống nhận được cảnh báo nguy hiểm từ Airbag. Điều này chứng minh ba ECU không hoạt động độc lập riêng lẻ mà có khả năng phối hợp thông qua mạng CAN để tạo thành một hệ thống đa ECU hoàn chỉnh.

Từ kết quả thực nghiệm, có thể đánh giá rằng hệ thống đã đáp ứng được yêu cầu hoạt động đồng thời của nhiều ECU trên cùng một bus CAN. Các frame chức năng và frame diagnostic được truyền xen kẽ nhưng vẫn giữ đúng CAN ID, đúng chức năng và không gây nhầm lẫn dữ liệu giữa các node.

4.4.7 Thực nghiệm 7: Giám sát lỗi ACK

Mục tiêu: Kiểm tra khả năng phát hiện lỗi ACK của node phát khi không có node nhận xác nhận frame CAN.

Trong thực nghiệm này, hệ thống được tạo tình huống một node phát frame CAN nhưng không nhận được tín hiệu ACK từ các node còn lại. Khi không có node xác nhận frame, bộ điều khiển CAN của node phát sẽ ghi nhận ACK Error và làm thay đổi các thông số lỗi như LEC và TEC. Các giá trị diagnostic được quan sát thông qua UART log để đánh giá phản ứng của hệ thống khi xảy ra lỗi ACK. Kết quả thực nghiệm được trình bày trong Bảng 4.15.

Bảng 4.15 Kết quả thực nghiệm lỗi ACK

Trạng thái bus	Điều kiện test	LEC	TEC	REC	EWGF	EPVF	BOFF	Hiện tượng	Đánh giá
Bình thường	3 ECU hoạt động	0	0	0	0	0	0	Truyền nhận OK	Đạt
Lỗi ACK	Chỉ Airbag hoạt động	3	128	0	1	1		Airbag không	Đạt

							0	nhận ACK	
Phục hồi	Bật lại ABS/Airbag	0	3006	0	0	0	0	Truyền nhận lại	Đạt

```
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=39 | TX_FAIL=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=0
[SENSOR] NODE=AIRBAG | RAW_X=9 | RAW_Y=-159 | RAW_Z=-30 | MG_X=54 | MG_Y=-954 | MG_Z=-180 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=1
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=2
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=41 | RESULT=OK | CNT=41 | FAIL=1
[SENSOR] NODE=AIRBAG | RAW_X=1 | RAW_Y=-158 | RAW_Z=-40 | MG_X=6 | MG_Y=-948 | MG_Z=-240 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=3
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=4
[SENSOR] NODE=AIRBAG | RAW_X=7 | RAW_Y=-155 | RAW_Z=-36 | MG_X=42 | MG_Y=-930 | MG_Z=-216 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=5
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=6
[SENSOR] NODE=AIRBAG | RAW_X=11 | RAW_Y=-159 | RAW_Z=-33 | MG_X=66 | MG_Y=-954 | MG_Z=-198 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=ACK_ERROR
```

Hình 4.19 Log ECU Airbag khi xảy ra ACK Error.

```
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=3(ACK ERROR) | TEC=128 | REC=0 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=40 | TX_FAIL=665
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=125 | REC=0 | EWGF=1 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=41 | TX_FAIL=665
[SENSOR] NODE=AIRBAG | RAW_X=10 | RAW_Y=-158 | RAW_Z=-35 | MG_X=60 | MG_Y=-948 | MG_Z=-210 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=42 | RESULT=OK | CNT=42 | FAIL=664
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=123 | REC=0 | EWGF=1 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=42 | TX_FAIL=665
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=121 | REC=249 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=43 | TX_FAIL=665
[SENSOR] NODE=AIRBAG | RAW_X=-1 | RAW_Y=-161 | RAW_Z=-36 | MG_X=-6 | MG_Y=-966 | MG_Z=-216 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=121 | REC=249 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=43 | TX_FAIL=666
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=121 | REC=249 | EWGF=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=43 | TX_FAIL=667
[SENSOR] NODE=AIRBAG | RAW_X=7 | RAW_Y=-155 | RAW_Z=-34 | MG_X=42 | MG_Y=-930 | MG_Z=-204 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
```

Hình 4.20 Log ECU Airbag trong quá trình phục hồi sau lỗi ACK.

```
[SENSOR] NODE=AIRBAG | RAW_X=7 | RAW_Y=-159 | RAW_Z=-32 | MG_X=42 | MG_Y=-954 | MG_Z=-192 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=22 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=92 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=20 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=93 | TX_FAIL=677
[SENSOR] NODE=AIRBAG | RAW_X=10 | RAW_Y=-157 | RAW_Z=-47 | MG_X=60 | MG_Y=-942 | MG_Z=-282 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=18 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=94 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=16 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=95 | TX_FAIL=677
[SENSOR] NODE=AIRBAG | RAW_X=13 | RAW_Y=-156 | RAW_Z=-34 | MG_X=78 | MG_Y=-936 | MG_Z=-204 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=14 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=96 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=12 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=97 | TX_FAIL=677
[SENSOR] NODE=AIRBAG | RAW_X=12 | RAW_Y=-145 | RAW_Z=-43 | MG_X=72 | MG_Y=-870 | MG_Z=-258 | DEPLOY=0
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=99 | RESULT=OK | CNT=99 | FAIL=675
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=10 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=98 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=8 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=99 | TX_FAIL=677
[SENSOR] NODE=AIRBAG | RAW_X=11 | RAW_Y=-158 | RAW_Z=-31 | MG_X=66 | MG_Y=-948 | MG_Z=-186 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=6 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=100 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=4 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=101 | TX_FAIL=677
[SENSOR] NODE=AIRBAG | RAW_X=5 | RAW_Y=-155 | RAW_Z=-38 | MG_X=30 | MG_Y=-930 | MG_Z=-228 | DEPLOY=0
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=2 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=102 | TX_FAIL=677
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=0(NO ERROR) | TEC=0 | REC=0 | EWGF=0 | EPVF=0 | BOFF=0 | STATE=ACTIVE | MODE=STARTED | TX_OK=103 | TX_FAIL=677
```

Hình 4.21 Log ECU Airbag sau khi TEC trở về 0 và trạng thái CAN phục hồi bình thường.

Khi chỉ còn ECU Airbag hoạt động trên bus, các frame do Airbag phát ra không nhận được tín hiệu ACK từ node khác. Do đó, bộ điều khiển CAN của Airbag ghi nhận lỗi ACK. Kết quả log diagnostic cho thấy LEC chuyển sang ACK Error và TEC tăng so với trạng thái bình thường. Điều này phù hợp với cơ chế lỗi của CAN, trong đó lỗi xảy ra trong quá trình phát làm tăng bộ đếm lỗi truyền của node phát. Khi các node nhận được cấp nguồn trở lại, quá trình truyền nhận được phục hồi.

Từ kết quả trên, thực nghiệm được đánh giá là đạt yêu cầu vì hệ thống đã ghi nhận đúng lỗi ACK và phục hồi truyền thông khi các node được kết nối trở lại.

4.4.8 Thực nghiệm 8: Lỗi đường truyền CAN

Mục tiêu: Kiểm tra phản ứng của hệ thống khi đường truyền CAN_H hoặc CAN_L bị tác động bất thường.

Bảng 4.16 Kết quả thực nghiệm lỗi đường truyền CAN

Trường hợp	Cách tạo lỗi	LEC	TEC	REC	EWGF	EPVF	BO FF	Hiện tượng quan sát	Đánh giá
Bình thường	Không tác động	0	0	0	0	0	0	Truyền nhận ổn định	Đạt/Không đạt
Tác động CAN_H	Ngắt/tác động CAN_H	3	125	0	1	1	0	Log lỗi xuất hiện	Đạt
Tác	Ngắt/tác	3	125	0	1	1	0	Log	Đạt

động CAN_L	động CAN_L							lỗi xuất hiện	
Bỏ node	Không cấp nguồn module CAN	5	0	Tăng mạnh	1	0	1	Bus off	Đạt
Phục hồi	Kết nối lại	0	0	0	0	0	0	CAN hoạt động lại	Đạt

Trong thực nghiệm lỗi đường truyền CAN, hệ thống được kiểm tra bằng cách thay đổi trạng thái kết nối của một node trên bus và tác động vào điều kiện truyền thông của mạng CAN. Mục đích của thực nghiệm này là quan sát sự thay đổi của các thông số diagnostic như LEC, TEC, REC, EWGF, EPVF và BOFF khi bus CAN không còn ở trạng thái truyền nhận bình thường.

Ở trường hợp thứ nhất, node Airbag được rút khỏi bus CAN trong khi các node còn lại vẫn hoạt động. Khi một node bị ngắt khỏi bus, các frame truyền trên mạng CAN có thể không còn nhận được phản hồi hoặc xác nhận như trong trạng thái bình thường. Kết quả log diagnostic cho thấy bộ điều khiển CAN ghi nhận sự thay đổi của mã lỗi LEC và các cờ trạng thái lỗi. Điều này chứng minh hệ thống có khả năng phát hiện sự bất thường khi một node bị mất kết nối khỏi bus.

[SENSOR] NODE=AIRBAG	RAW_X=10	RAW_Y=-157	RAW_Z=-36	MG_X=60	MG_Y=-942	MG_Z=-216	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=8	REC=249	EWGF=1	EPVF=1	BOFF=0	STATE=PASSIVE	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	DLC=2	DEPLOY=0	SEQ=43	RESULT=OK	CNT=43	FAIL=0											
CAN BUS-OFF detected																			
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=0	REC=126	EWGF=1	EPVF=0	BOFF=1	STATE=BUS-OFF	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=10	RAW_Y=-155	RAW_Z=-44	MG_X=60	MG_Y=-930	MG_Z=-264	DEPLOY=0												
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=0	REC=29	EWGF=1	EPVF=0	BOFF=1	STATE=BUS-OFF	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=0	REC=45	EWGF=1	EPVF=0	BOFF=1	STATE=BUS-OFF	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=9	RAW_Y=-155	RAW_Z=-36	MG_X=54	MG_Y=-930	MG_Z=-216	DEPLOY=0												
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=160	REC=0	EWGF=1	EPVF=1	BOFF=0	STATE=PASSIVE	MODE=STARTED	TX_OK=43	TX_FAIL=0								
AICAN BUS-OFF detected																			
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=0	REC=81	EWGF=1	EPVF=0	BOFF=1	STATE=BUS-OFF	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=8	RAW_Y=-154	RAW_Z=-39	MG_X=48	MG_Y=-924	MG_Z=-234	DEPLOY=0												
CAN BUS-OFF detected																			
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=5 (BIT ERROR - DOMINANT)	TEC=0	REC=87	EWGF=1	EPVF=0	BOFF=1	STATE=BUS-OFF	MODE=STARTED	TX_OK=43	TX_FAIL=0								
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	RESULT=FAIL	REASON=BUS OFF															

Hình 4.22 Log diagnostic khi rút bỏ node CAN Airbag khỏi bus.

Ở trường hợp thứ hai, hệ thống ghi nhận lỗi Stuff Error trong quá trình truyền thông CAN. Stuff Error xảy ra khi quy tắc chèn bit trong frame CAN bị vi phạm, làm cho bộ điều khiển CAN nhận diện frame không còn hợp lệ. Trong log diagnostic, lỗi này được thể hiện thông qua giá trị LEC tương ứng với Stuff Error. Kết quả này cho thấy cơ chế diagnostic có thể ghi nhận được lỗi phát sinh trong quá trình mã hóa và truyền dữ liệu trên bus CAN.

[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=740	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=741	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=-16	RAW_Y=-155	RAW_Z=-34	MG_X=-96	MG_Y=-930	MG_Z=-204	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=742	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=743	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=-17	RAW_Y=-155	RAW_Z=-32	MG_X=-102	MG_Y=-930	MG_Z=-192	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=744	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=745	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=-17	RAW_Y=-156	RAW_Z=-33	MG_X=-102	MG_Y=-936	MG_Z=-198	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=746	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=747	TX_FAIL=0								
[CAN TX] NODE=AIRBAG	DST=MAIN	ID=0x111	DLC=2	DEPLOY=0	SEQ=235	RESULT=OK	CNT=747	FAIL=0											
[SENSOR] NODE=AIRBAG	RAW_X=-18	RAW_Y=-154	RAW_Z=-33	MG_X=-108	MG_Y=-924	MG_Z=-198	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=748	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=749	TX_FAIL=0								
[SENSOR] NODE=AIRBAG	RAW_X=-16	RAW_Y=-155	RAW_Z=-33	MG_X=-96	MG_Y=-930	MG_Z=-198	DEPLOY=0												
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=1 (STUFF ERROR)	TEC=0	REC=1	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=750	TX_FAIL=0								
[CAN DIAG] NODE=AIRBAG	ID=0x213	LEC=0 (NO ERROR)	TEC=0	REC=0	EWGF=0	EPVF=0	BOFF=0	STATE=ACTIVE	MODE=STARTED	TX_OK=751	TX_FAIL=0								

Hình 4.23 Log diagnostic khi xuất hiện Stuff Error trên bus CAN.

Ở trường hợp thứ ba, khi tác động đến nguồn điện hoặc trạng thái vật lý của bus, hệ thống ghi nhận lỗi Form Error. Form Error cho thấy một số trường cố định trong frame CAN không còn đúng định dạng theo quy định của giao thức CAN. Khi lỗi này xuất hiện, log diagnostic thể hiện sự thay đổi của LEC và các thông số lỗi liên quan. Điều này chứng minh rằng hệ thống có thể phản ánh các lỗi bất thường do nhiễu hoặc tác động vật lý đến quá trình truyền thông.

```
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=9
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=10
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=11
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=12
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | DLC=2 | DEPLOY=0 | SEQ=153 | RESULT=OK | CNT=153 | FAIL=11
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=13
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=14
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
[CAN TX] NODE=AIRBAG | DST=MAIN | ID=0x111 | RESULT=FAIL | REASON=MAILBOX_FULL_OR_BUSY
[CAN DIAG] NODE=AIRBAG | ID=0x213 | LEC=2(FORM ERROR) | TEC=0 | REC=254 | ENG=1 | EPVF=1 | BOFF=0 | STATE=PASSIVE | MODE=STARTED | TX_OK=152 | TX_FAIL=15
[SENSOR] NODE=AIRBAG | RESULT=READ_FAIL
```

Hình 4.24 Log diagnostic khi xuất hiện Form Error do tác động nguồn điện.

Tổng hợp các kết quả trên cho thấy khi trạng thái vật lý của bus CAN bị thay đổi hoặc khi frame truyền thông không còn hợp lệ, các trường diagnostic của bộ điều khiển CAN đều có sự thay đổi tương ứng. Các lỗi như ACK Error, Stuff Error hoặc Form Error có thể xuất hiện tùy theo cách tạo lỗi và trạng thái của bus tại thời điểm thực nghiệm. Điều này chứng minh cơ chế giám sát lỗi CAN ở mức thanh ghi đã hoạt động và có thể hỗ trợ quá trình phát hiện, phân tích lỗi trong hệ thống đa ECU.

4.4.9 Thực nghiệm 9: Timeout và phản ứng an toàn của ECU ABS

Mục tiêu: Kiểm tra ECU ABS có đưa motor về trạng thái an toàn khi không nhận được frame điều khiển từ ECU Main hay không.

Theo chương trình điều khiển hiện tại, ECU ABS được thiết lập ngưỡng timeout 300 ms. Nếu quá thời gian này mà không nhận được lệnh từ ECU Main, ABS sẽ chuyển về trạng thái an toàn.

Trong thực nghiệm này, ECU Main được cho ngừng gửi frame điều khiển 0x121 đến ECU ABS để kiểm tra cơ chế timeout. Theo thiết kế phần mềm, nếu ECU ABS không nhận được lệnh điều khiển từ ECU Main trong khoảng thời gian lớn hơn 300 ms, hệ thống sẽ đưa motor về trạng thái dừng nhằm đảm bảo an toàn. Sau đó, ECU Main được cho gửi lại frame hợp lệ để kiểm tra khả năng phục hồi của ECU ABS. Kết quả thực nghiệm timeout được trình bày trong Bảng 4.17.

Bảng 4.17 Kết quả thực nghiệm timeout ABS

Trạng thái	Điều kiện	Thời gian	Hiện tượng	Đánh giá
------------	-----------	-----------	------------	----------

		mất frame 0x121	motor	
Bình thường	Main gửi đều	0	Motor chạy	Đạt/Không đạt
Mất lệnh	Main ngừng gửi	>300 ms	Motor dừng	Đạt
Phục hồi	Main gửi lại	0	Hoạt động lại theo thiết kế	Đạt

4.5 KẾT QUẢ THỰC NGHIỆM TỔNG HỢP

Qua các kịch bản thực nghiệm đã thực hiện, hệ thống đa ECU giao tiếp CAN đã hoạt động đúng theo mục tiêu thiết kế. Các ECU Main, ABS và Airbag đều có khả năng khởi động, truyền nhận dữ liệu và phản hồi trạng thái thông qua các frame CAN đã quy ước. Các frame chức năng như 0x121, 0x122 và 0x111 xuất hiện đúng trên bus CAN, có DLC phù hợp và payload phản ánh đúng trạng thái hoạt động của từng ECU.

Đối với quá trình truyền thông giữa ECU Main và ECU ABS, kết quả thực nghiệm cho thấy ECU Main gửi được frame điều khiển đến ABS, trong khi ECU ABS giải mã được các trường dữ liệu như ADC, BRAKE, DIR và SEQ. Dựa trên dữ liệu ADC nhận được, ECU ABS đã chuyển đổi thành tín hiệu PWM để điều khiển motor. Khi ADC nằm trong vùng chết, PWM được đưa về 0 nên motor không quay, giúp hạn chế hiện tượng motor bị giật khi tín hiệu điều khiển nhỏ. Khi ADC tăng, PWM tăng tương ứng và motor hoạt động ổn định theo thiết kế.

Đối với chiều phản hồi từ ECU ABS về ECU Main, frame 0x122 đã được truyền thành công và chứa các thông tin trạng thái như W1, W2, STATE và SEQ. Điều này chứng minh quá trình truyền thông giữa Main và ABS không chỉ hoạt động một chiều mà đã đảm bảo được cơ chế trao đổi hai chiều. ECU Main có thể nhận biết được trạng thái hoạt động của motor ABS, từ đó hỗ trợ việc giám sát hệ thống trong quá trình vận hành.

Đối với ECU Airbag, kết quả thực nghiệm cho thấy node này có khả năng đọc dữ liệu cảm biến gia tốc, xác định trạng thái va chạm và gửi cảnh báo về ECU Main thông qua frame 0x111. Khi chưa có va chạm, giá trị DEPLOY bằng 0 và

buzzer không hoạt động. Khi có tác động mạnh vượt ngưỡng, DEPLOY chuyển sang 1, buzzer được kích hoạt và ECU Main nhận được cảnh báo. Điều này cho thấy chức năng phát hiện va chạm và truyền cảnh báo của ECU Airbag hoạt động đúng theo thiết kế.

Trong thực nghiệm hệ thống đa ECU hoạt động đồng thời, các frame của Main, ABS, Airbag và các frame diagnostic đều xuất hiện trên cùng một bus CAN. Việc các frame có thể truyền xen kẽ mà không gây xung đột dữ liệu cho thấy cơ chế phân bổ CAN ID và chu kỳ truyền thông của hệ thống là phù hợp. ECU Main có thể vừa gửi lệnh điều khiển đến ABS, vừa nhận phản hồi từ ABS và Airbag, đồng thời hệ thống vẫn duy trì được khả năng giám sát lỗi thông qua các frame diagnostic.

Đối với cơ chế diagnostic CAN, các thông số như LEC, TEC, REC, EWGF, EPVF và BOFF đã phản ánh được trạng thái lỗi của từng node trong quá trình thực nghiệm. Khi hệ thống hoạt động bình thường, các giá trị lỗi duy trì ở mức thấp hoặc bằng 0. Khi tạo các tình huống lỗi như mất ACK, tác động đường truyền CAN_H/CAN_L hoặc thay đổi trạng thái node trên bus, các giá trị diagnostic thay đổi tương ứng. Điều này chứng minh việc đọc và truyền thông tin lỗi từ thanh ghi CAN ở mức thấp đã hoạt động đúng và có ý nghĩa trong việc giám sát trạng thái mạng CAN.

Thực nghiệm timeout của ECU ABS cũng cho thấy hệ thống có cơ chế phản ứng an toàn khi mất frame điều khiển từ ECU Main. Khi ABS không nhận được lệnh điều khiển trong thời gian vượt quá ngưỡng 300 ms, motor được đưa về trạng thái dừng nhằm tránh hoạt động ngoài kiểm soát. Khi ECU Main gửi lại frame hợp lệ, ECU ABS phục hồi hoạt động theo thiết kế. Kết quả này cho thấy hệ thống không chỉ truyền nhận dữ liệu bình thường mà còn có khả năng xử lý một số tình huống bất thường trong quá trình vận hành.

Tổng hợp các kết quả trên, có thể nhận xét rằng hệ thống thực nghiệm đã đáp ứng được các yêu cầu chính của đề tài. Việc triển khai CAN ở mức thanh ghi giúp quan sát được chi tiết quá trình truyền nhận và trạng thái lỗi của bộ điều khiển CAN. Đồng thời, việc tổ chức phần mềm theo kiến trúc AUTOSAR Classic thu gọn giúp hệ thống có cấu trúc rõ ràng, phân tách được giữa phần ứng dụng, lớp trung gian và lớp truy cập phần cứng. Tuy hệ thống vẫn còn ở quy mô mô hình thực nghiệm, các kết quả thu được cho thấy hướng tiếp cận của đề tài là phù hợp và có thể mở rộng cho các hệ thống đa ECU phức tạp hơn trong tương lai.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế, thi công và thực nghiệm, đề tài “Thiết kế hệ thống đa ECU giao tiếp CAN mức thanh ghi theo kiến trúc AUTOSAR Classic thu gọn” đã hoàn thành các mục tiêu chính đã đặt ra. Nhóm đã xây dựng được mô hình thực nghiệm gồm ba ECU: ECU Main, ECU ABS và ECU Airbag. Các ECU được triển khai trên nền tảng vi điều khiển STM32, kết nối với nhau thông qua mạng CAN sử dụng transceiver SN65HVD230 và trao đổi dữ liệu theo các CAN ID đã được thiết kế trước.

Về phần cứng, hệ thống đã tích hợp được các thành phần chính phục vụ cho mô hình đa ECU, bao gồm vi điều khiển STM32F407 cho ECU Main, STM32F103 cho ECU ABS và ECU Airbag, module CAN SN65HVD230, mạch điều khiển động cơ XY-160D, động cơ DC GA25-370 có encoder, cảm biến gia tốc LIS331HH, màn hình LCD và các nút nhấn điều khiển. Các khối phần cứng đã phối hợp được với nhau để tạo thành một mô hình thực nghiệm có khả năng mô phỏng cơ bản hoạt động của hệ thống điện tử ô tô gồm chức năng điều khiển ABS, phát hiện va chạm Airbag và giám sát trạng thái hệ thống từ ECU Main.

Về phần mềm, đề tài đã tổ chức chương trình theo hướng AUTOSAR Classic thu gọn. Các thành phần phần mềm được phân chia thành các lớp chức năng như Application Layer, RTE, các lớp truyền thông và lớp driver truy cập phần cứng. Việc phân lớp này giúp tách biệt phần xử lý ứng dụng với phần điều khiển phần cứng, làm cho cấu trúc chương trình rõ ràng hơn, dễ theo dõi và thuận lợi cho việc mở rộng trong tương lai. Đặc biệt, driver CAN được xây dựng theo phương pháp truy cập trực tiếp thanh ghi thay vì sử dụng thư viện mức cao, qua đó giúp quan sát rõ hơn quá trình cấu hình, truyền nhận và xử lý lỗi của bộ điều khiển CAN trên vi điều khiển STM32.

Về truyền thông CAN, hệ thống đã thiết kế và triển khai thành công các frame chức năng giữa các ECU. ECU Main gửi frame điều khiển 0x121 đến ECU ABS để truyền các thông tin như giá trị ADC, trạng thái phanh, chiều quay và bộ đếm SEQ. ECU ABS phản hồi trạng thái động cơ về ECU Main thông qua frame

0x122. ECU Airbag gửi trạng thái phát hiện va chạm về ECU Main thông qua frame 0x111. Ngoài ra, các frame diagnostic 0x211, 0x212 và 0x213 được sử dụng để giám sát trạng thái lỗi CAN của từng node trên bus.

Kết quả thực nghiệm cho thấy quá trình truyền nhận dữ liệu giữa các ECU hoạt động đúng theo thiết kế. ECU Main có thể gửi lệnh điều khiển đến ECU ABS, ECU ABS giải mã được dữ liệu và chuyển đổi giá trị ADC thành tín hiệu PWM để điều khiển động cơ. Khi giá trị ADC nằm trong vùng chết, PWM được đưa về 0 giúp hạn chế hiện tượng motor bị giật ở vùng tín hiệu nhỏ. Khi ADC tăng, PWM tăng tương ứng trong giới hạn thiết kế và motor hoạt động ổn định. Đồng thời, ECU ABS có thể gửi phản hồi trạng thái motor về ECU Main để phục vụ việc giám sát và hiển thị.

Đối với ECU Airbag, hệ thống đã thực hiện được chức năng đọc dữ liệu cảm biến gia tốc, xác định trạng thái va chạm và gửi cảnh báo về ECU Main. Khi chưa có va chạm, trạng thái DEPLOY bằng 0 và buzzer không hoạt động. Khi có tác động mạnh vượt ngưỡng, ECU Airbag chuyển trạng thái DEPLOY sang 1, kích hoạt buzzer và truyền frame cảnh báo lên bus CAN. ECU Main nhận được trạng thái cảnh báo và thực hiện phản ứng an toàn thông qua cơ chế emergency_stop, ưu tiên đưa hệ thống về trạng thái phanh hoặc dừng.

Bên cạnh các frame chức năng, đề tài cũng đã triển khai cơ chế giám sát lỗi CAN thông qua các thông số đọc từ thanh ghi trạng thái lỗi của bộ điều khiển CAN. Các giá trị như LEC, TEC, REC, EWGF, EPVF và BOFF được đóng gói vào frame diagnostic để phục vụ quá trình quan sát và phân tích. Trong các kịch bản tạo lỗi như lỗi ACK, tác động đường truyền CAN_H/CAN_L hoặc thay đổi trạng thái node trên bus, hệ thống đã ghi nhận được sự thay đổi của các thông số diagnostic. Điều này cho thấy việc triển khai giám sát lỗi CAN ở mức thanh ghi đã hoạt động và có thể hỗ trợ quá trình đánh giá cơ chế fault confinement trong mạng CAN.

Ngoài ra, cơ chế timeout trên ECU ABS cũng được kiểm chứng trong thực nghiệm. Khi ECU ABS không nhận được frame điều khiển từ ECU Main trong thời gian vượt quá ngưỡng 300 ms, motor được đưa về trạng thái dừng nhằm tránh hoạt động ngoài kiểm soát. Khi ECU Main gửi lại frame hợp lệ, ECU ABS phục hồi hoạt động theo thiết kế. Kết quả này cho thấy hệ thống không chỉ đáp ứng yêu cầu truyền nhận dữ liệu trong điều kiện bình thường mà còn có khả năng phản ứng an toàn trong một số tình huống bất thường.

Từ các kết quả đạt được, có thể kết luận rằng đề tài đã xây dựng thành công một mô hình thực nghiệm đa ECU giao tiếp CAN trên nền tảng STM32. Hệ thống đáp ứng được các yêu cầu cơ bản về truyền thông CAN, điều khiển động cơ ABS, phát hiện va chạm Airbag, giám sát diagnostic CAN và tổ chức phần

mềm theo kiến trúc AUTOSAR Classic thu gọn. Mặc dù mô hình vẫn còn ở quy mô thực nghiệm và chưa thể thay thế các hệ thống ECU thực tế trên ô tô, đề tài đã làm rõ được mối liên hệ giữa lý thuyết CAN, lập trình mức thanh ghi và thiết kế phần mềm phân lớp theo định hướng AUTOSAR. Đây là nền tảng quan trọng cho việc nghiên cứu, học tập và phát triển các hệ thống nhúng ô tô phức tạp hơn trong tương lai.

5.2 HƯỚNG PHÁT TRIỂN

Dựa trên những kết quả đã đạt được và các giới hạn còn tồn tại trong quá trình thực hiện, đề tài có thể tiếp tục được phát triển theo nhiều hướng nhằm nâng cao độ ổn định, tính thực tế và khả năng mở rộng của hệ thống.

Thứ nhất, hệ thống có thể được mở rộng thêm số lượng ECU và chức năng điều khiển. Trong phạm vi đề tài, mô hình mới triển khai ba ECU gồm Main ECU, ABS ECU và Airbag ECU. Trong tương lai, có thể bổ sung thêm các ECU khác như Engine ECU, Body ECU, Gateway ECU hoặc hệ thống đèn, xi-nhan, cảm biến khoảng cách để mô phỏng gần hơn kiến trúc điện – điện tử trên ô tô thực tế. Khi số lượng node tăng lên, việc đánh giá tải bus, độ trễ truyền thông và khả năng phân xử ưu tiên CAN ID cũng sẽ có ý nghĩa thực tiễn hơn.

Thứ hai, chức năng ABS có thể được phát triển sâu hơn. Hiện tại, ECU ABS chủ yếu mô phỏng quá trình nhận lệnh điều khiển, chuyển đổi ADC sang PWM, điều khiển motor và phản hồi tốc độ về ECU Main. Trong các nghiên cứu tiếp theo, có thể bổ sung thuật toán điều khiển chống bó cứng phanh dựa trên độ trượt bánh xe, tốc độ bánh xe và trạng thái phanh. Việc này sẽ giúp mô hình ABS không chỉ dừng ở mức điều khiển motor mà còn phản ánh rõ hơn nguyên lý hoạt động của hệ thống ABS trên ô tô.

Thứ ba, chức năng Airbag có thể được cải tiến theo hướng xử lý tín hiệu cảm biến chính xác hơn. Trong đề tài, trạng thái va chạm được xác định dựa trên ngưỡng gia tốc từ cảm biến LIS331HH. Để nâng cao độ tin cậy, có thể bổ sung các thuật toán lọc nhiễu, tính gia tốc tổng hợp, so sánh theo nhiều ngưỡng hoặc kết hợp thêm cảm biến khác. Điều này giúp giảm khả năng phát hiện sai trong các trường hợp rung động nhẹ nhưng chưa đủ điều kiện xem là va chạm.

Thứ tư, cơ chế diagnostic và fault confinement có thể được mở rộng theo hướng gần với AUTOSAR tiêu chuẩn hơn. Hiện tại, hệ thống đã đọc và truyền các thông số lỗi CAN như LEC, TEC, REC, EWGF, EPVF và BOFF. Trong tương lai, có thể phát triển thêm cơ chế lưu mã lỗi, phân loại lỗi theo mức độ nghiêm trọng, xây dựng bảng lỗi theo từng ECU hoặc mô phỏng một phần chức năng của các module AUTOSAR như DEM và DCM. Khi đó, hệ thống không

chỉ giám sát lỗi trên UART hoặc USB-CAN mà còn có thể phục vụ cho các bài toán chẩn đoán ECU hoàn chỉnh hơn.

Thứ năm, kiến trúc AUTOSAR Classic thu gọn có thể được hoàn thiện hơn. Trong phạm vi đề tài, các lớp phần mềm được xây dựng thủ công nhằm phù hợp với tài nguyên vi điều khiển và mục tiêu nghiên cứu. Ở các bước phát triển tiếp theo, có thể chuẩn hóa rõ hơn giao diện giữa các lớp Application, RTE, BSW và MCAL; bổ sung cấu hình tách riêng cho từng module; xây dựng cơ chế quản lý trạng thái ECU; hoặc phát triển thêm bộ sinh cấu hình đơn giản để giảm thao tác chỉnh sửa mã nguồn thủ công.

Thứ sáu, hệ thống có thể được cải thiện về khả năng kiểm thử và giám sát. Hiện tại, dữ liệu chủ yếu được quan sát thông qua UART log, LCD và USB-CAN tool. Trong tương lai, có thể xây dựng phần mềm giao diện trên máy tính để hiển thị trực quan các frame CAN, giá trị ADC, tốc độ motor, trạng thái Airbag, trạng thái diagnostic và biểu đồ thay đổi TEC/REC theo thời gian. Việc này sẽ giúp quá trình phân tích lỗi và đánh giá hệ thống trở nên trực quan, chính xác và thuận tiện hơn.

Thứ bảy, phần cứng của hệ thống có thể được tối ưu để tăng độ ổn định khi vận hành. Các hướng cải tiến bao gồm thiết kế PCB gọn hơn, bố trí đường CAN_H và CAN_L hợp lý hơn, bổ sung điện trở kết thúc bus đúng chuẩn, cải thiện khối nguồn, tách nguồn động cơ và nguồn logic rõ ràng hơn, đồng thời bổ sung các mạch bảo vệ chống nhiễu và chống đảo cực. Những cải tiến này giúp hệ thống hoạt động ổn định hơn khi mở rộng số lượng ECU hoặc khi thử nghiệm trong môi trường có nhiễu điện từ.

Thứ tám, đề tài có thể được phát triển theo hướng sử dụng các chuẩn truyền thông nâng cao hơn. Bên cạnh CAN 2.0A, có thể nghiên cứu thêm CAN FD để tăng tốc độ và dung lượng dữ liệu truyền, hoặc xây dựng mô hình gateway giữa CAN và các chuẩn truyền thông khác như LIN hoặc Ethernet Automotive. Đây là hướng phát triển phù hợp với xu thế các hệ thống điện – điện tử ô tô hiện đại ngày càng có yêu cầu truyền dữ liệu lớn và tích hợp nhiều miền chức năng khác nhau.

Cuối cùng, mô hình có thể được sử dụng như một nền tảng phục vụ đào tạo và nghiên cứu về hệ thống nhúng ô tô. Với cấu trúc gồm nhiều ECU, giao tiếp CAN, diagnostic lỗi, lập trình mức thanh ghi và kiến trúc phần mềm phân lớp, hệ thống có thể hỗ trợ sinh viên tiếp cận trực quan các nội dung như truyền thông CAN, điều khiển ngoại vi STM32, thiết kế phần mềm ECU, cơ chế fault confinement và tư tưởng AUTOSAR Classic. Đây là cơ sở để tiếp tục phát triển các đề tài nghiên cứu chuyên sâu hơn trong lĩnh vực hệ thống nhúng và điện tử ô tô.

PHỤ LỤC

Đường link code của ECU MAIN, ECU ABS, ECU AIRBAG:

**[https://drive.google.com/drive/folders/
1avJkBgSf8AOpawLdSlXwENgzzp2lUk9?usp=sharing](https://drive.google.com/drive/folders/1avJkBgSf8AOpawLdSlXwENgzzp2lUk9?usp=sharing)**

TÀI LIỆU THAM KHẢO

- [1] Navarro, J., et al., Automotive Embedded Systems Handbook, CRC Press, 2021.
- [2] Bosch, CAN Specification Version 2.0, Robert Bosch GmbH, 1991.
- [3] AUTOSAR Consortium, AUTOSAR Technical Overview, Release 4.4, 2022.
- [4] AUTOSAR Consortium, AUTOSAR Standard and Industry Adoption Report, 2023.
- [5] M. Broy, I. Krüger, A. Pretschner, C. Salzmann, "Engineering Automotive Software", Proceedings of the IEEE, Vol.95, No.2, 2007.
- [6] Kumar, R., Singh, P., "Design and Implementation of CAN-based Distributed Automotive Control System", International Journal of Automotive Technology, 2021.
- [7] Robert Bosch GmbH, Automotive Handbook, 10th Edition, 2018.
- [8] M. Broy, "Challenges in Automotive Software Engineering", ICSE 2006.
- [9] ISO 11898-1:2015, Road Vehicles – Controller Area Network (CAN).
- [10] H. Heinecke et al., "AUTOSAR – Current Results and Preparations for Exploiting Multi-Core Processors", SAE Technical Paper, 2013.
- [11] William B. Ribbens, Understanding Automotive Electronics, 8th Edition, 2017.
- [12] Ronald K. Jurgen, Automotive Electronics Handbook, McGraw-Hill, 1999.
- [13] AUTOSAR Consortium, AUTOSAR Classic Platform Overview, 2024.
- [14] Konrad Etschberger, Controller Area Network: Basics, Protocols, Chips and Applications, IXXAT Press, 2001.
- [15] Wilfried Voss, A Comprehensive Guide to Controller Area Network, Copperhill Technologies, 2020.

- [16] STMicroelectronics, RM0008 Reference Manual - STM32F10xxx bxCAN Controller.
- [17] AUTOSAR Consortium, AUTOSAR Classic Platform Overview, AUTOSAR, 2022.
- [18] AUTOSAR Consortium, Layered Software Architecture, Release R22-11, 2022.
- [19] UL Solutions, AUTOSAR – Part 1: The Classic Platform, 2018
- [20] MathWorks, What is AUTOSAR?, 2024.
- [21] AUTOSAR Consortium, AUTOSAR Runtime Environment Specification, Release R22-11.
- [22] MathWorks, AUTOSAR Runtime Environment (RTE) Overview, 2024.
- [23] AUTOSAR Consortium, Specification of Communication Stack, Release R22-11.
- [24] AUTOSAR Consortium, Specification of Microcontroller Abstraction Layer (MCAL), Release R22-11.
- [25] STMicroelectronics, Discovery Kit with STM32F407VG MCU (UM1472 User Manual), Rev. 8, Geneva, Switzerland: STMicroelectronics, 2025.
- [26] Texas Instruments, SN65HVD23x 3.3-V CAN Bus Transceivers, Datasheet, Rev. O, Apr. 2018.
- [27] STMicroelectronics, LIS331HH MEMS Motion Sensor High-g 3-Axis Digital Accelerometer, Datasheet.
- [28] JGA25-370 Geared Motor Datasheet,
<https://www.alldatasheet.com/datasheet-pdf/pdf/1567334/ETC2/JGA25-370.html>.